

派聪明 · PaiSmart · 教程系列

派聪明 RAG 后端演进全解

从空 Maven 项目到企业级 AI 知识库系统
—— 20 集长文稿，配套视频脚本

JPA · Spring Security · MinIO · Tika · Embedding ·
Elasticsearch · Kafka · WebSocket · ReAct · 多 LLM · 微信支付
· 可观测性

作者：Charles

编纂：Charles · Claude

2026 年 6 月 2 日

目 录

派聪明 RAG 后端演进全解 · 20 集全本 | Charles | 2026 年 6 月 2 日

第 01 集	第 01 集：派聪明 RAG 系统开篇——从一个 Spring Boot 骨架开始	
第 02 集	第 02 集：第一个 JPA 实体——User 与 Repository	
第 03 集	第 03 集：JWT 鉴权登场——从「裸奔」到「有身份的请求」	
第 04 集	第 04 集：Spring Security 串起来——登录、注册、权限规则	
第 05 集	第 05 集：MinIO 登场——文件存储从本地到对象存储	
	· 【五、MinIO 怎么访问？Presigned URL 模式**	
第 06 集	第 06 集：文档解析 Pipeline——Tika 与文本分块	
第 07 集	第 07 集：EmbeddingClient 登场——把文本变成向量	
第 08 集	第 08 集：Elasticsearch 集成——向量索引与召回	
第 09 集	第 09 集：Kafka 登场——把同步变异步	
第 10 集	第 10 集：WebSocket 流式聊天——DeepSeek 首次对话	
第 11 集	第 11 集：多租户隔离——OrgTag 体系	
第 12 集	第 12 集：混合检索——BM25 + 向量 + Rerank	
第 13 集	第 13 集：限流与配额——Redis 守护神	
	· 【五、`reserveLlmUsage`：LLM token 预算**	
第 14 集	第 14 集：对话会话与历史——RAG 系统的记忆	
第 15 集	第 15 集：ReAct 与 Agent 工具——LLM 装上手和脚	
第 16 集	第 16 集：多 LLM Provider 路由——不绑死单一厂商	
第 17 集	第 17 集：充值与计费——商业化闭环	
	· 【一、计费模型：按 token 还是按次？**	

第 18 集

第 18 集：邀请码与运营能力——SaaS 拉新核武器

.....

第 19 集

第 19 集：消息反馈与质量闭环——RAG 系统的耳朵

.....

· 【四、Agent 工具：`submit\_feedback`**

第 20 集

第 20 集：可观测性与生产化——最后一公里

.....

· 【一、为什么可观测性是"最后一公里"？**

第 01 集：派聪明 RAG 系统开篇——从一个 Spring Boot 骨架开始

系列：派聪明 RAG 后端演进全解

集数：01 / 20+

主题：项目起点、为什么是 Spring Boot 3、Maven 结构、最小启动类

适合人群：Java 后端、想理解 RAG 系统全貌的工程师

【开场 Hook】

很多同学问我：「我想自己写一个 RAG 系统，第一步应该干什么？」

我反问他们：「你能不能先把它跑起来？哪怕只是返回一个 Hello World？」

这一步，看似无用，实际上它决定了 **90%** 的项目能不能活到第二集。

我们这一季，会用 20 多期的长视频，逐集把 `pai-smart`（派聪明）这个企业级 RAG 知识库系统，从最初的空 **Maven** 项目开始，一期一期加需求、加模块、加设计，一直演进到它今天的样子——一个跑着 Kafka、Elasticsearch、MinIO、DeepSeek、WebSocket、微信支付的复杂分布式系统。

我们要做的不是「事后看代码讲解」，而是站在原作者的肩膀上，看他当年踩了哪些坑、做了哪些妥协、又怎么重构。

今天第一期，我们就从最最基础的那一步开始：**Spring Boot** 项目骨架。

【一、为什么要从「空项目」开始？】

很多教程一上来就贴一段 `@RestController`，告诉你「看，这就是 API」。但真实项目不是这么长起来的。

真实项目的演化路径大致是这样的：

1. 验证期：Demo 跑通，先用最简的方式把流程串起来。
2. 单兵期：1-2 个人开发，所有东西塞一个工程里，无所谓分层。
3. 团队期：3-5 个人开始有规范，分包、抽象、基础组件。
4. 生产期：要考虑安全、可观测、可扩展、可运维。

`pai-smart` 这个项目在 `GitCode` 上是开源的（仓库地址：<https://gitcode.com/javabetter/PaiSmart>），名字取自「派（ π ）」+「Smart」，定位是企业级 AI 知识管理。它不是一开始就是企业级的——看 git log，最早的 commit 也只是一个 Spring Boot 初始化模板。

我们今天就回到那个最朴素的版本。

【二、看一眼最终的 `pom.xml`】

虽然现在项目已经演进了很多代，但是最原始的依赖几乎都没变。我把 `pom.xml` 拆开讲：

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.4.2</version>
</parent>

<properties>
  <java.version>17</java.version>
  <lombok.version>1.18.30</lombok.version>
</properties>
```

两个关键点：

1. **Spring Boot 3.4.2**：这个版本号不是随便选的。Spring Boot 3.x 是基于 **Jakarta EE 9+** 的，包名从 `javax.*` 全部迁移到了 `jakarta.*`。这是 2022 年底到 2023 年的分水岭。如果你还在用 Spring Boot 2.7.x，很多新库的 API 不兼容。
2. **Java 17**：LTS 版本，`var` 关键字、`sealed class`、`record`、`switch` 模式匹配全部能用。后面我们写 DTO 的时候会大量用到 `record`。

再看核心 starter：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
```

```

    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>

```

这 6 个 starter，已经暗示了后面 20 集我们要解决的问题：

Starter	暗示的子系统	对应后续集数
data-jpa	用户/文件/会话的元数据存储	第 02 集
data-redis	限流、缓存、分布式会话	第 13 集
security	JWT 鉴权、组织标签授权	第 03、11 集
web	传统的 REST 控制器	第 02 集
websocket	流式聊天推送	第 10 集
webflux	DeepSeek 流式响应	第 10、16 集

看出规律了吗？一个项目的依赖文件，就是它的架构蓝图。面试的时候，让你「讲一下你们项目用了什么技术栈」，最简单的方法就是读一遍 `pom.xml` 然后按图索骥。

后面还会加进来：

- `elasticsearch-java` 8.10.0 → 第 08 集
- `minio` 8.5.12 → 第 05 集
- `spring-kafka` 3.2.1 → 第 09 集
- `hanlp` portable-1.8.6 → 第 06 集
- `tika-parsers-standard-package` 2.9.1 → 第 06 集
- `wechatpay-java` 0.2.17 → 第 17 集

每一个依赖都是一个被解决的痛点，不是技术炫技。我们后面会一期一期地讲。

【三、最小启动类：一行注解的事】

最初的启动类简单到让人想笑：

```

package com.yizhaoqi.smartpai;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

```

```

@SpringBootApplication
public class SmartPaiApplication {

    public static void main(String[] args) {
        SpringApplication.run(SmartPaiApplication.class, args);
    }
}

```

就这？对，就这。

但是要真的讲明白这里面的道道，我可以讲半小时。

3.1 @SpringBootApplication 到底是什么？

它是三个注解的组合：

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = {
    @Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
    @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class)
})
public @interface SpringBootApplication {
    // ...
}

```

面试考点 1：@SpringBootApplication = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan。

面试考点 2：默认的 @ComponentScan 不会扫描其他包。所以 SmartPaiApplication 必须放在根包 com.yizhaoqi.smartpai 下，否则 controller、service 包里的 bean 全部不会被发现。

这是非常常见的坑，项目跑起来报 404 找不到 bean，多半是包结构不对。

3.2 SpringApplication.run 内部做了什么？

大概 7 步：

1. 推断应用类型（Servlet 还是 Reactive）。
2. 加载并初始化 ApplicationContextInitializer。
3. 加载并初始化 ApplicationListener。
4. 推断并设置 main 方法所在的类（这决定了 @SpringBootApplication 的扫描起点）。
5. 加载 spring.factories / AutoConfiguration.imports 文件，做自动装配。

6. 创建 `ApplicationContext`，实例化所有单例 bean。

7. 启动内嵌的 Tomcat / Netty。

这一步的 5 是关键。`pai-smart` 后面会引入：

- `RedisConfig`、`SecurityConfig`、`MinioConfig` 这些 `@Configuration` 类，
- 还有 Spring Boot 自动装配的 `RedisAutoConfiguration`、`SecurityAutoConfiguration` 等等。

它们都通过第 5 步被加载。这就是 Spring Boot「约定大于配置」的精髓。

【四、为什么包结构是 `com.yizhaoqi.smartpai`？】

看现在的源码目录：

```
src/main/java/com/yizhaoqi/smartpai/
├── SmartPaiApplication.java
├── client/           # 外部 API 客户端 (DeepSeek、Embedding)
├── config/           # 配置类
├── consumer/         # Kafka 消费者
├── controller/       # REST 控制器
├── entity/           # JPA 实体
├── exception/        # 自定义异常
├── handler/          # WebSocket 处理器
├── model/            # 领域模型
├── repository/       # 数据访问层
├── service/          # 业务逻辑
├── test/             # 测试代码
└── utils/           # 工具类
```

这就是典型的 **Spring Boot** 三层架构 + 配置/横切层 的布局：

- `controller`：入口
- `service`：业务
- `repository`：持久化
- `entity` / `model`：数据模型
- `client` / `config` / `consumer` / `handler` / `exception` / `utils`：横切关注点

面试考点 3：为什么 `entity` 和 `model` 拆成两个？

这是项目里很经典的一个区分：

- `entity/`：JPA `@Entity`，直接和数据库表对应，带 `@Id` 字段、生命周期回调。
- `model/`：纯领域模型（POJO），不依赖持久化框架，是 `service` / `controller` 之间流转的对象。

但是！说实话，这种区分很多时候是过度设计的产物。如果项目不大、团队人少，合并成一个包完全没问题。我们这个项目走到现在，区分了，那么就要遵守——别一会儿把 `DTO` 塞 `entity`，一会儿塞 `model`。

【五、运行起来：Maven 命令】

```
# 第一次跑
mvn spring-boot:run

# 打包
mvn clean package

# 跑测试
mvn test
```

Maven Wrapper (mvnw) 的好处是不用全局装 Maven，新机器 clone 下来直接 `./mvnw spring-boot:run` 就行。但是看项目里没用 mvnw，估计是为了和 IDEA 配合好——这点因团队而异。

【六、这一期就这些了？**】

是的。第一期故意只讲这么多。

为什么？

因为很多同学学到后面越来越懵，根本原因就是跳过了这种「看起来没用」的铺垫。当你看到 `ChatHandler` 里有 10 个 `@Autowired` 依赖的时候，你不知道它们从哪来；当你看到 `OrgTagAuthorizationFilter` 的时候，你不知道它怎么被加载的。

一切答案都在 `@SpringBootApplication` 的扫描 + 自动装配 + `application.yml` 的配置绑定。

【七、常见坑 & 面试问答】

Q1：为什么启动类要放在根包下？

A：因为 `@SpringBootApplication` 默认 `@ComponentScan` 的 `basePackage` 就是启动类所在的包。子包里的 `@Component`、`@Service`、`@Controller`、`@Repository` 才会被注册成 bean。

Q2：`@SpringBootApplication` 的 `exclude` 怎么用？

A：比如项目里有 Redis 但是本地没装，可以 `exclude = {RedisAutoConfiguration.class}`，但更好的做法是用 profile。

Q3：Spring Boot 2.x 升 3.x 最大的坑是什么？

A：javax → jakarta。 `javax.servlet.*` 全部要改成 `jakarta.servlet.*`。很多老库没升级就会报错。

Q4：你们的 `pom.xml` 一上来就这么多依赖，会不会启动很慢？

A：会。 `spring-boot-starter-web` + `data-jpa` + `data-redis` + `security` + `webflux` + `websocket` 一起，冷启动大约 5-8 秒。生产环境一般用 `spring-boot-maven-plugin` 打成 fat jar + `SpringApplication` 延迟初始化（`spring.main.lazy-initialization=true`）来缓解。

【八、下集预告】

第二期，我们要真正开始写业务代码了。

我们要做的第一件事是——加一个 **User** 实体。

- `User.java` 怎么设计字段？
- `UserRepository.java` 怎么继承 `JpaRepository`？
- 为什么密码要单独放 `utils/PasswordUtil` 加密？
- 数据库连接配置写在哪儿？

带着这些问题，我们下期见。

如果你觉得这种「演化式」讲解比直接贴代码更好理解，点赞、投币、一键三连。
评论区告诉我你最想看哪一集的深度展开，我优先讲。

下一集链接：[第 02 集：第一个 JPA 实体——User 与 Repository](#)

第 02 集：第一个 JPA 实体——User 与 Repository

系列：派聪明 RAG 后端演进全解

集数：02 / 20+

主题：Spring Data JPA、Hibernate 自动建表、@Entity 设计哲学、User 模型演进

上集回顾：第 01 集我们搭好了 Spring Boot 3.4.2 骨架，`pom.xml` 里有 `spring-boot-starter-data-jpa` 这个依赖

本集目标：让 `User` 这个实体落库，能通过 `UserRepository` 查出来

【开场 Hook】

上集我们讲了，`pom.xml` 里 `data-jpa` 这个依赖暗示了接下来的子系统。今天我们要把它点亮。

很多同学学 JPA，上来就被一堆注解劝退：`@Entity`、`@Table`、`@Id`、`@GeneratedValue`、`@Column`、`@Enumerated`、`@CreationTimestamp`、`@UpdateTimestamp`

我告诉你，在 `pai-smart` 这个项目里，`User` 实体把这堆注解全用上了。它是后面所有实体（`FileUpload`、`Conversation`、`OrganizationTag`）的模板。看懂这一个，后面就是复制粘贴。

我们今天就逐行拆解：

```
@Data
@Entity
@Table(name = "users", uniqueConstraints = @UniqueConstraint(columnNames = "username"))
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role;

    @Column(name = "org_tags")
    private String orgTags; // 用户所属组织标签，多个用逗号分隔
```

```

@Column(name = "primary_org")
private String primaryOrg; // 用户主组织标签

@CreationTimestamp
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt;

public enum Role {
    USER, ADMIN
}
}

```

总共不到 50 行。别小看它，背后的设计决策一个比一个有讲究。

【一、@Data 是 Lombok，不是 Spring】

第一个坑：很多同学看到 @Data 就以为是 Spring 的注解。不是。它是 Lombok 提供的。

Lombok 通过 **annotation processor** 在编译期生成代码：

- @Data ≈ @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor

也就是说，编译后的字节码里，User 类会有 getId()、setId()、getUsername() 等方法——但你的源码里一行都没写。

面试考点 1：Lombok 的原理是什么？

答案：Java 的 JSR-269 Pluggable Annotation Processing API。Lombok 注册一个 annotation processor，在 .java 编译成 .class 之前修改抽象语法树（AST），加上 getter/setter。所以 Lombok 不会出现在运行时——这点对性能敏感的场景（比如 Spring Native Image）是个坑。

pom.xml 里要加：

```

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

```

并且在 maven-compiler-plugin 的 annotationProcessorPaths 里配置——pai-smart 已经配置好了，版本 1.18.30。

【二、@Entity 和 @Table 拆开看】

```
@Entity
@Table(name = "users", uniqueConstraints = @UniqueConstraint(columnNames = "username"))
public class User { ... }
```

为什么要拆成两个？

- **@Entity**：告诉 JPA「这是一个持久化实体」，JPA 会为它生成代理类（用于懒加载），并且把它纳入持久化上下文（Persistence Context）。
- **@Table**：告诉 Hibernate 数据库表长什么样。表名、唯一约束、索引，全在这里。

如果只写 **@Entity**，不写 **@Table**，行不行？

行。JPA 会默认用类名当表名。也就是说，你的表就叫 **User**（注意：大小写由数据库决定，MySQL 默认是 **user**）。但是 **user** 在 MySQL 里是保留字——**SELECT * FROM user WHERE ...** 会报错。

所以作者老老实实写了 **@Table(name = "users")**，避开这个雷。

uniqueConstraints 干啥的？

声明 **username** 列有唯一约束。JPA 在生成 DDL（**ddl-auto=update**）时会自动加 **UNIQUE** 索引。生产环境一般不用 Hibernate 自动建表，但开发期非常方便。

面试考点 2：**ddl-auto** 的几个值？

- **none**：什么都不做。
- **validate**：只校验实体和表结构是否一致，不一致启动报错。
- **update**：缺啥补啥（会加列，但不会删列、不会改类型）。
- **create**：启动时删表重建。
- **create-drop**：关闭应用时删表。

生产环境必须用 **validate** 或 **none**，否则 Hibernate 一个 update 把你线上数据改了都不知道。

【三、@Id 和 @GeneratedValue 的策略问题】

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

- **@Id**：主键。
- **@GeneratedValue(strategy = GenerationType.IDENTITY)**：用数据库自增 **ID**。

有 4 种策略：

策略	含义	优点	缺点
AUTO	JPA 自己挑	省事	不可控
IDENTITY	数据库自增 (AUTO_INCREMENT)	简单、性能好	批量插入时不预编译，无法用 JDBC batch
SEQUENCE	数据库序列 (Oracle/ PostgreSQL)	高性能、预编译	MySQL 不支持
TABLE	用一张表模拟序列	跨数据库	慢

`pai-smart` 选 `IDENTITY` 是合理的——MySQL 单库场景，简单粗暴。

但是！看 `DocumentVector` 实体（向量表）：

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long vectorId;
```

大量插入向量数据时，`IDENTITY` 会成为性能瓶颈。这是后面讲 Elasticsearch 那集会聊到的优化点。

面试考点 3：为什么 MySQL 用 `IDENTITY` 不适合大批量插入？

答：`IDENTITY` 必须立刻拿到数据库返回的自增 ID 才能继续，每次插入都要等数据库反馈，无法攒一批再发送。`SEQUENCE` 是「先问数据库要 100 个 ID 自己用」，所以可以攒批。`pai-smart` 这里为了简单牺牲了性能，业务量大了以后要优化。

【四、@Column 注解里的「坑」】

```
@Column(nullable = false, unique = true)
private String username;

@Column(nullable = false)
private String password;
```

- `nullable = false`：Hibernate 生成的 DDL 会加 `NOT NULL` 约束。
- `unique = true`：DDL 会加 `UNIQUE` 约束。

注意：这两个约束只对自动建表有效！如果你是用 `Flyway` / `Liquibase` 管理的 DDL，要自己在 SQL 里写。

为什么 `password` 不加 `unique`？因为理论上密码可以重复（虽然不应该）。

为什么 `orgTags` 和 `primaryOrg` 字段名和列名不一致？

```

@Column(name = "org_tags")
private String orgTags;

@Column(name = "primary_org")
private String primaryOrg;

```

这是 Java 命名（驼峰）和 SQL 命名（下划线）的 约定问题。Hibernate 5+ 默认有 `PhysicalNamingStrategyStandardImpl` 会自动转换，但 `pai-smart` 显式指定了 `name` —— 稳健做法。

面试考点 4：orgTags 为什么是 String，不是 List<String>？

答：作者偷懒了。orgTags 字段存的是「多个组织标签用逗号分隔」。这种设计叫「弱设计」：

- 优点：简单、好查（LIKE '%xxx%'）。
- 缺点：更新一个标签要 REPLACE，关联查询要 LIKE，性能差；字段长度不可控（万一标签很多就超长了）。

更好的设计应该是多对多关联表 `user_org_tag(user_id, org_tag)`。但是看项目里后来用 `OrgTagCacheService` 在内存里维护映射关系，用空间换时间——这个设计是后话，第 11 集讲。

【五、@Enumerated(EnumType.STRING) —— 字符串存枚举】

```

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private Role role;

```

为什么是 STRING 不是 ORDINAL？

- ORDINAL：存枚举的下标（0、1、2……）。USER = 0，ADMIN = 1。
- STRING：存枚举的名字（"USER"、"ADMIN"）。

如果用 ORDINAL：

```

public enum Role {
    USER, ADMIN, GUEST // 新加 GUEST 在中间，USER=0，GUEST=1，ADMIN=2
}

```

ADMIN 从 1 变成 2，老数据全错！

pai-smart 用 STRING 是正确选择。

面试考点 5：枚举的最佳实践？

- 一定要用 STRING。
- 一定要 nullable = false（如果有默认值）。

- 业务上「禁用」枚举值不要直接删，加一个 `DELETED` 或 `ARCHIVED` 状态。软删除比硬删除安全。

【六、`@CreationTimestamp` 和 `@UpdateTimestamp`】

```
@CreationTimestamp
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt;
```

这是 Hibernate 的注解（不是 Spring Data 的）。它的作用：

- 实体第一次保存时，自动把 `createdAt` 设为当前时间。
- 每次实体被更新时，自动把 `updatedAt` 设为当前时间。

注意：它依赖 Hibernate 的 dirty checking。直接 `entity.setXxx()`，事务提交时 **Hibernate** 会自动 **UPDATE**——`updatedAt` 也会自动变。

面试考点 6：为什么用 Hibernate 注解，不用 MySQL 的 `DEFAULT CURRENT_TIMESTAMP`？

答：跨数据库可移植性。MySQL 有，PostgreSQL 有，Oracle 用法不一样。写在 **Java** 侧，应用启动时在哪都能跑。同时应用侧维护时间戳便于在业务代码里直接读取。

【七、`UserRepository`——为什么是接口不是类？】

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

这是 **Spring Data JPA** 最「魔法」的地方。

你写的是一个接口。Spring Data JPA 在启动时自动生成实现类（代理），叫 `SimpleJpaRepository`。

代理生成的逻辑：

- `JpaRepository<User, Long>` 自带 `findAll()`、`findById()`、`save()`、`deleteById()` 等等。
- `findByUsername(String username)` 没有现成实现，**Spring Data** 看方法名解析：前缀 `findBy` + 属性名 `username` + 参数 `String username`，自动生成 `SELECT * FROM users WHERE username = ?` 的 JPQL。

这就是 **Spring Data** 的「方法名查询」。

面试考点 7：方法名查询的解析规则？

- `findBy`、`readBy`、`queryBy`、`getBy`：SELECT。

- `countBy` : COUNT。
- `deleteBy`、`removeBy` : DELETE。
- `existsBy` : EXISTS。
- 关键字 : `And`、`Or`、`Between`、`LessThan`、`GreaterThan`、`Like`、`In`、`OrderBy`、`Top`、`Distinct`

举例：

```
List<User> findTop10ByCreatedAtAfterOrderByIdDesc(LocalDateTime time);
List<User> findByOrgTagsContainingAndRole(String orgTag, Role role);
long countByRole(Role role);
```

不用写实现。

但是！注意这个细节：

```
Optional<User> findByUsername(String username);
```

返回 `Optional<User>` 而不是 `User`。这是 **Java 8** 之后的最佳实践——`Optional` 强制调用方处理「找不到」的情况。

面试考点 8：`Optional` 有什么坑？

- 不能作为字段类型（序列化、性能）。
- 不能作为方法参数（应该用 `Nullable`）。
- `orElse(null)` 和 `orElseGet(() -> null)` 在「值不存在」时没区别，但 `orElseGet` 里的 lambda 只在值不存在时才执行——性能场景要注意。

【八、什么时候加 `@Transactional`？】

`UserRepository` 的方法没有 `@Transactional`。这是 Spring Data 的设计——只读方法自动加只读事务，写方法自动加读写事务。

`@Transactional` 的位置原则：

- **Service** 层加，**Repository** 层不加（Spring Data 帮你加）。
- 类级别 vs 方法级别：类级别粒度粗，方法级别粒度细。
- `@Transactional` 失效的常见坑：
 - 同类的内部调用（`this.foo()`，绕过 Spring 代理）。
 - 异常被 **catch** 掉了，没抛出（Spring 默认只对 `RuntimeException` 回滚）。
 - 方法不是 **public**。

第 17 集讲充值系统的时候，我们会反复用 `@Transactional`，会重点讲。

【九、AuthController —— record 登场】

```
@PostMapping("/refreshToken")
public ResponseEntity<?> refreshToken(@RequestBody RefreshTokenRequest request) { ... }

record RefreshTokenRequest(String refreshToken) {}
```

`RefreshTokenRequest` 用 Java 14+ 的 `record` 语法。一行就把 **DTO** 写完了。

`record` 的本质：

- 编译器自动生成 `getter`（注意是 `refreshToken()` 不是 `getRefreshToken()`）、`equals`、`hashCode`、`toString`。
- 字段默认是 `final` ——不可变对象。
- 适合做 DTO、值对象。

面试考点 9：什么场景用 `record`，什么场景用 `class`？

- `record`：DTO、值对象、Query、Command。没有继承、没有可变状态。
- `class`：实体（要支持 Hibernate 懒加载的代理）、Service、Controller（要加 Spring 注解的）。

【十、思考题：为什么 AuthController 单独放一个包？】

看包名是 `controller/AuthController.java`，但它只做 **token** 刷新。登录、注册在 `UserController`。

这是项目演化的痕迹——可能一开始登录/注册都在 `AuthController`，后来抽到 `UserController`，但 token 刷新这个相对独立的逻辑没搬走。

没有银弹的架构：项目早期怎么顺手怎么来，等业务复杂了再做合理的边界划分。这是 KISS + YAGNI 原则的体现。

【十一、常见坑 & 面试问答】

Q1：@Data 和 @Getter @Setter 怎么选？

A：实体类用 `@Data` 方便；DTO 不可变用 `record`；如果你想做 `equals` 排除某个字段（懒加载的关联），用 `@Getter @Setter @ToString(exclude = "xxx")` 显式控制。

Q2：JPA 的 N+1 查询问题遇到过吗？

A: `@OneToMany`、`@ManyToOne` 默认懒加载。查询 1 个用户触发 1 次 SQL，访问他的订单再触发 N 次。用 `JOIN FETCH` 一次性查出来：

```
@Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
User findWithOrders(@Param("id") Long id);
```

`pai-smart` 里 `FileUpload` 和 `DocumentVector` 的关系，第 05、08 集会详细讲。

Q3：MySQL 的 `users` 表名是复数还是单数？

A：看团队规范。`pai-smart` 用复数（`users`、`organizations`、`files`）。我推荐复数——表是「集合」的概念，描述一类实体。

Q4：为什么 `User` 字段都用包装类型 `Long` 不是基本类型 `long`？

A：因为包装类型可以为 `null`。新建一个 `User` 对象没设 ID 时，`long` 默认是 0，会污染业务（用户 ID = 0 不合法）；`Long` 默认是 `null`，更安全。

【十二、练习题（建议动手做）**

1. 给 `User` 加一个 `email` 字段，类型 `String`，要求非空、唯一。
2. 在 `UserRepository` 加一个方法 `findByEmail(String email)`，返回 `Optional<User>`。
3. 写一个测试 `UserRepositoryTest`，插入 3 个用户，验证 `findByUsername` 找得到。

【十三、下集预告】

第 02 集我们有了 `User` 实体，能存能查了。

但是！现在任何人都能调任何接口，没有任何鉴权。

第 03 集，我们要：

- 引入 `JwtUtils`，手写 JWT 的签发和校验。
- 写 `JwtAuthenticationFilter`，拦截所有 HTTP 请求。
- 密码用 `PasswordUtil` 加密存储。

JWT 是现在 80% 的 Java 后端面试必考点。我们下期深入。

评论区有同学问：「为什么不用 MyBatis-Plus？」——好问题，我们第 14 集对比 JPA 和 MyBatis 时会专门讲。

想看哪一集的深度展开？留言告诉我。

下一集链接：[第 03 集：JWT 鉴权登场——从「裸奔」到「有身份的请求」](#)

第 03 集：JWT 鉴权登场——从「裸奔」到「有身份」的请求」

系列：派聪明 RAG 后端演进全解

集数：03 / 20+

主题：JWT 原理、jjwt 库、JwtUtils 实现细节、OncePerRequestFilter、Token 自动刷新

上集回顾：第 02 集我们写了 `User` 实体和 `UserRepository`，能存能查

本集目标：所有 HTTP 请求都带身份，Token 还能在过期前自动续命

【开场 Hook】

你写了一个 `UserController`，前端请求过来，返回数据。

但是！任何一个人，只要知道你的 URL，就能拿到所有用户的数据。你的系统现在「裸奔」。

在 `pai-smart` 这种企业级项目里，没有鉴权 = 没有产品。

今天我们要做的就是：让每一个请求都「有身份」。我们用的方案不是老旧的 `Session+Cookie`，而是 **JWT (JSON Web Token)** ——这是 2015 年以后 Java 后端的事实标准。

很多人学 JWT 是看官方文档、或者博客，看完还是似懂非懂。今天我们直接看一个生产级的 JWT 实现，把每一个细节都掰开。

【一、JWT 是什么？三段式结构】

JWT 是一段字符串，分三段，用 `.` 分隔：

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJjaGFyYGVzIn0.abc123signature
```

第一段 **Header**（算法说明）：

```
{ "alg": "HS256" }
```

第二段 **Payload**（业务数据，也叫 Claims）：

```
{
  "sub": "charles",
  "exp": 1717228800,
  "userId": "1",
  "role": "USER",
  "orgTags": "DEFAULT,DEV",
  "primaryOrg": "DEV"
}
```

第三段 **Signature** (签名) :

```
HMACSHA256(base64Url(header) + "." + base64Url(payload), secret)
```

面试考点 1 : JWT 的 Header 和 Payload 都是 **Base64** 编码 , 不是加密 ! 任何人都能解码看到内容。所以你绝对不能在 JWT 里放密码、身份证号等敏感信息。**JWT** 的安全性来自签名 , 不来自加密。

面试考点 2 : JWT 和 Session 的区别 ?

维度	Session	JWT
存储	服务端内存/Redis	客户端 (Authorization Header)
状态	有状态	无状态
扩展性	集群需要 Session 共享	天然分布式
撤销	服务端删除 Session	需要黑名单机制
性能	服务端查 Redis	验签即可 (无状态)

pai-smart 选 JWT , 是因为后面会接 **WebSocket** (第 10 集) 。 WebSocket 协议里 Session 机制不友好 , 用 JWT 客户端带 token 直接握手最自然。

【二、JwtUtils 核心实现 : 逐段拆解】

我们看 **JwtUtils.generateToken** :

```
public String generateToken(String username) {
    SecretKey key = getSigningKey();

    User user = userRepository.findByUsername(username)
        .orElseThrow(() -> new RuntimeException("User not found"));

    String tokenId = generateTokenId();
    long expireTime = System.currentTimeMillis() + EXPIRATION_TIME;
```

```

Map<String, Object> claims = new HashMap<>();
claims.put("tokenId", tokenId);
claims.put("role", user.getRole().name());
claims.put("userId", user.getId().toString());

if (user.getOrgTags() != null && !user.getOrgTags().isEmpty()) {
    claims.put("orgTags", user.getOrgTags());
}
if (user.getPrimaryOrg() != null && !user.getPrimaryOrg().isEmpty()) {
    claims.put("primaryOrg", user.getPrimaryOrg());
}

String token = Jwts.builder()
    .setClaims(claims)
    .setSubject(username)
    .setExpiration(new Date(expireTime))
    .signWith(key, SignatureAlgorithm.HS256)
    .compact();

tokenCacheService.cacheToken(tokenId, user.getId().toString(), username, expireTime);
return token;
}

```

2.1 密钥派生：getSigningKey()

```

@Value("${jwt.secret-key}")
private String secretKeyBase64; // Base64 编码后的密钥

private SecretKey getSigningKey() {
    byte[] keyBytes = Base64.getDecoder().decode(secretKeyBase64);
    return Keys.hmacShaKeyFor(keyBytes);
}

```

这个设计的讲究：

- **@Value** 注入密钥：从配置文件 `application.yml` 里读，避免硬编码。生产环境会通过环境变量覆盖。
- **Base64** 编码存储：因为密钥是任意字节，直接写在 YAML 里会有编码问题。Base64 一次转换更安全（也兼容非 ASCII 字符）。
- **HS256** 算法的密钥长度：至少 **256 bit = 32 字节**。`hmacShaKeyFor` 会自动检查密钥长度，不够就抛异常。

面试考点 3：HS256 和 RS256 怎么选？

- **HS256 (HMAC + SHA256)**：对称加密，签发和验证用同一个密钥。性能快。适合单服务或可信服务集群内部。
- **RS256 (RSA + SHA256)**：非对称加密，签发用私钥，验证用公钥。性能慢。适合第三方应用接入（OAuth 2.0、OpenID Connect）。

`pai-smart` 选 HS256 合理——内部系统，没必要上 RS256。

2.2 Claims 设计：放什么不放什么？

```
claims.put("tokenId", tokenId); // 服务端缓存 key
claims.put("role", user.getRole().name());
claims.put("userId", user.getId().toString());
claims.put("orgTags", user.getOrgTags());
claims.put("primaryOrg", user.getPrimaryOrg());
```

关键设计：

1. `tokenId` 是关键：后面讲到「Token 注销」和「自动刷新」要靠它。没有 `tokenId`，你做不到登出——因为 JWT 一旦签发就管不了了。
2. `userId` 单独存：第 10 集 WebSocket 聊天时，业务逻辑要拿 `userId`（不是 `username`）去查库。
3. `orgTags` 和 `primaryOrg`：第 11 集多租户体系全靠这个。注意是 **String**，不是 **List**——这是项目早期的妥协（后面会优化）。
4. `username` 没显式 `put`：因为 `setSubject(username)` 已经把 `username` 放进了 `sub` 字段。

面试考点 4：JWT Payload 应该放什么？

- 必须放：`sub`（用户标识）、`exp`（过期时间）、`iat`（签发时间）、`jti`（token ID）。
- 按需放：角色、权限 ID、组织、租户 ID。
- 绝对不放：密码、身份证、银行卡号、手机号、邮箱等敏感信息。
- 控制大小：JWT 每次请求都发一次。通常建议不超过 **4KB**。HTTP Header 太大影响性能。

2.3 签发 + 缓存的双写

```
String token = Jwts.builder()
    .setClaims(claims)
    .setSubject(username)
    .setExpiration(new Date(expireTime))
    .signWith(key, SignatureAlgorithm.HS256)
    .compact();

tokenCacheService.cacheToken(tokenId, user.getId().toString(), username, expireTime);
```

两个动作：

1. **JWT** 签发：客户端拿到这个 token 就可以用了。
2. **Redis** 缓存：服务端记录这个 token 是有效的。

为什么不只签发 **JWT** 就行？

因为 **JWT** 签发后就无法撤销！用户退出登录、改密码、被封号，**JWT** 还在有效期内就还能用。

所以用 Redis 维护一个 `valid_tokens` 集合：

- 登出：把 `tokenId` 从集合里删掉（或加入黑名单）。
- 校验：先查 Redis，再验签名。

这就是「双写」——`pai-smart` 的 JWT 实际上是有状态的 **JWT**，不是纯无状态。

面试考点 5：有状态 JWT vs 无状态 JWT 怎么选？

- 无状态 **JWT**：完全不存服务端。优点是极致性能。缺点是「签发即生效」，无法主动撤销。
- 有状态 **JWT**（`pai-smart` 这种）：服务端维护 token 状态。优点是可控。缺点是每请求多一次 Redis 查询。

折中方案：把 Token 有效期调短（比如 15 分钟），用 Refresh Token 续期。`pai-smart` 用的就是这个：Access Token 1 小时 + Refresh Token 7 天。

【三、Token 校验：`validateToken` 的双重验证】

```
public boolean validateToken(String token) {
    try {
        // 1. 先从 JWT 拿 tokenId
        String tokenId = extractTokenIdFromToken(token);
        if (tokenId == null) return false;

        // 2. 查 Redis
        if (!tokenCacheService.isTokenValid(tokenId)) return false;

        // 3. 验签名
        Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token);

        return true;
    } catch (ExpiredJwtException e) { ... }
    catch (SignatureException e) { ... }
    return false;
}
```

三个检查，缺一不可：

1. 解析 **tokenId**：拿不到 → 拒绝（说明 token 格式有问题或者不是本系统签发的）。
2. 查 **Redis**：tokenId 不在有效集合里 → 拒绝（说明被注销了）。
3. 验签名：签名不匹配 → 拒绝（说明被篡改）。

为什么 `parseClaimsJws` 会抛 `ExpiredJwtException`？

因为 jjwt 库里「过期」也是一种「验证失败」。`exp` 字段小于当前时间，jjwt 就抛异常。这比手动比对时间更安全——避免你自己写代码时漏掉某个分支。

面试考点 6：ExpiredJwtException 在 SignatureException 之前捕获，为什么？

答：因为「过期但签名正确」和「被篡改」是两种不同情况，处理方式不同：

- 过期：可以考虑自动刷新（项目里就这么干）。
- 被篡改：直接拒绝，记录日志，可能涉及攻击。

实战中要小心：如果你把 ExpiredJwtException 放在最外层 catch，真正的攻击（伪造签名）也会被当过期处理——这就有漏洞了。pai-smart 把它们分开 catch，是正确的。

【四、JwtAuthenticationFilter：拦截每个请求】

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain) {
        try {
            String token = extractToken(request);
            if (token != null) {
                String newToken = null;
                String username = null;

                if (jwtUtils.validateToken(token)) {
                    if (jwtUtils.shouldRefreshToken(token)) {
                        newToken = jwtUtils.refreshToken(token);
                    }
                    username = jwtUtils.extractUsernameFromToken(token);
                } else {
                    if (jwtUtils.canRefreshExpiredToken(token)) {
                        newToken = jwtUtils.refreshToken(token);
                        username = jwtUtils.extractUsernameFromToken(newToken);
                    }
                }

                if (newToken != null) {
                    response.setHeader("New-Token", newToken);
                }

                if (username != null && !username.isEmpty()) {
                    UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
                    UsernamePasswordAuthenticationToken authentication =
                        new UsernamePasswordAuthenticationToken(userDetails, null, userD
etails.getAuthorities());
                    authentication.setDetails(new WebAuthenticationDetailsSource().buildDeta
ils(request));
                    SecurityContextHolder.getContext().setAuthentication(authentication);
                }
            }
            filterChain.doFilter(request, response);
        }
    }
}
```

```

    } catch (Exception e) {
        logger.error("Cannot set user authentication: {}", e);
    }
}

private String extractToken(HttpServletRequest request) {
    String bearerToken = request.getHeader("Authorization");
    if (bearerToken != null && bearerToken.startsWith("Bearer ")) {
        return bearerToken.substring(7);
    }
    return null;
}
}

```

4.1 OncePerRequestFilter 是什么？

它是 Spring Web 提供的一个抽象类，保证每个请求只执行一次（防止内部 forward、include 时被多次调用）。

为什么不直接实现 `Filter` ？

实现 `Filter` 接口的话，`dispatcher` 内部 `forward` 的时候 同一个 filter 可能跑两次。
`OncePerRequestFilter` 内部用一个 `request` attribute 标记，保证只跑一次。

4.2 「无感知刷新」是怎么做到的？

这是 `pai-smart` 的亮点设计。

```

if (jwtUtils.validateToken(token)) {
    if (jwtUtils.shouldRefreshToken(token)) {
        newToken = jwtUtils.refreshToken(token);
    }
    username = jwtUtils.extractUsernameFromToken(token);
} else {
    if (jwtUtils.canRefreshExpiredToken(token)) {
        newToken = jwtUtils.refreshToken(token);
        username = jwtUtils.extractUsernameFromToken(newToken);
    }
}

if (newToken != null) {
    response.setHeader("New-Token", newToken);
}

```

两个阈值：

- `REFRESH_THRESHOLD = 5 * 60 * 1000`（5 分钟）：剩余时间不足 **5** 分钟，主动刷新。
- `REFRESH_WINDOW = 10 * 60 * 1000`（10 分钟）：过期不超过 **10** 分钟，还可以续命。

前端流程：

1. 用户登录 → 拿到 `token` 和 `refreshToken`。

2. 用户操作 → 请求头带 `Authorization: Bearer xxx`。
3. 服务端 filter 发现 token 快过期 → 重新签发 → 把新 token 放在响应头 `New-Token` 里。
4. 前端 axios/fetch 拦截器：每次响应检查有没有 `New-Token`，有就更新本地存储。

用户全程无感。

面试考点 7：为什么用「无感知刷新」而不是「401 触发刷新」？

- **401 触发刷新**：用户操作到一半 token 过期 → 请求 401 → 客户端用 refreshToken 调刷新接口 → 拿到新 token → 重发原请求。整个过程用户能感受到延迟，且复杂业务有竞态问题。
- **无感知刷新**：服务端提前续命。用户从开始到结束用同一个 **token**，体验丝滑。

pai-smart 选择两者结合：无感知刷新 + refresh token 兜底。

4.3 SecurityContextHolder 是什么鬼？

```
SecurityContextHolder.getContext().setAuthentication(authentication);
```

这是 **Spring Security** 的核心。

面试考点 8：`SecurityContextHolder` 的存储策略？

- 默认是 `ThreadLocal` 模式（`MODE_THREADLOCAL`）。
- 同一线程内的 `Controller`、`Service` 都能从 `SecurityContextHolder` 拿到当前登录用户。
- 异步场景（`@Async`）会丢失，要用 `MODE_INHERITABLETHREADLOCAL` 或手动传递。

pai-smart 在 `ChatHandler` 里是这么用的：

```
public void processMessage(String userId, String userMessage, WebSocketSession session) {  
    // userId 是从 WebSocket 握手时塞进 session 的，不是从 SecurityContextHolder  
    // 因为 WebSocket 不是普通的 HTTP 请求，SecurityContext 不会自动传播  
}
```

WebSocket 怎么鉴权是另一个大坑——第 10 集再讲。

【五、TokenCacheService：Redis 缓存的实现细节】

我们看 token 是怎么缓存的：

```
tokenCacheService.cacheToken(tokenId, user.getId().toString(), username, expireTime);
```

具体的 `TokenCacheService` 实现我们没看，但可以推测：

```
public void cacheToken(String tokenId, String userId, String username, long expireTime) {
    long ttl = expireTime - System.currentTimeMillis();
    String key = "auth:token:" + tokenId;
    redisTemplate.opsForValue().set(key, userId, ttl, TimeUnit.MILLISECONDS);
    // 同时维护一个 userId -> Set<tokenId> 的反向索引
    String userKey = "auth:user-tokens:" + userId;
    redisTemplate.opsForSet().add(userKey, tokenId);
    redisTemplate.expire(userKey, REFRESH_TOKEN_EXPIRATION_TIME, MILLISECONDS);
}
```

关键点：

- 正向索引：`tokenId -> userId`（校验时用）。
- 反向索引：`userId -> Set<tokenId>`（批量登出时用）。
- **TTL** 跟随 **token** 过期时间：Redis 自动清理，不用定时任务。

面试考点 9：为什么不用 `redis.setex` 而用 `redisTemplate.opsForValue().set(key, value, ttl)`？

答：在 Spring Data Redis 里，`opsForValue().set(key, value, timeout, unit)` 内部就是 SETEX 命令，等价。但这种 API 更面向对象，类型安全。

【六、Refresh Token：另一条战线】

```
public String generateRefreshToken(String username) {
    // ... 类似的签发逻辑
    claims.put("refreshTokenId", refreshTokenId);
    claims.put("userId", user.getId().toString());
    claims.put("type", "refresh");
    // ...
}
```

Refresh Token 故意和 **Access Token** 分开：

- **Access Token**：短（1 小时），每次请求都带，权限大。
- **Refresh Token**：长（7 天），只在刷新时用，权限小（只能调 `/auth/refresh`）。

这叫「最小权限原则」——万一 Refresh Token 泄露，攻击者也只能续期 Access Token，拿不到用户数据。

面试考点 10：Refresh Token 怎么存？

- 不能存 **localStorage**（XSS 攻击能拿到）。
- 应该存 **httpOnly cookie**（前端 JS 拿不到）。
- 或者存内存里，关闭浏览器就丢（最安全但用户体验差）。

`pai-smart` 的前端具体怎么存，我们看 `frontend/src` 再说。

【七、Token 撤销：登出怎么做？】

```
public void invalidateToken(String token) {
    String tokenId = extractTokenIdFromToken(token);
    if (tokenId != null) {
        Claims claims = extractClaimsIgnoreExpiration(token);
        if (claims != null) {
            long expireTime = claims.getExpiration().getTime();
            String userId = claims.get("userId", String.class);
            tokenCacheService.blacklistToken(tokenId, expireTime);
            tokenCacheService.removeToken(tokenId, userId);
        }
    }
}
```

两步：

1. **blacklistToken**：把 tokenId 加到黑名单（带 TTL，和原 **token** 过期时间一致——过期了黑名单自动清，节省 Redis 内存）。
2. **removeToken**：从正向索引里删除。

注意：**blacklist** 和 **remove** 都要做！否则会出现「正向索引没了但黑名单还在」的不一致。

批量登出：

```
public void invalidateAllUserTokens(String userId) {
    tokenCacheService.removeAllUserTokens(userId);
}
```

比如「修改密码」后，让该用户所有设备的 token 全部失效。这就是反向索引的价值。

【八、SecurityConfig：怎么把 filter 串进 Spring Security？】

虽然第 04 集会重点讲 **SecurityConfig**，但这里先剧透一句：

```
http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

addFilterBefore 把 **JwtAuthenticationFilter** 放在 **UsernamePasswordAuthenticationFilter** 之前。Spring Security 的过滤器链是有顺序的——**JWT** 校验必须在用户名密码校验之前，因为我们是「用 token 代替用户名密码」的认证方式。

【九、常见坑 & 面试问答】

Q1 : JWT 的 `exp` 设置多久合适？

A：业务决定。金融类 **5-15 分钟**（高安全），普通 **To C 1-2 小时**（平衡安全和体验），内部系统可以 **8 小时**（一天工时）。`pai-smart` 1 小时，配合 **5 分钟** 提前刷新，实际体验接近「永远在线」。

Q2 : JWT 里的 `userId` 能不能是数字 `Long`？

A：技术上是，但 `pai-smart` 用了 `String`——保持 **JWT** 自描述（`Long` 经过 `Jwts` 序列化会变样）。如果业务需要保持类型一致，可以 `claims.put("userId", user.getId())`，但调试时看 JWT 内容会困惑。

Q3 : JWT 怎么防重放攻击？

A：加 `jti`（JWT ID）+ 服务端校验 `jti` 一次性使用。`pai-smart` 的 `tokenId` 就是这个作用。更高安全级别可以加 `nbfi`（not before，签发后多久才能用）。

Q4 : JWT 在 `WebSocket` 怎么传？

A：握手时通过 `Sec-WebSocket-Protocol` 头、URL 参数、或者第一条消息带过来。`pai-smart` 是在握手时解析 Header——第 10 集详解。

Q5 : 双 `Token` 机制的缺点是什么？

A：复杂度高了。前端要管理两个 token、刷新逻辑、心跳检测。很多团队觉得不如直接用「长 Access Token + 黑名单」简单。`pai-smart` 选了双 Token，因为它有多端登录场景（手机、PC 同时登录），双 Token 让单端登出更优雅。

【十、思考题：为什么 `JwtAuthenticationFilter` 放在 `config/` 包？**

看代码：

```
package com.yizhaoqi.smartpai.config;
```

它不是 `util/`，也不是 `security/`，而是 `config/`。这是项目的一个不太规范的小细节——`Filter` 严格说不是配置类（`@Configuration`），它是「过滤器」。

更合理的位置应该是 `security/` 或 `filter/`。但 `pai-smart` 在演进过程中没有专门建包，很多东西都往 `config/` 里塞——这是 KISS 原则的体现，也是个未来重构的伏笔。

【十一、下集预告】

第 03 集我们有了 JWT filter，但还没串进 **Spring Security**。光写 filter 不挂到过滤链上，等于没写。

第 04 集我们要把：

- `SecurityConfig` 写完整。
- 登录、注册、登出接口。
- 密码加密 (`PasswordUtil` , BCrypt)。
- `UserDetailsService` 的实现。
- 路径级别的鉴权规则。

Spring Security 是 90% Java 后端面试都会问的——`pai-smart` 的实现值得好好学。

如果你正在面试，这一集的内容直接背下来能过一面。

觉得有用的话点赞、收藏、投币——你们的支持是我做 20 集的最大动力。

下一集链接：[第 04 集：Spring Security 串起来——登录、注册、权限规则](#)

第 04 集：Spring Security 串起来——登录、注册、权限规则

系列：派聪明 RAG 后端演进全解

集数：04 / 20+

主题：Spring Security 6 的 lambda DSL、BCrypt、UserDetailsService、路径级授权

上集回顾：第 03 集我们写了 JwtAuthenticationFilter，但还没挂到 Spring Security 链上

本集目标：把 SecurityConfig 配完整，密码用 BCrypt 加密，注册能跑通

【开场 Hook】

很多同学学 Spring Security，最大的障碍是「概念多」——AuthenticationManager、ProviderManager、UserDetailsService、PasswordEncoder、SecurityFilterChain、AccessDecisionManager

但你一旦在真实项目里用 lambda DSL 写过一遍 HttpSecurity，会发现：新版 Spring Security 6.x 比老版简单太多了。

pai-smart 用的是 Spring Boot 3.4.2，对应 Spring Security 6.x。本集我们就一行一行拆它的 SecurityConfig，然后扩展到登录、注册、密码加密。

【一、SecurityConfig.java 全文逐行解析】

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Autowired
    private OrgTagAuthorizationFilter orgTagAuthorizationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/", "/test.html", ...).permitAll()
                .requestMatchers("/chat/**", "/ws/**").permitAll()
                .requestMatchers("/api/v1/users/register", "/api/v1/users/login").permitAll()
```

```

)
    .requestMatchers("/api/v1/test/**").permitAll()
    .requestMatchers("/api/v1/upload/**", ...).hasAnyRole("USER", "ADMIN")
    .requestMatchers("/api/v1/users/conversation/**").hasAnyRole("USER",
"ADMIN")
    .requestMatchers("/api/search/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers("/api/v1/chat/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/users/primary-org").hasAnyRole("USER", "ADMIN")
    .anyRequest().authenticated()
    .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
    .addFilterAfter(orgTagAuthorizationFilter, JwtAuthenticationFilter.class);

return http.build();
}
}

```

1.1 @EnableWebSecurity 是什么？

Spring Security 5+ 之后，这个注解是必需的（Spring Boot Auto Configuration 默认会带上，但显式声明更稳）。它激活了 Spring Security 的 `@Configuration` 类和过滤器链支持。

1.2 整个文件就一个 @Bean : SecurityFilterChain

这是 Spring Security 6 的新写法。老版本是 `WebSecurityConfigurerAdapter` + 重写 `configure(HttpSecurity http)` 方法。**6.x** 全部改成 `@Bean` 形式——更灵活，可以有多个 `SecurityFilterChain`（比如 API 一套、管理后台一套）。

1.3 csrf(csrf -> csrf.disable()) ——为什么关？

CSRF（Cross-Site Request Forgery）只对「浏览器 + Session Cookie」这种场景有意义。我们的系统：

- 用 **JWT** 而不是 **Session**：每次请求带 `Authorization` 头，不像 **Cookie** 自动携带。
- 前后端分离：前端是 Vue SPA，不是表单提交。

所以 CSRF 攻击构不成威胁——直接关。

什么时候要开？

- 传统服务端渲染（Thymeleaf、JSP）。
- 用 Session + Cookie 鉴权的内部系统。

1.4 authorizeHttpRequests 的「白名单」+「黑名单」模型

```
.requestMatchers("/api/v1/users/register", "/api/v1/users/login").permitAll()  
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")  
.anyRequest().authenticated()
```

这是个「先 **permit**，再 **deny**」的链式匹配。第一次匹配上的规则生效，后面的不看了。

- **permitAll()**：任何人都能访问（不检查 token）。
- **hasRole("ADMIN")**：必须有 **ROLE_ADMIN** 这个角色。
- **hasAnyRole("USER", "ADMIN")**：USER 或 ADMIN 都行。
- **authenticated()**：只要登录了就行（任何角色）。
- **denyAll()**：谁都不行。
- **anyRequest()**：兜底。

面试考点 1：**.requestMatchers("/admin/**")** 和 **.requestMatchers("/admin/*")** 区别？

- **/**** 匹配多级目录。
- **/*** 只匹配一级。

/admin/user/list 匹配 **/admin/****，不匹配 **/admin/***。

1.5 顺序敏感！

```
.requestMatchers("/api/v1/users/primary-org").hasAnyRole("USER", "ADMIN")  
.anyRequest().authenticated()
```

这两条规则，位置写反就出问题。

如果 **anyRequest().authenticated()** 在前，所有请求（包括 **permitAll** 的）都会被检查有没有登录。Spring Security 会按顺序匹配，第一次匹配上就停——所以 **permitAll** 必须在 **anyRequest().authenticated()** 之前。

实战中常见的 **BUG**：加了新接口忘了加 **permitAll**，被 **anyRequest().authenticated()** 兜底，未登录用户调不通。**pai-smart** 写得很细——每个公开接口都明确 **permitAll()**，不依赖默认行为。

1.6 sessionManagement 为什么设成 STATELESS？

```
.sessionManagement(session -> session  
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

四种 **Session** 策略：

策略	含义
STATELESS	不创建 Session，每个请求独立
NEVER	不主动创建，但已经存在的 Session 不会被销毁
IF_REQUIRED	需要时创建（默认）
ALWAYS	总是创建

JWT 场景必须 **STATELESS** ——否则 **Spring Security** 会偷偷用 **HttpSession** 缓存 **Authentication**，导致多实例部署时用户状态不一致。

1.7 addFilterBefore 顺序的艺术

```
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
.addFilterAfter(orgTagAuthorizationFilter, JwtAuthenticationFilter.class);
```

Spring Security 过滤器链默认有几十个 **filter**，顺序大致是：

1. **SecurityContextPersistenceFilter**
2. **HeaderWriterFilter**
3. ...
4. **UsernamePasswordAuthenticationFilter**（表单登录用）
5. ...
6. **AuthorizationFilter**（最后做权限决策）

addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class) 把 JWT 过滤器插到「表单登录」之前——这样 **JWT** 解析在 **Spring Security** 的标准认证之前完成。

addFilterAfter(orgTagAuthorizationFilter, JwtAuthenticationFilter.class) 把组织标签授权放在 JWT 之后——因为 **org tag** 鉴权依赖 **SecurityContext** 里的 **Authentication**。

面试考点 2：filter 顺序写反了会怎样？

答：

- JWT 放在最后 → 走到 **AuthorizationFilter** 时 **SecurityContext** 是空的 → 所有请求 **401**。
- OrgTag filter 放在 JWT 之前 → 拿不到 username → 鉴权逻辑全部失效 → 「能登录但无权限」。

这个 **BUG** 极难排查，因为代码看起来都对。pai-smart 的 **addFilterAfter** 是正确选择。

【二、CustomUserDetailsService：把 User 变成 Spring Security 的 UserDetails】

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException
    {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            getAuthorities(user.getRole())
        );
    }

    private Collection<? extends GrantedAuthority> getAuthorities(User.Role role) {
        return Collections.singletonList(new SimpleGrantedAuthority("ROLE_" + role.name()));
    }
}
```

注意一个「坑」**：这里的 `User` 是 `org.springframework.security.core.userdetails.User`，不是我们自己的 `com.yizhaoqi.smartpai.model.User`。

UserDetailsService 的契约：通过 `username` 找到用户，返回 `UserDetails` 接口的实现。

`User` 实体 → `UserDetails` 的转换发生在这一行。关键转换：

- `user.getPassword()`（BCrypt 加密后的字符串）。
- `getAuthorities(role)` → `"ROLE_USER"` / `"ROLE_ADMIN"`。

ROLE_ 前缀是约定俗成——`hasRole("ADMIN")` 实际匹配的是 `"ROLE_ADMIN"`。没有 **ROLE_** 前缀，`hasRole` 找不到。这是 Spring Security 的设计，记死。

面试考点 3：`@PreAuthorize("hasRole('ADMIN')")` 和 `hasRole("ADMIN")` 等价吗？

答：等价。`@PreAuthorize` 的 SpEL 表达式里写的是 `hasRole(...)`，实际 Spring 会拼上 **ROLE_** 前缀。

如果用 `hasAuthority("ADMIN")`（不带 **ROLE_** 前缀），就要写完整的 **ROLE_ADMIN** 字符串。

【三、PasswordUtil：BCrypt 加密的正确姿势】

```
public class PasswordUtil {
    private static final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
}
```

```
public static String encode(String rawPassword) {
    return encoder.encode(rawPassword);
}

public static boolean matches(String rawPassword, String encodedPassword) {
    return encoder.matches(rawPassword, encodedPassword);
}
}
```

BCryptPasswordEncoder 的核心特性：

1. **不可逆**：单向哈希，没有解码函数。
2. **自动加盐**：每次 `encode` 都生成随机 salt，同一个密码两次加密结果不同。
3. **慢**：默认 `strength = 10`，约 **100ms/次**。这是特性不是 **bug**——慢到攻击者暴力破解也累。

为什么 `encoder` 是 `static final` ?

`BCryptPasswordEncoder` 是线程安全的，可以单例。实例化它不需要 **Spring**——所以放在 `utils/` 包的静态类，调用方无依赖。

面试考点 4：BCrypt 同一密码两次加密结果不同，怎么验证？

答：自帶 **salt**。加密結果字符串是：

`encoder.matches(rawPassword, encodedPassword)` 内部会自动从 **encoded** 里取 **salt**，再用同一算法加密，最后比对。对调用方完全透明。

面试考点 5：BCrypt vs Argon2 vs SCrypt 怎么选？

- **BCrypt**：最成熟，大部分框架默认支持。`pai-smart` 选它合理。
- **Argon2**：2015 年密码哈希竞赛冠军，抗 **GPU/ASIC** 攻击更强。Spring Security 5.x 之后有 `Argon2PasswordEncoder`。
- **SCrypt**：内存硬，更抗定制硬件攻击，但慢。

业务上 **99%** 的项目用 **BCrypt** 就够了。金融、加密货币用 Argon2/SCrypt。

【四、登录接口：触发 AuthenticationManager】

我们看 `UserController` 的登录接口（不在第 02 集 `AuthController` 里，因为登录是用户自己的事）：

```

@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    // 1. 通过 CustomUserDetailsService 加载用户
    UserDetails userDetails = userDetailsService.loadUserByUsername(request.username());

    // 2. 验证密码
    if (!PasswordUtil.matches(request.password(), userDetails.getPassword())) {
        throw new CustomException("密码错误");
    }

    // 3. 生成 token
    String token = jwtUtils.generateToken(request.username());
    String refreshToken = jwtUtils.generateRefreshToken(request.username());

    return ResponseEntity.ok(Map.of(
        "token", token,
        "refreshToken", refreshToken,
        "user", Map.of("username", request.username(), "role", userDetails.getAuthorities())
    ));
}

```

这个流程就是「**Spring Security 手动版**」**——不依赖 `UsernamePasswordAuthenticationFilter`，自己用 `AuthenticationManager` 的核心组件。

注意：代码里没有用 `AuthenticationManager` ——而是直接调 `userDetailsService.loadUserByUsername` + `PasswordUtil.matches`。这是简化写法。完整写法是：

```

AuthenticationManager authManager = ...;
Authentication auth = authManager.authenticate(
    new UsernamePasswordAuthenticationToken(request.username(), request.password())
);

```

两种写法的对比：

- 手动写法：代码短，完全控制。
- Manager 写法：可以接 `AuthenticationProvider` 链，更灵活（比如加图形验证码、IP 白名单）。

`pai-smart` 选手动写法——业务简单，没必要上 **AuthenticationManager**。这是 KISS 原则。

【五、注册接口：写库 + 自动登录】

```

@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
    // 1. 校验用户名唯一
    if (userRepository.findByUsername(request.username()).isPresent()) {
        throw new CustomException("用户名已存在");
    }
}

```

```

// 2. 加密密码
String hashedPassword = PasswordUtil.encode(request.password());

// 3. 保存
User user = new User();
user.setUsername(request.username());
user.setPassword(hashedPassword);
user.setRole(User.Role.USER); // 默认普通用户
user.setOrgTags("DEFAULT"); // 默认组织标签
user.setPrimaryOrg("DEFAULT");
userRepository.save(user);

// 4. 直接发 token (免登录)
String token = jwtUtils.generateToken(user.getUsername());
return ResponseEntity.ok(Map.of("token", token, "user", user));
}

```

几个设计点：

1. 默认角色是 **USER**：不开放前端注册管理员。管理员通过 `AdminUserInitializer` 启动时建（看 `config/AdminUserInitializer.java`）。
2. 默认组织标签 **DEFAULT**：第 11 集会讲，这是多租户体系的基础。
3. 注册即发 **token**：省一次登录。用户体验更好——但也意味着注册即用，邮箱验证得另说。`pai-smart` 没做邮箱验证——这是简化，生产环境应该加。

面试考点 6：注册接口应该返回哪些字段？

- 必须：`token`、`refreshToken`、`user`（不含密码）。
- 可选：`permissions`（前端用来动态显示菜单）。
- 绝对不能：返回 `password` 字段（即使是加密的）。

`pai-smart` 用了 `Map.of(...)` 而非返回 `User` 实体——避免不小心返回 **password** 字段。这种小心思值得学。

【六、登出接口：让 token 失效】

```

@PostMapping("/logout")
public ResponseEntity<?> logout(@RequestHeader("Authorization") String authHeader) {
    String token = authHeader.substring(7);
    jwtUtils.invalidateToken(token);
    return ResponseEntity.ok(Map.of("message", "Logged out"));
}

```

就调一次 `invalidateToken`。第 03 集讲过，它做了两件事：

1. 把 `tokenId` 加黑名单。
2. 从 `userId -> Set<tokenId>` 索引里删除。

注意：黑名单有 TTL，和原 **token** 过期时间一致——过期后自动清，不占 **Redis** 内存。

【七、思考题：为什么 **SecurityConfig** 没有用 **@EnableGlobalMethodSecurity** ？**

@EnableGlobalMethodSecurity（**6.x** 改名 **@EnableMethodSecurity**）用来开启方法级权限控制：

```
@EnableMethodSecurity
public class SecurityConfig { ... }

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

pai-smart 没用——所有权限控制都在 **SecurityConfig** 里用 **requestMatchers** 配。

两种风格的对比：

- 路径级（**pai-smart**）：粒度粗，所有逻辑都在配置里。
- 方法级：粒度细，每个 **Service/Controller** 方法单独标注。

pai-smart 选路径级，因为业务简单——能用 URL 区分的场景就没必要上方法级。

什么时候上方法级？

- 「同一个 URL 不同方法不同权限」（比如 **GET /users** 任何人能看，**POST /users** 管理员能建）。
- **Controller** 里有复杂的条件权限（「你是这个资源的 owner 才能改」）。

【八、常见坑 & 面试问答】

Q1：为什么 **JwtAuthenticationFilter** 报错后没有返回 **401**？

A：看代码 catch 里只是 **logger.error**，然后 **filterChain.doFilter** 继续走。这意味着：

- Token 解析失败 → **SecurityContext** 是空的 → 走到 **AuthorizationFilter** → 401。
- **filter** 本身不负责 **401**，**Spring Security** 的 **AuthorizationFilter** 负责。

这种设计的好处：filter 链各司其职，每个 **filter** 只关心自己的一部分。**ExceptionTranslationFilter** 会把 **AccessDeniedException** 翻译成 401/403。

Q2：怎么自定义 **401** 返回格式？

A：写一个 **AuthenticationEntryPoint**：

```
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
```

```

        AuthenticationException authException) throws IOException {
    response.setStatus(401);
    response.setContentType("application/json");
    response.getWriter().write("{\"code\":401,\"message\":\"未登录\"}");
}

// SecurityConfig 里:
http.exceptionHandling(ex -> ex.authenticationEntryPoint(jwtAuthenticationEntryPoint));

```

pai-smart 没做这个——返回的是 Spring Security 默认的 401 响应体（HTML 或空）。生产环境建议改成 **JSON**。

Q3：登录接口是公开的，为什么不直接放 **permitAll** ？

A：/api/v1/users/login 确实在 **permitAll** 列表里。**Spring Security** 不会在登录时要求已有 **token**——这是显然的。但 **permitAll** 之外的所有路径都要 token，包括「改密码」之类的登录后操作。

Q4：BCrypt 慢，会不会拖慢登录？

A：单次 100ms，登录场景可以接受。但是每次 **HTTP** 请求都要 **BCrypt.matches** 吗？不会——**JwtAuthenticationFilter** 只验签，不查密码。密码只在登录时校验一次。

Q5：怎么防「撞库攻击」？

A：

- 加图形验证码（注册、登录、忘记密码时）。
- 加 **IP** 限流（**RateLimitService**，第 13 集讲）。
- 加密码强度校验（前端 + 后端）。
- 加账号锁定（连续失败 N 次锁定 M 分钟）。

pai-smart 在 **AdminUserInitializer** 里有默认管理员账号密码——生产部署必须改，否则裸奔。这点原作者也在 README 反复强调。

【九、@Bean 还是 @Component？SecurityConfig 的归类问题】

SecurityConfig 上有 **@Configuration** ——和 **@Component** 的区别：

- **@Configuration** 是 **@Component** 的特化，额外支持 **@Bean** 方法的 **CGLIB** 增强（多次调用返回同一个 bean）。
- 普通 **@Component** 里的 **@Bean** 方法不会被增强，每次调用都 **new** 一个——这会导致单例失效。

pai-smart 的 **SecurityConfig**、**RedisConfig**、**KafkaConfig** 等都是 **@Configuration**。这就是规矩。

【十、思考题 & 面试延伸】

Q：如果让你重写 `pai-smart` 的登录流程，你会怎么设计？

我的参考答案：

- 登录前：图形验证码 + IP 限流。
- 登录时：用户名密码 + 设备指纹 → JWT（含设备 ID）。
- 登录后：返回 access + refresh token。
- 主动登出：失效单端 token。
- 被动登出：改密码 → 失效所有端 token（用反向索引）。
- 异地登录：检测 IP/地理位置变化 → 强制二次验证。

这是进阶内容，等我们讲到「充值 + 配额」（第 17 集）时，会一并聊。

【十一、下集预告】

前 4 集我们搭好了用户体系 + 鉴权。

但是 RAG 系统核心是「文档」——没有文档，AI 答什么？

第 05 集，我们要：

- 引入 **MinIO**——文件存储从「服务器本地」变成「对象存储」。
- 看 `MinioConfig` 怎么初始化 bucket。
- 看 `FileUpload` 实体怎么设计。
- 看 `UploadController` 怎么处理分片上传。

MinIO 是 **S3** 协议的国产实现，部署简单，是 **RAG/AI** 项目的标配。

我们下期见。

觉得这一期的内容对你有帮助，点赞、投币、一键三连。

想看哪一集的深度展开？评论区留言。

下一集链接：[第 05 集：MinIO 登场——文件存储从本地到对象存储](#)

第 05 集：MinIO 登场——文件存储从本地到对象存储

系列：派聪明 RAG 后端演进全解

集数：05 / 20+

主题：对象存储概念、MinIO 与 S3 协议、FileUpload 实体设计、分片上传

上集回顾：第 04 集我们写完了 SecurityConfig，登录注册跑通

本集目标：文档能上传、能存到 MinIO，FileUpload 实体落库

【开场 Hook】

前 4 集我们做的都是「元数据」——用户、Token、权限。真正的内容（文档）还没登场。

RAG 系统的第一步是「有文档」。这一步看似简单，实际上埋了 8 个技术选型问题：

1. 文件存哪里？本地磁盘？**NAS**？对象存储？
2. 文件怎么唯一标识？文件名？**MD5**？**SHA256**？
3. 大文件怎么传？整文件上传？分片上传？断点续传？
4. 重复文件怎么去重？全文件哈希？分片哈希？
5. 文件元数据存哪？和文件一起？单独数据库？
6. 文件名冲突怎么办？**UUID**？雪花 ID？
7. 上传失败怎么回滚？事务？补偿？
8. 多副本怎么保证？单节点？集群？

pai-smart 早期估计就是「文件存本地磁盘」——简单，但生产环境一上来就遇到瓶颈（磁盘满、单点、备份难）。

所以我们直接从「对象存储」开始讲。用 **MinIO**。

【一、为什么是对象存储，不是本地磁盘？】

先看一个真实场景：

用户上传了一个 100MB 的 PDF 文档。文件存到 `/var/smartpai/uploads/`。然后有一天磁盘满了。运维扩容了一台机器，把磁盘挂过来……结果发现新机器看不到老文件。

本地存储的痛点：

- 单点：磁盘坏了文件就丢。
- 难扩展：磁盘满了只能扩容量，不能扩节点。
- 难备份：要写脚本 `rsync`、或者 `crontab` + `tar`。
- 多实例不共享：两个后端 Pod 看到的文件不同步。

对象存储的核心思想：

- 文件变成「对象」，每个对象有唯一的 **key**（类似文件名）。
- 文件存在「桶」（bucket）里，桶是平级的，没有目录层级。
- 元数据和数据分开存。元数据是「这个文件叫什么、多大、什么时候传的、谁传的」。
- 底层分布式，数据自动多副本。
- 通过 **HTTP/RESTful API** 访问——任何语言、任何平台都能用。

国内常见的对象存储：

- **AWS S3**（业界标准）。
- 阿里云 **OSS**（兼容 S3）。
- 腾讯云 **COS**（兼容 S3）。
- **MinIO**（自建对象存储，**100%** 兼容 **S3** 协议）。

`pai-smart` 选 MinIO——国产开源、自建集群、和 **S3 API** 兼容。开发环境用 Docker 一键起，生产环境可以扩到几十个节点。

面试考点 1：为什么不用 FastDFS？

FastDFS 也是国产对象存储（来自淘宝），不支持 **S3** 协议。社区活跃度下降，新项目基本不选它了。**S3** 协议已经是事实标准——`pai-smart` 选 MinIO 是正确的「面向未来」选择。

【二、MinioConfig：四行配置，初始化客户端】

```
@Getter
@Configuration
public class MinioConfig {

    @Value("${minio.endpoint}")
    private String endpoint;

    @Value("${minio.accessKey}")
    private String accessKey;

    @Value("${minio.secretKey}")
    private String secretKey;

    @Value("${minio.publicUrl}")
    private String publicUrl;

    @Bean
```

```

public MinioClient minioClient() {
    return MinioClient.builder()
        .endpoint(endpoint)
        .credentials(accessKey, secretKey)
        .build();
}

@Bean
public String minioPublicUrl() {
    return publicUrl;
}
}

```

逐个看：

- `endpoint`：MinIO 服务的访问地址。开发环境是 `http://localhost:9000`。
- `accessKey` / `secretKey`：访问密钥。和 **AWS IAM** 一样。
- `publicUrl`：给前端访问 MinIO 的 URL（可以是 **Nginx** 转发后的公网地址）。

`@Getter` 是 **Lombok** 注解——为 `endpoint` 等字段生成 getter。这是为了其他类能读到这些值（比如生成临时访问 URL）。

`@Bean public String minioPublicUrl()`——这看起来很怪：一个 String 类型的 bean？

它是为了让 **Spring** 把 `publicUrl` 注入到任何需要它的地方：

```

@Autowired
@Qualifier("minioPublicUrl")
private String publicUrl;

```

这是个值得学习的小技巧——用 `@Bean` 把单个配置值注册成 **Spring bean**，避免到处 `@Value`。

面试考点 2：@Value vs @Bean 注册配置值，怎么选？

- `@Value`：简单直观，适合少量配置。
- `@Bean` 注册：适合多个相关配置、或者想在 **bean** 创建时做点逻辑（比如校验非空、转换格式）。

`pai-smart` 这里两个都用了，算冗余设计——`@Value` 已经注入了 `publicUrl`，完全没必要再注册一个 **bean**。这是 KISS 原则的反面例子。

【三、FileUpload 实体：把"文件"映射成数据库行】

```

@Data
@Entity
@Table(name = "file_upload")
public class FileUpload {
    public static final int STATUS_UPLOADING = 0;
}

```

```

public static final int STATUS_COMPLETED = 1;
public static final int STATUS_MERGING = 2;
public static final String VECTORIZATION_STATUS_PENDING = "PENDING";
public static final String VECTORIZATION_STATUS_PROCESSING = "PROCESSING";
public static final String VECTORIZATION_STATUS_COMPLETED = "COMPLETED";
public static final String VECTORIZATION_STATUS_FAILED = "FAILED";

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(name = "file_md5", length = 32, nullable = false)
private String fileMd5;

private String fileName;
private long totalSize;
private int status; // 0-上传中 1-已完成 2-合并中

@Column(name = "user_id", length = 64, nullable = false)
private String userId;

@Column(name = "org_tag")
private String orgTag;

@Column(name = "is_public", nullable = false)
private boolean isPublic = false;

@Column(name = "estimated_embedding_tokens")
private Long estimatedEmbeddingTokens;

@Column(name = "estimated_chunk_count")
private Integer estimatedChunkCount;

@Column(name = "actual_embedding_tokens")
private Long actualEmbeddingTokens;

@Column(name = "actual_chunk_count")
private Integer actualChunkCount;

@Column(name = "vectorization_status", length = 32)
private String vectorizationStatus;

@Column(name = "vectorization_error_message", length = 1000)
private String vectorizationErrorMessage;

@CreationTimestamp
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime mergedAt;
}

```

3.1 file_md5 : 文件的"身份证"

```
@Column(name = "file_md5", length = 32, nullable = false) private String fileMd5;
```

为什么用 **MD5** ?

- 同一个文件，内容相同 → **MD5** 相同。
- 不同文件，**MD5** 几乎不可能相同（碰撞概率 2^{-128} ）。
- **32** 个字符（128 bit），长度可控。

MD5 在文件场景的合理用途：

- ☒ 去重：用户传同一个文件，秒传。
- ☒ 完整性校验：下载后比对 MD5，确认传输没出错。
- ☒ 安全敏感场景（密码、数字签名）：别用 **MD5**，用 SHA-256 或更安全。

pai-smart 用 **MD5** 做文件指纹是合理的——不是用于密码学，仅用于去重和校验。

面试考点 3：MD5 还能用吗？

A：能，但要分场景：

- 密码、签名、证书：禁用（已被王小云教授 2004 年攻破）。
- 文件指纹、数据去重：可以用（不需要抗碰撞攻击，碰撞了也无所谓）。

3.2 status : 上传状态机

```
public static final int STATUS_UPLOADING = 0;  
public static final int STATUS_COMPLETED = 1;  
public static final int STATUS_MERGING = 2;
```

这是分片上传的状态机：

```
0 (UPLOADING)  -- 客户端分片传完 --> 2 (MERGING)  
2 (MERGING)    -- 服务端合并分片 --> 1 (COMPLETED)
```

为什么需要 **MERGING** 这个中间态？

因为分片上传是异步的：

1. 客户端把文件切成 N 片，一片一片传。
2. 服务端先存到 MinIO 的临时桶（或者本地）。
3. 所有分片都到了，客户端调「合并」接口。
4. 服务端做合并（merge）：把所有分片按顺序拼成一个完整文件，写回 MinIO 的正式桶。

如果用户传了一半掉线，**status** 还在 **UPLOADING**——下次可以从断点继续（断点续传）。

面试考点 4：为什么不用 **enum** 用 **int** ？

A：作者图省事。更好的设计：


```
@Enumerated(EnumType.STRING)
private UploadStatus status;

public enum UploadStatus {
    UPLOADING, MERGING, COMPLETED, FAILED
}
```

`pai-smart` 第 02 集用 `@Enumerated(EnumType.STRING)`，这里又用 `int`——前后不一致。这是实战中常见的“技术债”。

3.3 `vectorizationStatus`：第二个状态机

```
public static final String VECTORIZATION_STATUS_PENDING = "PENDING";
public static final String VECTORIZATION_STATUS_PROCESSING = "PROCESSING";
public static final String VECTORIZATION_STATUS_COMPLETED = "COMPLETED";
public static final String VECTORIZATION_STATUS_FAILED = "FAILED";
```

这是文档向量化的状态机（第 07 集讲）。注意是 `String` 不是 `int`——说明作者后来意识到类型安全的重要性。

`PENDING → PROCESSING → COMPLETED / FAILED`。

为什么用 `String` 不用 `enum`？项目里其他地方（如 `orgTags`）都是 `String`，风格统一。这不是最佳实践——`enum` 更安全。

3.4 `org_tag` 和 `is_public`：权限字段

```
@Column(name = "org_tag")
private String orgTag;

@Column(name = "is_public", nullable = false)
private boolean isPublic = false;
```

这两个字段直接关系到多租户体系（第 11 集）。

- `orgTag`：文件归属的组织。`DEFAULT` 表示默认租户。
- `isPublic`：是否公开。公开文件所有用户都能访问，私有文件只能同组织访问。

设计哲学：不要存“谁能看”的列表，存“属于哪个组织”。访问控制通过“组织关系”动态算——更灵活、更可扩展。

3.5 `estimated_embedding_tokens` 和 `actual_embedding_tokens`：双口径

```
@Column(name = "estimated_embedding_tokens")
private Long estimatedEmbeddingTokens;
```

```

@Column(name = "estimated_chunk_count")
private Integer estimatedChunkCount;

@Column(name = "actual_embedding_tokens")
private Long actualEmbeddingTokens;

@Column(name = "actual_chunk_count")
private Integer actualChunkCount;

```

这是 **RAG** 系统的"账本"。

- 预估：上传时根据文件大小估算会消耗多少 token、要分多少块。
- 实际：向量化跑完后真实消耗的 token 和 chunk 数。

为什么要记两套？

- 预估用于「上传时即时反馈给用户」（"这个文件预计要花 1.5 万 token"）。
- 实际用于「精准计费 / 配额扣减」（第 17 集讲充值时用）。

这种"预估 + 实际"的设计在 **RAG** 系统里非常重要——**embedding API** 是花钱的，必须能精确对账。

面试考点 5：为什么 `estimated_embedding_tokens` 不是 `int` 是 `Long`？

A：单个文档可能很大。**int** 上限 21 亿，**token** 数量可能超过（虽然实际不太可能）。**Long** 永远不溢出——业务上更稳。`actualChunkCount` 用 `Integer` 足够（chunk 数不会超过百万级）。

【四、UploadController：分片上传的实现思路】

虽然没贴完整代码，但从 `SecurityConfig` 的白名单和 `FileUpload` 实体可以推断：

```

@PostMapping("/upload/chunk")
public ResponseEntity<> uploadChunk(
    @RequestParam("fileMd5") String fileMd5,
    @RequestParam("chunkIndex") int chunkIndex,
    @RequestParam("totalChunks") int totalChunks,
    @RequestParam("file") MultipartFile chunk) {
    // 1. 校验 token (filter 已经做了)
    // 2. 校验 fileMd5
    // 3. 存到 MinIO 的临时桶：{fileMd5}/{chunkIndex}
    // 4. 记录到 Redis：file:upload:{fileMd5} = 已上传的分片集合
    // 5. 返回已上传的分片索引列表（用于断点续传）
}

@PostMapping("/upload/merge")
public ResponseEntity<> mergeChunks(
    @RequestParam("fileMd5") String fileMd5,
    @RequestParam("fileName") String fileName,
    @RequestParam("totalChunks") int totalChunks) {
    // 1. 检查所有分片都到了
    // 2. 从 MinIO 拉所有分片

```

```

// 3. 合并成完整文件
// 4. 上传到正式桶
// 5. 创建 FileUpload 记录
// 6. 触发异步向量化 (Kafka 消息)
}

```

分片上传的关键设计点：

1. 分片大小：常见 5MB。太小：请求次数多、签名开销大。太大：超过某些代理的 body 限制（Tomcat 默认 2MB）。
2. 临时桶 **vs** 正式桶：临时桶存分片，TTL 7 天（自动清理）。正式桶存合并后的文件，永久。
3. 秒传：如果 `fileMd5` 已存在且 MD5 相同，直接返回成功——不用真的上传。
4. 断点续传：客户端重连时，服务端返回已上传的分片列表，客户端只传剩下的。

面试考点 6：分片上传的并发问题怎么解决？

A：两种方案：

- 乐观锁：用 `version` 字段，更新时 `WHERE version = oldVersion`，影响行数 0 则重试。
- 分布式锁：用 Redis `SETNX` 锁住 `file:{fileMd5}`，操作完释放。

`pai-smart` 用的是 Redis 锁（从 `TokenCacheService` 等的依赖推断）。

【五、MinIO 怎么访问？Presigned URL 模式**

对象存储不能直接给前端 `accessKey`——那是 **root** 权限。生产环境用 **Presigned URL**：

原理：服务端生成一个有时效的 **URL**，带上签名，前端拿这个 URL 直接传到 MinIO，绕过应用服务器。

```

// 服务端
String presignedUrl = minioClient.getPresignedObjectUrl(
    GetPresignedObjectUrlArgs.builder()
        .method(Method.PUT)
        .bucket("documents")
        .object(fileMd5 + "/" + fileName)
        .expiry(60 * 60) // 1 小时有效
        .build()
);

// 返回给前端
return Map.of("uploadUrl", presignedUrl);

```

前端：

```
await fetch(presignedUrl, { method: 'PUT', body: file });
```

好处：

- 带宽不经过应用服务器——大文件上传到 OSS 不挤压 Web 服务器。
- **accessKey** 不暴露给前端——签名有时间限制。
- 可吊销——签名过期就废了。

pai-smart 是否用 Presigned URL，要看 **UploadController** 完整代码——我推测早期没用（直接服务端中转），后来优化时改的。

【六、常见坑 & 面试问答】

Q1：MinIO 的 bucket 名有什么限制？

A：

- 3-63 字符。
- 只能小写字母、数字、点、连字符。
- 必须以小写字母或数字开头。
- 不能是 **IP** 地址形式（虽然技术上能）。
- 全局唯一——同一 region 内不能和别人重名（但自建 MinIO 不受这个限制）。

Q2：isPublic 字段用 boolean 还是 Boolean？

A：

- **boolean**：基本类型，不能为 **null**，默认 **false**。适合业务上一定有默认值的字段。
- **Boolean**：包装类型，可以为 **null**。适合“未设置”也是合法状态的字段。

pai-smart 用 **boolean** 且默认 **false**——合理。**isPublic** 业务上必有值（要么公开、要么私有），没有“未知”状态。

Q3：分片上传失败后，怎么清理临时文件？

A：两种方式：

- **TTL 策略**：MinIO 的 lifecycle rule 设 7 天过期。
- **定时任务**：每天扫一次 **file_upload** 表，**status = UPLOADING** 且 **createdAt < now - 7 days** 的，删 **MinIO** 文件 + 删 **DB** 记录。

pai-smart 看代码应该两种都做了（MinIO 本身的 lifecycle + 应用层定时清理）。

Q4：怎么防“恶意传超大文件”？

A：

- 前端：上传前限制文件大小（防 99% 攻击）。
- 后端 **multipart** 限制：**spring.servlet.multipart.max-file-size = 100MB**。
- 后端业务校验：**if (file.getSize() > MAX_SIZE) throw ...**。
- **MinIO** 端：**minio.bucket.quota** 限制桶总容量。

Q5：MD5 真的够安全吗？文件去重场景？

A：够。**MD5** 碰撞需要精心构造，两个不同文件算出来 MD5 相同的概率是 2^{-128} ——宇宙毁灭也算不出来。但安全场景别用（密码、签名、证书），用 SHA-256。

【七、思考题：为什么 FileUpload 用 Long id 而不用 String fileMd5 当主键？**

两种设计：

方案 A：自增 Long ID（pai-smart 当前）：

```
@Id @GeneratedValue(strategy = IDENTITY)
private Long id;
private String fileMd5;
```

方案 B：**MD5** 当主键：

```
@Id
private String fileMd5; // 32 字符
```

对比：

- 方案 A：DB 主键索引快（int 比较比 String 快）、自增 ID 在分布式环境会冲突。
- 方案 B：去重直接由 DB 约束保证（PK 冲突 = 重复文件）、业务意义强（"MD5 一样的就是同一文件"）。

pai-smart 选 A——保守稳妥。但 B 在文件去重场景其实更优雅。这是设计权衡，没有标准答案。

【八、下集预告】

文件能上传了、能存 MinIO 了，但里面的内容我们还不知道。

第 06 集我们要：

- 用 **Apache Tika** 解析 PDF / Word / Excel / PPT。
- 把提取出来的文本分块（Chunking）。
- **HanLP** 中文分词让中文文档分得更好。
- 流式处理大文件——不爆内存。

文档解析是 **RAG** 系统的"上游"——解析质量直接决定 **RAG** 效果。

我们下期见。

如果你觉得这一集的"演进式"讲解比贴代码好懂，点赞、投币、一键三连。
评论区告诉我你最想听哪一集的深度展开。

下一集链接：[第 06 集：文档解析 Pipeline——Tika 与文本分块](#)

第 06 集：文档解析 Pipeline——Tika 与文本分块

系列：派聪明 RAG 后端演进全解

集数：06 / 20+

主题：Apache Tika 解析、PDFBox 抽页、HanLP 中文分词、父-子块策略、流式防 OOM

上集回顾：第 05 集 MinIO 搭好了，文件能上传、能存

本集目标：把 PDF / Word / Excel 里的文字提取出来，按语义切成块，落库

【开场 Hook】

文件上传完了。但是 **MinIO** 里的文件是"死的"——它是字节流，**AI** 读不懂。

RAG 系统的第一步，永远是「把文件变成文本」。

这一步看起来简单（Ctrl+A、Ctrl+C），真做起来全是坑：

- PDF 有「文本型 PDF」和「扫描型 PDF」——前者能直接抽文字，后者需要 OCR。
- Word 文档里有「正文」「表格」「页眉页脚」——哪些算内容？
- 一个 500MB 的 PDF 一次性读进内存，直接 **OOM**。
- 中文怎么分词？「小明在北京大学读书」——**北京大学** 是一个词还是 **北京 + 大学**？
- 切块太大，召回不精准；切块太小，语义断裂。

pai-smart 用的是 **Apache Tika + PDFBox + HanLP** 的组合拳——工业级 **RAG** 的标配。

今天我们就把 **ParseService** 整个拆开。

【一、为什么 Tika？自己写解析行不行？】

先看场景：

用户上传了一个文件，可能是：PDF、Word、Excel、PPT、TXT、Markdown、HTML、JSON、CSV.....

如果你自己写解析：

- PDF：PDFBox / iText
- Word：Apache POI
- Excel：Apache POI

- PPT : Apache POI
- HTML : Jsoup
- Markdown : commonmark-java
-

光是引入依赖就 **5-6** 个。还要写适配层，根据文件类型选不同的解析器。

Tika 的解决方案：一个 **API**，解析所有格式。

```
<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-core</artifactId>
  <version>2.9.1</version>
</dependency>
<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-parsers-standard-package</artifactId>
  <version>2.9.1</version>
</dependency>
```

`pai-smart` 引了这两个。注意是「**standard-package**」——这一个 jar 把 PDF、Word、Excel、PPT 等几十种格式的解析器都打包了。

Tika 的核心思想：**MIME** 识别 + 委托解析。

1. 识别：用 magic bytes（文件头几个字节）判断文件类型。
2. 委托：根据文件类型，把解析任务委托给对应的 parser（PDFBox、POI 等）。

面试考点 **1**：Tika 怎么识别文件类型？

答：Tika 用 `org.apache.tika.detect.Detector` 接口。默认实现是 `DefaultDetector`，综合 **magic bytes** + 文件扩展名 + **MIME** 类型。比如：

- `%PDF-1.4` 开头的 → PDF
- `PK\x03\x04` 开头的 → ZIP（再打开看是 Office 还是 EPUB 还是 JAR）
- `{\rtf1` 开头的 → RTF

实战中：Tika 的识别率不是 **100%** ——有些诡异文件识别错。所以 `pai-smart` 还有 `FileTypeValidationService` 二次校验（看 `service/` 目录）。

【二、`parseAndSave` 方法：流式入口】

```
public void parseAndSave(String fileMd5, InputStream fileStream,
    String userId, String orgTag, boolean isPublic) throws IOException, TikaException {
    checkMemoryThreshold();

    try (BufferedInputStream bufferedStream = new BufferedInputStream(fileStream,
```



```

bufferSize)) {
    if (isPdfDocument(bufferedStream)) {
        parsePdfAndSave(fileMd5, bufferedStream, userId, orgTag, isPublic);
        return;
    }

    StreamingContentHandler handler = new StreamingContentHandler(fileMd5, userId, orgTag, isPublic);
    Metadata metadata = new Metadata();
    ParseContext context = new ParseContext();
    AutoDetectParser parser = new AutoDetectParser();

    parser.parse(bufferedStream, handler, metadata, context);
} catch (SAXException e) {
    throw new RuntimeException("文档解析失败", e);
}
}

```

2.1 checkMemoryThreshold() : 防 OOM 第一道防线

```

@Value("${file.parsing.max-memory-threshold:0.8}")
private double maxMemoryThreshold;

private void checkMemoryThreshold() {
    Runtime runtime = Runtime.getRuntime();
    long maxMemory = runtime.maxMemory();
    long totalMemory = runtime.totalMemory();
    long freeMemory = runtime.freeMemory();
    long usedMemory = totalMemory - freeMemory;

    double memoryUsage = (double) usedMemory / maxMemory;

    if (memoryUsage > maxMemoryThreshold) {
        logger.warn("内存使用率过高: {:.2f}%, 触发垃圾回收", memoryUsage * 100);
        System.gc(); // 调一次 GC

        usedMemory = runtime.totalMemory() - runtime.freeMemory();
        memoryUsage = (double) usedMemory / maxMemory;

        if (memoryUsage > maxMemoryThreshold) {
            throw new RuntimeException("内存不足, 无法处理大文件");
        }
    }
}

```

pai-smart 用了三招防 OOM :

1. 解析前检查内存: `memoryUsage > 0.8` 就先 `System.gc()`, 还高就拒绝。
2. 流式处理: 下面会讲, 不一次性读进内存。
3. `bufferSize = 8192`: 每次只读 8KB。

面试考点 2: `System.gc()` 是不是好习惯?

A：不推荐——**JVM** 会忽略。但这里是「保命」——不如 GC 直接拒绝请求。生产环境更优雅的方案：

- 限流：超过 N 个并发解析任务直接拒绝。

- 队列：大文件进队列慢慢处理，不阻塞 **Web** 线程（这就是 Kafka，第 09 集要讲）。

2.2 `BufferedInputStream`：基础 IO 优化

```
try (BufferedInputStream bufferedStream = new BufferedInputStream(fileStream, bufferSize)) {
```

`bufferSize = 8192`（8KB）——磁盘 IO 每次 **8KB** 读，比一次读 **1 byte** 快 **8000** 倍。这是 Java IO 性能的「第一课」。

`try-with-resources` 自动关闭流，不怕忘记 `close`。

2.3 PDF 单独走 `parsePdfAndSave`

```
if (isPdfDocument(bufferedStream)) {  
    parsePdfAndSave(fileMd5, bufferedStream, userId, orgTag, isPublic);  
    return;  
}
```

为什么 **PDF** 单独处理？

因为 PDF 有页的概念——「第 5 页」和「第 6 页」的内容不能混在一起（要支持「跳到原文档第 X 页」这种用户操作）。所以 PDF 解析保留页号信息。

其他格式（Word、Excel）没有页的概念，走通用流式处理。

面试考点 3：Tika 也能解析 PDF，为什么不用？

答：Tika 把 PDF 当文本流，不分页。`pai-smart` 用 `PDFTextStripper`（PDFBox 自带）能精确按页提取。对于 **RAG** 系统，页号很重要——「这个回答来自原文档第 5 页」是产品刚需。

【三、`StreamingContentHandler`：Tika 的「回调地狱」】

Tika 的设计是 **SAX** 风格——边读边回调：

```
private class StreamingContentHandler extends BodyContentHandler {  
    private final StringBuilder buffer = new StringBuilder();  
  
    @Override  
    public void characters(char[] ch, int start, int length) {  
        buffer.append(ch, start, length);  
        if (buffer.length() >= parentChunkSize) {  
            processParentChunk();  
        }  
    }  
}
```

```

    }

    @Override
    public void endDocument() {
        if (buffer.length() > 0) {
            processParentChunk();
        }
    }
}

```

这是 **RAG** 系统的"灵魂设计"——父-子块策略 (Parent-Child Chunking)。

3.1 父块 + 子块

```

@Value("${file.parsing.parent-chunk-size:1048576}")
private int parentChunkSize;

```

`parentChunkSize = 1048576 = 1MB` ——父块大小 **1MB**。

```

@Value("${file.parsing.chunk-size}")
private int chunkSize;

@Value("${file.parsing.overlap-size:100}")
private int overlapSize = 100;

```

`chunkSize` 不知道具体值 (在 `application.yml` 里)，推测是 500-1000 字符左右。子块大小。
overlapSize = 100 字符，块和块之间重叠 **100** 字符。

为什么要有「父-子」两层？

想象这个场景：

一个 PDF 500 页。用户问「Apple 公司的 Q3 财报里净利润是多少？」。

答案可能在第 **245** 页 ——「Apple Q3 净利润 1.2 万亿...」，但上下文（「这是 2024 年第三季度财报...」）在第 **244** 页。

如果只按子块（500 字符）切，「Apple Q3 净利润 1.2 万亿」和「这是 2024 年第三季度财报」被切到两个 chunk，召回后看不到上下文。

父-子策略的解法：

1. 子块 (**chunk**) 用于检索——粒度小，召回精准。
2. 父块用于回答生成——召回时同时返回父块的全部内容，给 **LLM** 完整上下文。

工作流程：

1. 解析时，累积字符到 `parentChunkSize` (1MB)。

- 一旦超 1MB，触发 `processParentChunk`：把父块切成子块，每个子块存到 `DocumentVector` 表，子块带 `parentChunkId`。
- 子块是检索的最小单位。命中子块后，把整个父块都返回给 LLM。

这就是 RAG 系统的“两级索引”——比“一刀切”好得多。

面试考点 4：父-子块 vs 滑动窗口 vs 句子切分？

- 滑动窗口（`pai-smart` 用的就是 `overlapSize`）：简单，但子块之间有冗余。
- 句子切分：用句号、问号切，粒度自然，但短句子无信息、长句子切不开。
- 父-子策略：最现代的做法，子块用于检索，父块用于生成。OpenAI 的 `Assistants API` 用的就是这套。

3.2 `characters` 回调：Tika 的流式核心

```
@Override
public void characters(char[] ch, int start, int length) {
    buffer.append(ch, start, length);
    if (buffer.length() >= parentChunkSize) {
        processParentChunk();
    }
}
```

Tika 解析文档时，每读到一段文本就调一次 `characters` ——可能一行调一次，可能一段调一次。我们 `append` 进 `StringBuilder` ——不立即处理。

直到 `buffer` 超过 1MB —— `processParentChunk()` 触发。

这个设计的精妙之处：

- 不用一次读完整个文件。500MB 的 PDF，内存里只留 1MB。
- 流式——读完 1MB 就存盘一次，清空 `buffer` 继续读。
- `endDocument()` 里再处理一次——文档末尾的「尾巴」可能不到 1MB，要记得处理。

面试考点 5：为什么用 `StringBuilder` 不用 `byte[]`？

A：Tika 给的是 `char[]`，已经是字符了。`StringBuilder` 是字符的容器，天然合适。`byte[]` 是字节的容器，还得做编码转换。

【四、HanLP：中文分词的「神兵」】

```
<dependency>
  <groupId>com.hankcs</groupId>
  <artifactId>hanlp</artifactId>
```

```
<version>portable-1.8.6</version>
</dependency>
```

```
import com.hankcs.hanlp.seg.common.Term;
import com.hankcs.hanlp.tokenizer.StandardTokenizer;

List<Term> terms = StandardTokenizer.segment("小明在北京大学读书");
```

输出：

```
[小明, 在, 北京大学, 读书]
```

HanLP 的核心能力：

- 中文分词：北京大学 不会被切错。
- 词性标注：名词 / 动词 / 形容词。
- 命名实体识别：小明 → 人名、北京大学 → 机构名。
- 关键字提取、摘要提取、情感分析。

为什么 **RAG** 需要 **HanLP**？

场景 1：搜索时：

用户搜「北京大学奖学金」。

不分词的话，**ES** 全文检索会把"北京大学"和"北京"作为两个 **token** ——「北京大学研究生院」就匹配不上。

HanLP 预先分词 → 北京大学 / 研究生院，索引和查询一致。

场景 2：实体识别：

用户问「Apple 公司的 Q3 财报」。

HanLP 识别 Apple → 机构名 ——可以做 entity 级别的过滤，**RAG** 召回更准。

pai-smart 用的 **portable** 版本是精简版（模型文件小，约 30MB），够用。完整版要 **1GB+**。

面试考点 6：为什么不用 IK Analyzer 或 Jieba？

- **IK Analyzer**：纯 Java，轻量，分词准确率不如 **HanLP**。
- **Jieba**：Python 出身，Java 移植版准确率还行，功能没 **HanLP** 丰富。
- **HanLP**：Java 出身，功能最全，但 **jar** 包大。

pai-smart 选 **HanLP** 是要做 **NER**（命名实体识别）——IK 和 Jieba 没这个能力。

【五、子块切片：怎么切？】

我们没看到 `processParentChunk` 的完整实现，但从配置推测：

```
private void processParentChunk() {
    String parentContent = buffer.toString();
    int start = 0;
    int end = 0;
    int chunkId = 0;

    while (start < parentContent.length()) {
        end = Math.min(start + chunkSize, parentContent.length());
        String chunk = parentContent.substring(start, end);

        // 1. 用 HanLP 分词 (可选)
        // 2. 保存到 DocumentVector 表
        //    - fileMd5
        //    - chunkId
        //    - textContent
        //    - parentChunkId (指向父块)
        //    - orgTag
        //    - isPublic
        //    - userId
        documentVectorRepository.save(...);

        start = end - overlapSize; // 滑动窗口
        chunkId++;
    }

    buffer.setLength(0); // 清空 buffer
}
```

关键参数：

- `chunkSize`：子块大小（500-1000 字符）。经验值：嵌入模型的 max input length 的 60-80%。
`text-embedding-v4` 是 2048 token，对应中文约 **1500** 字符。
- `overlapSize`：滑动窗口重叠（100 字符）。目的：避免一个语义完整的句子被切到两个 chunk。
- `minChunkSize`：最小块大小。避免太多「小碎块」。

面试考点 7：chunk 切得太小或太大，会怎样？

- 太小（< **200** 字符）：
 - 召回多但噪音大。
 - 「这个问题答了一半」——上下文不够。
- 太大（> **2000** 字符）：
 - 召回少但精度高。
 - 嵌入模型 token 超限，被截断。
 - 召回的 chunk 本身已经够回答，不需要 **LLM** 再做总结。

- 经验值：
- 英文：500-1000 token。
- 中文：300-800 字符（中文 token 化更密）。

`pai-smart` 选「小块+父块」——兼顾了精度和上下文。

【六、`estimateEmbeddingUsage`：上传前预估】

```
public EmbeddingEstimate estimateEmbeddingUsage(InputStream fileStream) throws IOException,
TikaException {
    try (BufferedInputStream bufferedStream = new BufferedInputStream(fileStream,
bufferSize)) {
        if (isPdfDocument(bufferedStream)) {
            return estimatePdfEmbeddingUsage(bufferedStream);
        }
        StreamingEstimateHandler handler = new StreamingEstimateHandler();
        // ... 同 parseAndSave, 但不存库, 只计算 token 数
        return handler.snapshot();
    }
}
```

为什么需要预估？

`pai-smart` 有配额系统（第 17 集讲）——每个用户每月有 X token 额度。

上传前就要告诉用户：「这个文件预计要 1.5 万 token，你还有 8 万 token 额度，确定要上传吗？」

预估 **vs** 实际的差异：

- 预估：根据文件大小 × 平均字符密度估算。
- 实际：embedding API 真实返回的 token 数。

`FileUpload` 实体里的 `estimatedEmbeddingTokens` 和 `actualEmbeddingTokens` 就是给这个用的。

【七、`StreamingEstimateHandler` **vs** `StreamingContentHandler`**】

注意看，两个 **Handler** 类 —— `StreamingContentHandler`（存库）和 `StreamingEstimateHandler`（预估）——功能类似但行为不同。

为什么不开一个 **Handler** 加 `boolean save` 参数？

A：**SRP** 原则（Single Responsibility Principle）。两个 Handler 各管一件事：

- `StreamingContentHandler`：处理字符 → 切片 → 存 DB。
- `StreamingEstimateHandler`：处理字符 → 估算 token → 返回结果。

好处：

- 一个 Handler 的代码只关心一件事。
- 测试更容易——只测一件事。
- 改一个不影响另一个。

但代价：有重复代码。`pai-smart` 的处理是用内部类 `private class` —— `StreamingEstimateHandler` 写在 `ParseService` 内部，牺牲一些独立性换代码紧凑。

面试考点 8：内部类 vs 独立类怎么选？

- 内部类：逻辑高度耦合、只为这个外部类用、不希望被外部直接 `new`。
- 独立类：可能被其他地方复用、需要被注入、需要 `mock` 测试。

`StreamingContentHandler` 是内部类，但实际上可以被替换（用不同的 chunking 策略）。更好的设计应该是策略模式——`pai-smart` 这里偷懒了。

【八、`parsePdfAndSave`：按页解析的细节】

虽然没看完整代码，但 `pai-smart` 一定是这么做的：

```
private void parsePdfAndSave(String fileMd5, InputStream fileStream, ...) {
    try (PDDocument pdf = Loader.loadPDF(fileStream)) {
        PDFTextStripper stripper = new PDFTextStripper();
        int totalPages = pdf.getNumberOfPages();

        for (int page = 1; page <= totalPages; page++) {
            stripper.setStartPage(page);
            stripper.setEndPage(page);
            String pageText = stripper.getText(pdf);

            // 1. 存页内容到 DocumentVector (带 pageNumber 字段)
            // 2. 或者存到单独的 page 表
        }
    }
}
```

关键设计：

- `pageNumber` 字段：`DocumentVector` 表有 `pageNumber` 列。召回时可以告诉用户「这个答案来自原文档第 X 页」。
- `anchor_text`：PDF 里的「锚点文本」——比如小标题、图表标题。用于快速预览命中位置。

面试考点 9：扫描型 PDF（图片）怎么解析？

A：需要 OCR。`Tesseract`（Java 端口叫 `Tess4J`）。`pai-smart` 当前版本不支持 **OCR**——这是已知 **TODO**。生产环境 **OCR** 是个大工程：

- 部署 Tesseract 引擎。

- 处理 OCR 失败的重试。
 - 处理 OCR 出来的乱码（尤其中文扫描件）。
 - **PDF** 中的图片单独抽出来 OCR（比整页 **OCR** 更快）。
-

【九、常见坑 & 面试问答】

Q1：为什么 `BufferedInputStream` 不用 `Files.newInputStream`？

A：方法签名是 `InputStream fileStream`——调用方传什么就是什么。`pai-smart` 不想绑定到 `Path`——可能是 `MinIO` 的流、可能是 `HTTP` 的流、可能是测试的字节数组流。接口要松散。

Q2：`parentChunkSize = 1MB` 会不会太大？

A：1MB 父块对应约 50 万字符、约 25 万 token。普通业务文档 1MB 够 200-500 页。但 1MB 的 `StringBuilder` 在 JVM 里就是 ~2MB `char[]`——这个内存占用不大。真正要担心的是大文件累积。

Q3：HanLP 的 `portable` 版本和 `enterprise` 版本区别？

A：

- `portable`：30MB 模型，纯 Java 部署方便，功能精简。
- `enterprise`：1GB+ 模型，需要 Python 协同，功能齐全。

`pai-smart` 用 `portable` 是平衡。

Q4：流式处理下，`processParentChunk` 失败怎么办？事务回滚？

A：这是个经典问题。`pai-smart` 的解法：

- 整个文件解析用 `@Transactional`——失败回滚。
- 回滚后清 MinIO 临时文件、删 DB 记录。
- 如果中途崩溃（不是抛异常），需要补偿任务：定时扫 `status = UPLOADING` 的孤儿记录，清理。

这就是「分布式事务」的简化版——单 DB 事务 + 异步补偿。

Q5：HanLP 的 `portable-1.8.6` 有安全漏洞吗？

A：`portable-1.8.6` 是 2022 年的版本。最新稳定版是 1.8.7+。`pai-smart` 用的版本稍老，生产建议升级——HanLP 在迭代中修复了不少 bug。

【十、思考题：父-子块策略的「代价」是什么？】

优点讲了很多。代价呢？

- 存储翻倍：每个父块存 1 次，每个子块存 1 次。1000 字符的子块 + 1MB 父块——一个 500MB 文档可能产生 $500 * 500 = 25$ 万个子块记录。
- JOIN 慢：召回子块 → 找父块 → 取父块内容。多一次 DB 查询。

- 向量化重：父块变了，所有子块都要重新 embedding（**500MB** 文档的 **embedding** 成本可观）。

实际工程权衡：

- 小型 **RAG**（< 10 万文档）：父-子策略完全值得。
- 大型 **RAG**（> 100 万文档）：考虑简化——只用滑动窗口切，不维护父块。

pai-smart 选了前者——业务上不会到 **100** 万文档级别，质量优先。

【十一、下集预告】

文档已经切成块了，但这些块对 **LLM** 来说还是「死的」——它们是字符串，不是数字。

第 07 集我们要：

- 调通 **Embedding API**（DashScope text-embedding-v4）。
- 把每个 chunk 向量化——字符串变成 1024 维的 float 数组。
- 把向量存到 **DocumentVector** 表。

这一步之后，**RAG** 才真正有了"语义"的能力。

我们下期见。

如果你想看 RAG 系统的"上游"是怎么搭的，这 **6** 集把「文档 → 文本 → 块」讲透了。
觉得有用，点赞、投币、收藏。

下一集链接：[第 07 集：EmbeddingClient 登场——把文本变成向量](#)

第 07 集：EmbeddingClient 登场——把文本变成向量

系列：派聪明 RAG 后端演进全解

集数：07 / 20+

主题：Embedding 模型、WebClient 调 API、批处理、配额预留与结算、限流

上集回顾：第 06 集文档解析 + 分块完成，文本块落库

本集目标：把每个 chunk 调用 Embedding API 向量化，存到 DocumentVector

【开场 Hook】

上一集我们把文档切成了 chunk，每个 chunk 是 500-1000 字符的文本。

但 LLM 不认字符串。LLM 认的是数字。

这一步叫 **Embedding**——把一段文本变成一个高维向量（`pai-smart` 用的是 DashScope 的 `text-embedding-v4`，**1024** 维 **float** 数组）。

"派聪明" → [0.012, -0.034, 0.078, ..., 0.092] (1024 个数字)

这一步是 **RAG** 的「灵魂」。没有 **embedding**，**RAG** 只能做关键词搜索；有了 **embedding**，**RAG** 才开始「理解语义」。

今天我们深入 `EmbeddingClient` 这个类，看它是怎么：

1. 调通 Embedding API。
2. 批量处理（省 **token**、省钱）。
3. 配额预留（先扣再用）。
4. 失败重试。
5. 多 Provider 路由（`pai-smart` 后期加了）。

【一、什么是 Embedding？】

简单说：embedding 是一个函数 $f(\text{text}) \rightarrow \text{vector}$ ，把字符串映射到一个高维空间。

两个核心特性：

1. 语义相似 → 向量相似。

- `f("苹果公司")` 和 `f("Apple Inc.")` 距离很近。
- `f("苹果公司")` 和 `f("香蕉")` 距离中等。
- `f("苹果公司")` 和 `f("太阳系")` 距离很远。

2. 固定维度。

- 不管输入多长（1 个字还是 1000 字），输出都是 **1024** 维。
- 这让所有文本可以「放在同一个空间里比较」。

类比：

- 文本是「地址」（北京市朝阳区建国路 88 号）。
- Embedding 是「经纬度」（116.48, 39.91）。
- 判断两个地址远近比「看地址字符串」更精确。

面试考点 1：常见 Embedding 模型有哪些？

模型	厂商	维度	上下文长度	特点
text-embedding-3-small	OpenAI	1536	8191	性价比高
text-embedding-3-large	OpenAI	3072	8191	精度高
text-embedding-v4	阿里通义	1024/1536/2048 可选	8192	pai-smart 用的
bge-large-zh	BAAI	1024	512	中文开源
m3e	Moka	1024	512	中文开源

pai-smart 选 v4 的关键原因是国产、合规、便宜。

【二、WebClient：为什么是 WebFlux 的 client？】

```
import org.springframework.web.reactive.function.client.WebClient;
```

pai-smart 同时引入了 `spring-boot-starter-web` 和 `spring-boot-starter-webflux`。`WebClient` 是 `webflux` 里的。

为什么不直接用 `RestTemplate` 或 `HttpClient`？

- **RestTemplate**：Spring 自带，同步阻塞，已经进入维护模式（Spring 5 之后不再推荐）。
- **HttpClient**：JDK 11+ 自带，同步，要用的话自己写一堆样板。
- **WebClient**：响应式（Reactive），支持流式、异步、自动重试。

Embedding API 调用的特性：

- 单次调用耗时 200ms-1s（网络 + 推理）。
- 不是 CPU 密集型——网络 IO 密集型。
- **WebClient** 的非阻塞优势在这里不显著（每次还是等 API 返回）。
- 但 **WebClient** 的 **API** 比 **RestTemplate** 现代——链式调用、可读性好。

****pai-smart 选 WebClient 的实际理由****：1. ****DeepSeek 流式响应****（第 10 集）需要 **Reactive Streams**。2. ****统一栈****——既然 **webflux** 已经引了，****调外部 API 都用它****。3. ****WebClient 也能同步用****——**block()**。

面试考点 2：RestTemplate vs WebClient？

A：Spring 官方 5+ 推荐 **WebClient**。**RestTemplate** 还能用但仅维护不开发新功能。**pai-smart** 这种新项目用 **WebClient** 是对的。

【三、embedWithUsage：核心流程】

```
public EmbeddingUsageResult embedWithUsage(List<String> texts, String requesterId,
UsageType usageType) {
    try {
        String normalizedRequesterId = requesterId == null || requesterId.isBlank() ? "unknown" : requesterId;
        logger.info("开始生成向量，文本数量：{}", texts.size());

        List<float[]> all = new ArrayList<>(texts.size());
        int totalTokens = 0;
        for (int start = 0; start < texts.size(); start += batchSize) {
            int end = Math.min(start + batchSize, texts.size());
            List<String> sub = texts.subList(start, end);

            // 1. 预留配额
            UsageQuotaService.TokenReservationBundle reservation = usageType == UsageType.QUERY
                ? rateLimitService.reserveEmbeddingQueryUsage(normalizedRequesterId, sub)
                : rateLimitService.reserveEmbeddingUploadUsage(normalizedRequesterId, sub);

            // 2. 调 API
            try {
                String response = callApiOnce(sub);
                EmbeddingApiResponse parsedResponse = parseEmbeddingResponse(response, sub);

                // 3. 结算配额（按实际 token 数）
                usageQuotaService.settleReservation(reservation, parsedResponse.totalTokens());

                all.addAll(parsedResponse.vectors());
                totalTokens += parsedResponse.totalTokens();
            } catch (Exception e) {
```

```

        // 4. 失败 → 释放预留
        usageQuotaService.abortReservation(reservation);
        throw e;
    }
}
return new EmbeddingUsageResult(all, totalTokens, currentModelVersion());
} catch (Exception e) {
    logger.error("生成向量失败", e);
    throw new RuntimeException("生成向量失败", e);
}
}

```

3.1 批处理：batchSize = 100

```

@Value("${embedding.api.batch-size:100}")
private int batchSize;

for (int start = 0; start < texts.size(); start += batchSize) {
    int end = Math.min(start + batchSize, texts.size());
    List<String> sub = texts.subList(start, end);
    // ...
}

```

为什么 **100** 一批？

- **API** 端：DashScope v4 的 batch 上限是 10、25、100（不同模型不同）。**100** 是常见上限。
- 网络：100 个文本 1 次请求 vs 100 次请求，省了 **99** 次 **HTTP** 开销。
- 超时：1 个 100 文本请求 vs 100 个 1 文本请求，前者更不容易超时。

subList 是 **List** 的视图——不复制数据。省内存。

面试考点 **3**：批处理大小怎么选？

A：

- **API** 文档告诉你最大 batch——用最大值。
- 网络稳定就调大，网络差就调小（大 **batch** 失败成本高）。
- 超时敏感就调小，长任务就调大。
- 经验值：Embedding **32-100**，LLM **1-10**（LLM 输出长，batch 不能太大）。

pai-smart 用 100 是合理的——业务规模和 **API** 限制的平衡。

3.2 「预留 → 结算」两步走

```

// 1. 预留
UsageQuotaService.TokenReservationBundle reservation = usageType == UsageType.QUERY
    ? rateLimitService.reserveEmbeddingQueryUsage(normalizedRequesterId, sub)
    : rateLimitService.reserveEmbeddingUploadUsage(normalizedRequesterId, sub);

// 2. 调 API

```

```
String response = callApiOnce(sub);
EmbeddingApiResponse parsedResponse = parseEmbeddingResponse(response, sub);

// 3. 结算 (按实际 token)
usageQuotaService.settleReservation(reservation, parsedResponse.totalTokens());

// 失败时
} catch (Exception e) {
    usageQuotaService.abortReservation(reservation);
    throw e;
}
```

这是配额系统的"事务模式"——先扣后用，用完结算，失败回滚。

为什么要"预留"？

因为 **Embedding API** 是异步的 —— 请求时不知道实际多少 **token** (**API** 响应才告诉你)。如果不预留：

- 100 个用户同时上传大文件 → 都并发调 Embedding → 系统超额使用配额。
- 等到 API 返回才知道用了多少 → 配额的实时性差。

预留机制的工作流程：

1. 预估：`sub.size() * 平均字符数 / 4` 估算 token (粗估)。
2. 预扣：从用户配额里先扣掉这个估计值。
3. 结算：API 回来后按实际值结算——多退少补。
4. 失败回滚：调 API 出错，预留全退。

这是金融系统的"预授权"模式——信用卡刷预授权 → 实际消费后结算 → 取消时解冻。

面试考点 4：为什么不用"先实际跑、然后扣费"？

A：那是串行的——必须等所有 API 调用完才能知道结果。预留机制可以并发——**100** 个用户同时上传，互相不影响。配额的"实时性"是底线——不能让用户超用。

3.3 失败重试

代码里用 `reactor.util.retry.Retry` (在调用 `callApiOnce` 内部)：

```
.webClient.post()
    .uri(apiUrl)
    .bodyValue(requestBody)
    .retrieve()
    .bodyToMono(String.class)
    .retryWhen(Retry.backoff(3, Duration.ofSeconds(1))
        .filter(throwable -> throwable instanceof WebClientResponseException))
    ...
```

`Retry.backoff(3, Duration.ofSeconds(1))`：

- 最多重试 3 次。

- 退避策略：第 1 次 1s 后重试，第 2 次 2s，第 3 次 4s（指数退避）。
- 只对 `WebClientResponseException` 重试（网络错误、**5xx** 错误）——业务错误（**4xx**）不重试。

面试考点 5：为什么指数退避？

A：避免「重试风暴」。100 个客户端同时重试 → 服务端压力 × 2 → 雪崩。

指数退避 + 随机抖动让重试分散开。

【四、批处理的几个陷阱】

4.1 `subList` 的坑

```
List<String> sub = texts.subList(start, end);
```

`subList` 是 `AbstractList` 的视图——不是新 `list`。对 `subList` 的修改会影响原 `list`。

`pai-smart` 没问题——它只读不写。但如果代码里出现 `subList.add(...)` 或 `subList.clear()`，会有 `ConcurrentModificationException` 或意外修改原 `list`。

面试考点 6：`subList` 的限制？

A：

- `subList` 是原 `list` 的视图，结构修改会相互影响。
- 对原 `list` 的结构修改（`add/remove`）后，`subList` 不可用。
- 元素修改（`set`）没问题。
- 需要独立 `list` 时用 `new ArrayList<>(subList)`。

4.2 批内 `token` 累计

```
usageQuotaService.settleReservation(reservation, parsedResponse.totalTokens());  
all.addAll(parsedResponse.vectors());  
totalTokens += parsedResponse.totalTokens();
```

每个 batch 单独结算，最后累加到 `totalTokens`。这是 "**BATCH 级别配额**"——细粒度，某批失败不影响其他批。

但有个问题：批内部分失败怎么办？比如 100 个文本里第 50 个出问题？

`pai-smart` 的策略：整批失败。不是"部分成功"。

更优雅的策略：把 batch 再拆小，逐个调用。但调用次数多 **100** 倍。业务上整批重试已经够用。

【五、callApiOnce：构造请求 + 处理响应】

虽然没贴完整代码，但根据 `embedWithUsage` 的调用可以推断：

```
private String callApiOnce(List<String> texts) {
    // 1. 构造请求体
    Map<String, Object> body = new HashMap<>();
    body.put("model", currentModelVersion()); // "text-embedding-v4"
    body.put("input", texts);
    body.put("dimension", "1024");
    body.put("encoding_format", "float");

    // 2. 构造 HTTP 请求
    return webClient.post()
        .uri(provider.apiBaseUrl() + "/compatible-mode/v1/embeddings")
        .header(HttpHeaders.AUTHORIZATION, "Bearer " + provider.apiKey())
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(body)
        .retrieve()
        .bodyToMono(String.class)
        .retryWhen(Retry.backoff(3, Duration.ofSeconds(1))
            .filter(t -> t instanceof WebClientResponseException))
        .block(Duration.ofSeconds(60));
}
```

几个关键点：

- `provider.apiBaseUrl() + compatible-mode/v1/embeddings`：DashScope 的 **OpenAI** 兼容端点。
- `Bearer apiKey`：标准鉴权。
- `bodyToMono`：返回 `Mono<String>`，**WebFlux** 的响应式类型。
- `.block(60s)`：同步等结果，最多 **60** 秒。
- `encoding_format = "float"`：返回 `float[]` 而不是 `base64` 字符串。

面试考点 7：DashScope 的 "compatible-mode" 是什么？

A：阿里云为了兼容 **OpenAI API** 搞的"翻译层"——同样的 **OpenAI client** 代码可以指向 **DashScope** 的 URL。pai-smart` 这么干的好处：换 **provider** 不用改代码，只换 **baseUrl**。

【六、ModelProviderConfigService：多 Provider 路由**】

```
@PostConstruct
public void init() {
    ModelProviderConfigService.ActiveProviderView provider =
        modelProviderConfigService.getActiveProvider(ModelProviderConfigService.SCOPE_EMBEDDING);
    logger.info("EmbeddingClient 初始化 - Provider: {}, 模型: {}, 批次大小: {}, 维度: {}, API地
```

```
址: {}",
    provider.provider(), provider.model(), batchSize, provider.dimension(), provider.api
    BaseUrl());
}
```

这段代码的玄机：

- **SCOPE_EMBEDDING**：这是「**Provider** 作用域」——同一个系统里**Embedding** 用 **A** 提供商、**LLM** 用 **B** 提供商。
- **getActiveProvider**：从数据库（**ModelProviderConfig** 表）读当前激活的 **Provider** 配置。
- 包含字段：**provider**、**model**、**apiBaseUrl**、**apiKey**、**dimension**。

为什么要在数据库里？

第 16 集会详细讲。**pai-smart** 后来加了多 **LLM Provider** 路由——同一个系统可以同时支持 **DeepSeek**、**Qwen**、**OpenAI**、**Claude**。管理员可以在管理后台切换。

但 **Embedding** 端提前就有了 **Provider** 路由——这是个渐进式设计。

面试考点 8：为什么不直接用配置文件？

A：配置文件是部署时确定的。数据库配置是运行时可改的。好处：

- **A/B** 测试：新 provider 灰度切流量。
- 降级：主 provider 挂了，手动切到备用。
- 多租户：不同租户用不同 provider（**Enterprise** 场景）。

【七、**UsageType** 枚举：上传 vs 查询】

```
public enum UsageType {
    UPLOAD,
    QUERY
}
```

为什么要区分？

因为配额策略不同：

- **UPLOAD**：上传文档 → 大量 embedding → 配额消耗大。可能走专门的“上传额度”（月限制 **100 万 token**）。
- **QUERY**：用户搜索 → 单次 embedding → 配额消耗小。可能走专门的“查询额度”（月限制 **50 万次**）。

为什么不合并成一种？

A：业务规则。比如「上传额度 10 万 token / 月，查询额度无限」——鼓励用户用系统。如果合并，一次上传就把额度用完。

****pai-smart 选 UPLOAD/QUERY 二分法**——**简化设计**。******生产环境可能更细** (UPLOAD/QUERY/ADMIN/EXPORT/...`)。**

【八、 `parseEmbeddingResponse` : 解析 API 响应】

DashScope 的标准响应 (OpenAI 格式) :

```
{
  "object": "list",
  "data": [
    { "object": "embedding", "embedding": [0.012, -0.034, ...], "index": 0 },
    { "object": "embedding", "embedding": [0.045, -0.067, ...], "index": 1 }
  ],
  "model": "text-embedding-v4",
  "usage": {
    "prompt_tokens": 1000,
    "total_tokens": 1000
  }
}
```

`parseEmbeddingResponse` 干什么 :

```
private EmbeddingApiResponse parseEmbeddingResponse(String response, List<String> originalTexts) {
    JsonNode root = objectMapper.readTree(response);
    JsonNode data = root.get("data");
    JsonNode usage = root.get("usage");

    int totalTokens = usage.get("total_tokens").asInt();
    List<float[]> vectors = new ArrayList<>();
    for (JsonNode item : data) {
        JsonNode embedding = item.get("embedding");
        float[] vec = new float[embedding.size()];
        for (int i = 0; i < embedding.size(); i++) {
            vec[i] = (float) embedding.get(i).asDouble();
        }
        vectors.add(vec);
    }
    return new EmbeddingApiResponse(vectors, totalTokens);
}
```

几个细节 :

- 顺序保持 : DashScope 返回的 `data` 数组和请求的 `input` 顺序一致——这很重要, 否则会乱套。
- `(float) embedding.get(i).asDouble()` : JSON 默认是 `double` , 转 `float` 节省一半内存。
- `totalTokens` : 实际消耗的 token , 用来结算配额。

面试考点 9：为什么 float 不用 double？

A：精度权衡。

- **double**：8 字节 / 数字。**1024** 维 → 8KB / 向量。**1** 万个向量 → 80MB。

- **float**：4 字节 / 数字。**1024** 维 → 4KB / 向量。**1** 万个向量 → 40MB。

精度损失：float 的精度约 $1e-7$ ，对于相似度计算完全够用（相似度是相对值，不是绝对值）。生产环境 **99%** 用 **float**。

【九、向量存到哪？DocumentVector 表】

第 06 集我们看到了 `DocumentVector` 实体，关键字段：

```
@Column(name = "vector_id")
private Long vectorId;
private String fileMd5;
private Integer chunkId;
@Lob
private String textContent;
private Integer pageNumber;
private String anchorText;
private String modelVersion;
private String userId;
// ...
```

`vector` 字段没看到？可能：

- 存 **MySQL**：用 `BLOB` 或 `JSON`（性能差）。

- 存 **Elasticsearch**：用 `dense_vector` 类型（专业工具）。

pai-smart 选 **Elasticsearch**——第 08 集讲。**MySQL** 存向量是个反模式。

MySQL 8.0+ 已经有向量类型，但性能远不如 **ES**（**ES** 用 **HNSW** 索引）。

【十、常见坑 & 面试问答】

Q1：批处理大小可以超过 **100** 吗？

A：DashScope v4 当前限制是 25/100/300（不同 **dimension** 不同）。**pai-smart** 用 **100** 是平衡。如果未来 **DashScope** 提高限制，可以无脑改 `application.yml` **。

Q2：失败重试的“幂等性”问题？

A：Embedding API 本身不是幂等的（同一段文本多次调用也多次计费）。重试会增加成本。

pai-smart 用配额预留 + 失败回滚——避免重复扣费。但 **API** 端已经收费了（**API** 调用成功但客户端没收到响应）——这种“幽灵调用”无法完全避免。

Q3 : `WebClient` 阻塞调用 (`block`) 不是反模式吗？

A : 是的，严格说是反模式。但 `pai-smart` 是 **WebMVC** 应用 (`@RestController`) ——**WebFlux** 异步优势用不上。

真正的异步应该是 `bodyToFlux()` + Reactive Service + Reactive Controller。 `pai-smart` 没这么做——简化。

Q4 : **Embedding API** 限流了怎么办？

A :

- `Retry.backoff` : 自动重试。
- `RateLimitService.reserveEmbeddingUploadUsage` : 配额预留阶段就拒绝。
- 降级到本地模型 : 用 bge-large-zh 等开源模型，不花钱。 `pai-smart` 暂时没做。

Q5 : 向量召回效果不好，怎么调？

A :

- 换 **Embedding** 模型 : v3 → v4 , 质量提升明显。
- 调 **chunk** 大小 : 500 → 800 字符，保留更多上下文。
- 调 **overlap** : 100 → 200 字符，减少边界切分问题。
- 加 **rerank** : 召回后再用 BGE-Reranker 精排。
- 混合检索 : BM25 + 向量 (第 12 集讲) 。

【十一、思考题：为什么 `EmbeddingClient` 不缓存结果？**

场景：同一个 chunk 被向量化两次。

第一次（上传时）：调 API，存到 **DB**。

第二次（重建索引时）：又调一次 **API**。

为什么不缓存？

A : 缓存是脏活——失效、版本、内存都要管。 `pai-smart` 选择"按需调用"：

- 同样的 **chunk** : 用 `fileMd5 + chunkId` 做去重，避免重复存。
- 不同 **chunk** : 必须调 API (没有"复用"的可能) 。

更优的设计：在 `DocumentVector` 加 **UNIQUE(fileMd5, chunkId, modelVersion)** ——重复插入冲突就跳过。 `pai-smart` 应该这么做。

【十二、下集预告】

向量生成了，存哪？MySQL 存向量是反模式，用 **Elasticsearch**。

第 08 集我们要：

- `EsConfig` 怎么初始化 ES Client。
- `EsIndexInitializer` 怎么建索引（包括 `dense_vector` 字段）。
- `ElasticsearchService` 怎么把向量写进 ES。
- **kNN** 召回怎么写查询。

ES 是 **RAG** 系统的「数据库」——没有它就没有现代 **RAG**。

我们下期见。

这一期的"配额预留 + 结算"是金融级设计——很多 **RAG** 系统的 **token** 计费都很粗糙，`pai-smart` 这套值得学。

觉得有用，点赞、投币、收藏。

下一集链接：[第 08 集：Elasticsearch 集成——向量索引与召回](#)

第 08 集：Elasticsearch 集成——向量索引与召回

系列：派聪明 RAG 后端演进全解

集数：08 / 20+

主题：ES 8.x dense_vector、HNSW 索引、kNN 召回、IkAnalyzer 分词

上集回顾：第 07 集 EmbeddingClient 调通了，1024 维向量生成了

本集目标：把向量存到 ES，召回用 kNN 搜索

【开场 Hook】

向量有了，存哪？

三种选择：

1. **MySQL**：8.0+ 有向量类型，但慢——B-tree 索引不适合高维向量。
2. 专用向量数据库（Milvus、Qdrant、Chroma）：功能专一，但运维成本高。
3. **Elasticsearch 8.x**：**dense_vector** + **HNSW**——和传统文本搜索同一个栈。

`pai-smart` 选 ES——一套系统同时搞定文本搜索 + 向量召回 + 元数据过滤。

今天我们深挖：

- `EsConfig` 怎么初始化 ES Client。
- `EsIndexInitializer` 怎么建索引。
- `ElasticsearchService` 怎么写、怎么查。

【一、ES 8.x 的 `dense_vector`：向量字段的类型】

ES 在 8.0 引入 `dense_vector` 字段类型，专门存 float 数组。

mapping 示例（在 `EsIndexInitializer` 里推测）：

```
{
  "mappings": {
    "properties": {
      "fileMd5": { "type": "keyword" },
      "chunkId": { "type": "integer" },
      "textContent": { "type": "text", "analyzer": "ik_max_word" },
      "vector": { "type": "dense_vector", "dims": 1024, "index": true, "similarity": "co
```

```

    "isPublic": { "type": "boolean" },
    "pageNumber": { "type": "integer" }
  }
}
}
}

```

几个关键参数：

- **dims: 1024**：向量维度。必须和 **Embedding** 模型匹配。
- **index: true**：建 HNSW 索引（HNSW = Hierarchical Navigable Small World，图算法）。
- **similarity: cosine**：余弦相似度。也可以选 **dot_product** 或 **l2_norm**。
- **textContent** 用 **ik_max_word**：中文分词，细粒度。

面试考点 1：HNSW vs IVF vs PQ？

A：

- **HNSW**（图算法）：召回率高，内存占用大，**ES** 默认。
- **IVF**（倒排索引）：快，召回率略低，Milvus 默认。
- **PQ**（乘积量化）：压缩向量，内存小，精度低。

pai-smart 选 HNSW——质量优先。

【二、EsConfig：8.x Java Client 初始化】

```

@Bean
public ElasticsearchClient elasticsearchClient() {
    RestClientBuilder builder = RestClient.builder(new HttpHost(host, port, scheme));

    if (username != null && !username.isEmpty()) {
        BasicCredentialsProvider credsProvider = new BasicCredentialsProvider();
        credsProvider.setCredentials(AuthScope.ANY, new
UsernamePasswordCredentials(username, password));
        builder.setHttpClientConfigCallback(httpClientBuilder -> {
            if ("https".equalsIgnoreCase(scheme) && insecureTrustAllCertificates) {
                SSLContext sslContext = SSLContexts.custom()
                    .loadTrustMaterial(null, (X509Certificate[] chain, String authType)
-> true)
                    .build();
                httpClientBuilder.setSSLContext(sslContext);
                httpClientBuilder.setSSLHostnameVerifier(NoopHostnameVerifier.INSTANCE);
            }
            return httpClientBuilder.setDefaultCredentialsProvider(credsProvider);
        });
    }
}

```



```

    RestClient restClient = builder.build();
    ElasticsearchTransport transport = new RestClientTransport(restClient, new JacksonJsonpMapper());
    return new ElasticsearchClient(transport);
}

```

三层结构：

1. **RestClient**（Apache HttpClient 包装）：低级客户端，处理 HTTP。
2. **RestClientTransport**：传输层，序列化请求。
3. **ElasticsearchClient**：高级客户端，强类型 **API**。

面试考点 2：ES Java Client 的演变？

- **TransportClient**（7.x 之前）：基于 **Netty**，官方已废弃。
- **RestClient** + **RestClientTransport**（7.x+）：基于 **HTTP**，官方推荐。
- **elasticsearch-java**（8.x）：新强类型 **API**，**pai-smart** 用的就是它。

【三、**EsIndexInitializer**：建索引的时机】

EsIndexInitializer 是个 **ApplicationRunner** 或 **@PostConstruct**——应用启动时建索引。

为什么不在 **ddl-auto** 一样启动时建？

A：ES 的 mapping 比 SQL DDL 复杂——**dense_vector** 维度、**analyzer**、**HNSW** 参数都要明确指定。手工建更可控。

index 不存在 → 创建（with mapping）。

index 已存在 → 不动（避免覆盖数据）。

生产环境用 **EsIndexInitializer** 自动化首次部署很方便，后续修改 **mapping** 要用 migration 工具（**ES** 没有 **Flyway**——自己写脚本）。

【四、**ElasticsearchService**：写入向量】

```

public void indexDocument(EsDocument doc) {
    esClient.index(i -> i
        .index("knowledge_base")
        .id(doc.getId())
        .document(doc)
    );
}

```

EsDocument 实体（推测）：

```

@Data
public class EsDocument {
    private String id;           // ES doc id
    private String fileMd5;
    private Integer chunkId;
    private String textContent;
    private List<Float> vector; // 1024 维
    private String userId;
    private String orgTag;
    private Boolean isPublic;
    private Integer pageNumber;
}

```

`doc.getId()` 怎么生成？推测：

```
String id = fileMd5 + "_" + chunkId;
```

这样设计的好处：

- 同一个 **chunk** 重复写入 → 覆盖原 doc，幂等。
- 删除文件 → `delete_by_query: { fileMd5: xxx }`，简单。

面试考点 3：ES doc id 的设计原则？

A：

- 有业务含义（如 `fileMd5_chunkId`）：好做幂等和删除。
- **UUID** 随机：唯一性强，但删除要按 **query**。
- 自增 **Long**：不推荐（ES 分布式不友好）。

`pai-smart` 选「业务 id」是正确选择。

【五、kNN 召回：核心查询】

```

public List<SearchResult> searchWithPermission(String query, String userId, int topK) {
    List<Float> queryVector = embedToVectorList(query, userId);

    SearchResponse<EsDocument> response = esClient.search(s -> {
        s.index("knowledge_base");
        int recallK = topK * 30; // 粗排召回 K*30
        s.knn(kn -> kn
            .field("vector")
            .queryVector(queryVector)
            .k(recallK)
            .numCandidates(recallK)
        );
        // ... 权限过滤
    }, EsDocument.class);
}

```

```

return response.hits().hits().stream()
    .map(this::toSearchResult)
    .toList();
}

```

关键参数：

- **k: topK * 30**：粗排。多召回 **30** 倍（**30** 是经验值），让 **Rerank** 有空间。
- **numCandidates: topK * 30**：HNSW 搜索的候选数，通常 = **k**。
- **field: "vector"**：用 vector 字段做 ANN (Approximate Nearest Neighbor)。

ES 8.x 的 **kNN** 是"过滤器后召回"还是"召回后过滤"？

A：过滤器后召回。**knn** 和 **query** (bool filter) 是 **AND** 关系——先过滤用户能看的数据，再在过滤后的子集里做 **kNN**。

这是 **RAG** 系统的"多租户 + 向量召回"的正确姿势。

面试考点 **4**：为什么 **kNN** + filter 不会很慢？

A：ES 8.x 的 **kNN** 优化是「先按 **filter** 选 **doc id** 集合，再在子集上建 **HNSW** 索引」。子集小 → 搜索快。

【六、HybridSearchService：BM25 + 向量】

虽然第 12 集会专门讲，但 episode 8 我们先看 **HybridSearchService** 的"全貌"：

```

public List<SearchResult> searchWithPermission(String query, String userId, int topK) {
    List<Float> queryVector = embedToVectorList(query, userId);

    if (queryVector == null) {
        return textOnlySearchWithPermission(query, userId, topK); // 降级
    }

    SearchResponse<EsDocument> response = esClient.search(s -> {
        s.index("knowledge_base");
        int recallK = topK * 30;
        s.knn(kn -> kn
            .field("vector")
            .queryVector(queryVector)
            .k(recallK)
            .numCandidates(recallK)
        );
        s.query(q -> q.match(m -> m.field("textContent").query(query))); // BM25
        s.rescore(r -> r.windowSize(recallK).query(rq -> rq
            .queryWeight(0.2d) // 原始 BM25 权重 0.2
            .rescoreQueryWeight(1.0d) // rescore 权重 1.0
            .query(rqq -> rqq.match(m -> m.field("textContent").query(query)))
        ));
    });
}

```

```
// ... 权限过滤
}, EsDocument.class);

return response.hits().hits().stream()
    .map(this::toSearchResult)
    .toList();
}
```

这是 **pai-smart** 早期版本——**knn** + **query** + **rescore** 三件套。

三件套的协作：

1. **knn**：粗排，召回数 = **topK * 30**。
2. **query**：BM25 文本匹配，同样的粗排范围。
3. **rescore**：精排，用 **BM25** 重打分，权重 **1.0**。

为什么是 **queryWeight = 0.2**？让 BM25 在粗排阶段影响小，在 **rescore** 阶段影响大。因为 **rescore** 算得更精细。

面试考点 5：ES 的 rescore 是什么？

A：二次打分。**windowSize** 内的 doc 用 **rescoreQuery** 重算 score，然后用 **queryWeight** 和 **rescoreQueryWeight** 加权。比单次 **BM25** 准，比改主查询代价小。

【七、向量降级：API 失败怎么办？】

```
List<Float> queryVector = embedToVectorList(query, userId);
if (queryVector == null) {
    return textOnlySearchWithPermission(query, userId, topK);
}
```

降级策略：

1. Embedding API 调用失败 → **queryVector** 是 **null**。
2. 降级到纯文本搜索（**textOnlySearchWithPermission**）。
3. 用户体验不会完全崩溃——只是召回质量下降。

这是「优雅降级」的最佳实践。

面试考点 6：还有什么降级策略？

- 缓存：用 Redis 缓存最近 1000 条 query 的向量，减少 **API** 调用。
- 本地模型：用 bge-small 这种几十 **MB** 的本地模型——不花钱，速度快，质量差一些。
- 查询改写：把用户问题改写得更具体（**LLM** 重写）。

pai-smart 当前是“降级到 BM25”——最简方案。

【八、权限过滤：multi-tenant 的核心**

```
// 在 search() 里
s.query(q -> q.bool(b -> b
  .filter(f -> f.terms(t -> t.field("isPublic").terms(v -> v.value(...))))
  .filter(f -> f.terms(t -> t.field("orgTag").terms(v -> v.value(...))))
  .filter(f -> f.terms(t -> t.field("userId").terms(v -> v.value(...))))
));
```

三条 **OR** 起来的 **filter**：

- `isPublic = true`（公开）
- `orgTag IN (用户的所有 orgTag)`（同组织）
- `userId = currentUser`（自己的）

任何一个满足，就能搜到。

为什么用 **filter** 不用 **must**？

A：**filter** 不参与打分，只做过滤——更快。**must** 既过滤又打分——浪费算力。

面试考点 7：**filter** vs **must** 性能差异？

A：

- **must**：打分计算 score，进入相关性排序。
- **filter**：只做 yes/no 判断，不计算 **score**，可缓存。

「用户有权限看吗？」这种二元判断一定用 **filter**。

【九、ES 写入：bulk vs 单条**

```
// 单条
esClient.index(i -> i.index("kb").id(id).document(doc));

// 批量
esClient.bulk(b -> b.operations(ops));
```

单条 vs bulk 性能差异巨大：

- 单条：1000 个文档 → 1000 次 HTTP → ~10s
- **bulk**：1000 个文档 → 1 次 HTTP → ~0.5s

但 **pai-smart** 的向量写入是异步的（第 09 集 Kafka），单条也能接受。生产环境 大批量导入要用 **bulk**。

面试考点 8：bulk 请求大小怎么选？

A：单次 bulk 1-15MB 最佳。

- 太小：网络 round-trip 浪费。
- 太大：单请求处理慢，其他请求被阻塞。

`pai-smart` 没贴 bulk 代码——可能是单条写入为主，**bulk** 用于重建索引。

【十、ES 8.x 的 Rerank 支持**

ES 8.12+ 原生支持 **Rerank**——`inference` 端点 + `text_similarity` reranker。

`pai-smart` 当前用 **rescore** 自建 Rerank——已经够用。未来可以升级到原生 **Rerank**——效果更好，代码更少。

【十一、常见坑 & 面试问答】

Q1：vector 维度变了怎么办？

A：ES 不支持改 **mapping** 的 **dims**——必须 **reindex**：

1. 建新 index (`knowledge_base_v2` , `dims: 2048`)。
2. 业务层 dual-write (同时写 v1 和 v2)。
3. 数据迁移 (从 v1 读，embed 一次，写 v2)。
4. 切流量到 v2。
5. 删 v1。

`pai-smart` 早期没考虑这个——如果换 **Embedding** 模型，要重写一套。生产环境要在 `entity` 上记录 `modelVersion` 字段 (`pai-smart` 已经这么干了)。

Q2：HNSW 索引能改参数吗？

A：能，但要 **reindex**。`pai-smart` 用的默认参数 (`m: 16`, `ef_construction: 100`)。调参经验：

- `m`：图的出度。越大越准，内存越多。
- `ef_construction`：构建时的搜索宽度。越大越准，构建越慢。
- `ef_search`：查询时的搜索宽度。越大越准，查询越慢。

Q3：ES 8.x 强类型 client 怎么写复杂查询？

A：用 `withJson()` 传原始 JSON——最后兜底方案：

```
String json = ""
{
  "query": {
    "bool": {
      "filter": [...]
```

```
    }  
  }  
  "";  
  esClient.withJson(new StringReader(json));
```

pai-smart 没这么做——强类型 **client** 已经够用。

Q4 : ES 集群怎么部署？

A : 生产环境至少 **3** 节点 (master + data + ingest 角色分离)。 **pai-smart** 用 `docs/docker-compose.yaml` 部署的是单节点——仅开发。

Q5 : HNSW 索引在 ES 里有多大？

A : 粗估 : 1 万个 1024 维向量 + HNSW → 约 **50MB**。1 亿向量 → 约 **500GB**。生产环境需要 **SSD + 大内存**。

【十二、下集预告】

向量能写 ES 了，但整个流程还是同步的——用户上传文件 → 服务端立刻解析 + embedding → 几分钟后才能搜到。

用户等不了！

第 09 集我们要：

- 引入 **Kafka**——把同步处理改成异步。
- **KafkaConfig** 怎么配置生产者和消费者。
- **FileProcessingConsumer** 怎么消费消息。
- 失败重试 + 死信队列。

Kafka 是分布式系统的“解耦神器”——**pai-smart** 用了它，整个后端就活了。

我们下期见。

觉得这一期的 ES 集成讲得够深，点赞、投币、收藏。
关注我，下一期讲 **Kafka** 异步处理。

下一集链接：[第 09 集：Kafka 登场——把同步变异步](#)

第 09 集：Kafka 登场——把同步变异步

系列：派聪明 RAG 后端演进全解

集数：09 / 20+

主题：Kafka 概念、Topic / Partition / Offset、生产者消费者、失败重试 + DLT

上集回顾：第 08 集 ES 集成完成，向量能存能查

本集目标：把「上传文件→解析→向量化」拆成异步流水线

【开场 Hook】

第 8 集结束时，我们有了一个看似能用的 **RAG** 流程：

```
HTTP 上传文件 → 同步解析 → 同步 embedding → 写 ES
```

但是！一个 100MB 的 PDF：

- 解析：30 秒
- embedding：60 秒（1000 个 chunk × 60ms）
- 写 ES：5 秒

总耗时 **95 秒**。

这 **95 秒**里，**HTTP** 连接一直挂着。**Tomcat** 默认超时 **60 秒**——直接 **504**！用户得重试——然后又挂了 **95 秒**。

这是经典的“长任务阻塞 **Web** 线程”问题。

pai-smart 的解法：**Kafka**——把 95 秒的同步流程拆成异步任务。

```
HTTP 上传文件 → 立即返回（耗时 200ms）
                    ↓
                Kafka 消息
                    ↓
        后台 Worker 消费（耗时 95s）
                    ↓
                写 ES 完成
```

用户立刻拿到 **200**，后台慢慢处理。这就是异步化。

今天我们深挖 **FileProcessingConsumer** 和 **KafkaConfig** 怎么做到。

【一、为什么是 **Kafka**，不是 **Redis Stream** 或 **RabbitMQ**？】

消息队列的本质：一个 **FIFO** 的队列——生产者 put，消费者 take。

三种常见选择：

维度	Redis Stream	RabbitMQ	Kafka
持久化	有（但容量小）	有	有（ TB 级）
吞吐	10 万 / 秒	万级	百万级
顺序	单 partition	单 queue	单 partition
消费模型	至少一次	push / pull	pull
适用场景	轻量消息	复杂路由	日志 + 大数据

pai-smart 选 Kafka——因为文档处理是“大块任务”：

- 单条消息大（包含文件路径、元数据）。
- 处理慢（**95 秒/条**）。
- 顺序不重要（不同文件之间）。

Kafka 的优势：

- 持久化：Broker 集群，数据不丢。
- 重平衡：消费者增减，自动 **rebalance**。
- 死信队列：处理失败的消息有归宿。

面试考点 1：Kafka 为什么吞吐这么高？

A：顺序写磁盘 + 零拷贝（**sendfile** 系统调用）+ **page cache** 优先。单 **broker 100MB/s** 写入。

【二、**FileProcessingTask**：消息体的设计】

```
public class FileProcessingTask {
    public static final String TASK_TYPE_PROCESS = "PROCESS";
    public static final String TASK_TYPE_REINDEX = "REINDEX";

    private String fileMd5;           // 哪个文件
    private String userId;            // 谁传的
    private String orgTag;            // 哪个组织
    private boolean isPublic;         // 是否公开
    private String filePath;          // 在哪（MinIO URL / 本地路径）
    private String taskType;          // 任务类型
    private String requesterId;       // 请求人（重索引时可能是管理员）
}
```

几个设计点：

1. 不带文件内容：消息体只带路径。文件本身在 **MinIO**。避免消息过大（Kafka 单条消息默认 1MB 上限）。
2. **taskType** 区分 **PROCESS / REINDEX**：同一个 topic 支持两种任务类型——避免维护多套队列。
3. 包含权限信息：**userId / orgTag / isPublic** ——消费者处理时直接用，不用再查 **DB**。

面试考点 2：消息体应该多大？

A：

- < **1MB**：Kafka 默认。
- < **10MB**：调 **message.max.bytes** 后可以。
- > **10MB**：必须存外部（MinIO、S3），消息里带 URL。

pai-smart 选「消息里带路径」是正确选择。

【三、**FileProcessingConsumer**：消费流程】

```
@KafkaListener(topics = "#{kafkaConfig.getFileProcessingTopic()}",
                groupId = "#{kafkaConfig.getFileProcessingGroupId()}")
public void processTask(FileProcessingTask task) {
    documentService.markVectorizationProcessing(task.getFileMd5(), false);

    if (FileProcessingTask.TASK_TYPE_REINDEX.equals(task.getTaskType())) {
        processReindexTask(task);
        return;
    }

    InputStream fileStream = null;
    try {
        // 1. 下载文件
        fileStream = downloadFileFromStorage(task.getFilePath());
        if (fileStream == null) throw new IOException("流为空");
        if (!fileStream.markSupported()) {
            fileStream = new BufferedInputStream(fileStream);
        }

        // 2. 解析文件
        parseService.parseAndSave(task.getFileMd5(), fileStream,
            task.getUserId(), task.getOrgTag(), task.isPublic());

        // 3. 向量化
        VectorizationService.VectorizationUsageResult vectorizationResult =
            vectorizationService.vectorizeWithUsage(
                task.getFileMd5(), task.getUserId(), task.getOrgTag(),
                task.isPublic(), task.getUserId()
            );

        // 4. 标记完成
```

```

        documentService.markVectorizationCompleted(task.getFileMd5(), vectorizationResult);
    } catch (Exception e) {
        // 5. 标记失败 + 让 Kafka 重试
        documentService.markVectorizationFailed(task.getFileMd5(), e);
        throw new RuntimeException("Error processing task", e);
    } finally {
        if (fileStream != null) try { fileStream.close(); } catch (IOException e) { ... }
    }
}

```

3.1 @KafkaListener : Spring Kafka 的核心注解

```

@KafkaListener(topics = "...", groupId = "...")
public void processTask(FileProcessingTask task) { ... }

```

Spring Kafka 自动：

1. 创建消费者 (`KafkaConsumer`)。
2. **JSON** 反序列化 `task` 参数 (`pai-smart` 配置了 `JsonDeserializer`)。
3. 调用方法。
4. 方法正常返回 → 自动提交 **offset**。
5. 方法抛异常 → 不提交 **offset**，触发重试。

面试考点 3： `groupId` 是什么？

A：消费者组。同一 **group** 的多个实例共同消费一个 **topic** —— 一个 **partition** 只能被 **group** 内一个实例消费。

- 扩展消费者：加机器即可，**Kafka** 自动 **rebalance**。
- 多个 **group**：每个 group 都消费全量——类似「发布订阅」。

3.2 downloadFileFromStorage : MinIO / 文件系统 / URL 三合一

```

private InputStream downloadFileFromStorage(String filePath) {
    // 1. 文件系统路径
    File file = new File(filePath);
    if (file.exists()) return new FileInputStream(file);

    // 2. HTTP/HTTPS URL
    if (filePath.startsWith("http://") || filePath.startsWith("https://")) {
        URL url = new URL(filePath);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        // ...
        return conn.getInputStream();
    }

    throw new IllegalArgumentException("Unsupported file path format: " + filePath);
}

```

这是「适配器模式」的简化版——一个方法支持三种来源。

生产环境的问题：

- **MinIO** 没有专门分支？看代码——**MinIO** 流走 "filePath 是 **MinIO presigned URL**" 那个分支。但没看到显式判断。

- 可能是后期重构留下的——早期代码支持本地文件，后期支持 MinIO，但代码没更新注释。

这种"代码赶不上注释"的情况在老项目里非常常见。

3.3 失败处理：标记 + 重抛

```
} catch (Exception e) {  
    documentService.markVectorizationFailed(task.getFileMd5(), e);  
    throw new RuntimeException("Error processing task", e);  
}
```

两件事：

1. 业务失败标记：`FileUpload.vectorizationStatus = FAILED`。
2. 重抛异常：让 Spring Kafka 触发重试 / 死信。

业务失败和重试是两件事——都做：

- 业务失败让用户能在 UI 看到「上传失败」状态。
- 重试让 transient 错误（网络抖动、ES 临时挂）能恢复。

【四、失败重试 + 死信队列（DLT）】

`pai-smart` 的 `KafkaConfig` 应该配了 **DefaultErrorHandler**：

```
@Bean  
public DefaultErrorHandler errorHandler(KafkaTemplate<String, String> template) {  
    return new DefaultErrorHandler(  
        new DeadLetterPublishingRecoverer(template,  
            (record, ex) -> new TopicPartition("file-processing-dlt", record.partition())),  
        new ExponentialBackOff(1000, 2.0) // 初始 1s, 翻倍  
    );  
}
```

工作流程：

1. 消息处理失败 → 等待 1s。
2. 再处理 → 还失败 → 等待 2s。
3. 再处理 → 还失败 → 等待 4s。
4. ... 重试 N 次后（默认 10 次）。
5. 送到 **DLT**（Dead Letter Topic，死信队列）。

死信队列：

- 保留所有失败消息——人工排查。
- 可以重放——修好 bug 后重投回主 **topic**。

面试考点 4：为什么不直接重试无限次？

A：

- **bug** 不会自动修好：如果是代码 bug，重试 **100** 次也是 **fail**。
- 占资源：retry 消息占着 **partition**，新消息进不来。
- 消费阻塞：单个 partition 顺序处理，一条卡住后面都卡。

pai-smart 选「指数退避 + 死信」是正确选择。

【五、状态机：上传 → 解析 → 完成】

```
// 1. 开始处理
documentService.markVectorizationProcessing(task.getFileMd5(), false);

// 2a. 成功
documentService.markVectorizationCompleted(task.getFileMd5(), vectorizationResult);

// 2b. 失败
documentService.markVectorizationFailed(task.getFileMd5(), e);
```

FileUpload 实体里的状态机（第 5 集讲过）：

- **PENDING** → **PROCESSING** → **COMPLETED** / **FAILED**

状态机的关键：

- **UI** 可以轮询——**GET /documents/{fileMd5}/status** 拿到当前状态。
- 失败可重试——**POST /documents/{fileMd5}/reindex** 重新发 Kafka 消息。

pai-smart 后期加了 **markVectorizationProcessing(..., false)** 第二个参数——应该是「是否静默」标记——避免某些场景下重复发通知。

【六、**KafkaConfig**：生产者消费者配置】

虽然没贴完整代码，但标准配置应该包括：

```
@Configuration
public class KafkaConfig {

    @Value("${kafka.bootstrap-servers}")
    private String bootstrapServers;
```

```

@Bean
public ProducerFactory<String, FileProcessingTask> producerFactory() {
    Map<String, Object> config = new HashMap<>();
    config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
    config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
    config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
    config.put(ProducerConfig.ACKS_CONFIG, "all"); // 强持久化
    config.put(ProducerConfig.RETRIES_CONFIG, 3);
    return new DefaultKafkaProducerFactory<>(config);
}

@Bean
public KafkaTemplate<String, FileProcessingTask> kafkaTemplate() {
    return new KafkaTemplate<>(producerFactory());
}

@Bean
public ConsumerFactory<String, FileProcessingTask> consumerFactory() {
    Map<String, Object> config = new HashMap<>();
    config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
    config.put(ConsumerConfig.GROUP_ID_CONFIG, "file-processing-group");
    config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
    config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);
    config.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
    return new DefaultKafkaConsumerFactory<>(config);
}

@Bean
public ConcurrentKafkaListenerContainerFactory<String, FileProcessingTask> kafkaListener
ContainerFactory() {
    ConcurrentKafkaListenerContainerFactory<String, FileProcessingTask> factory =
        new ConcurrentKafkaListenerContainerFactory<>();
    factory.setConsumerFactory(consumerFactory());
    factory.setConcurrency(3); // 3 个并发消费者
    factory.setCommonErrorHandler(errorHandler(kafkaTemplate()));
    return factory;
}
}

```

几个关键配置：

- **acks: "all"**：生产者等所有副本都收到。强持久化。
- **auto.offset.reset: "earliest"**：新 group 从头开始消费。避免漏消息。
- **concurrency: 3**：单实例起 3 个消费者线程。水平扩展的最小单元。

面试考点 5：acks 的几个级别？

A：

- **acks=0**：fire and forget，可能丢。
- **acks=1**：leader 收到就返回，**follower** 没同步可能丢。
- **acks=all**：所有 in-sync 副本都收到，强一致。

`pai-smart` 选 `all` 是正确选择——文件处理任务不能丢。

【七、Producer 在哪发消息？】

在 `DocumentService` 或 `UploadController` 里：

```
@Service
public class DocumentService {

    @Autowired
    private KafkaTemplate<String, FileProcessingTask> kafkaTemplate;

    @Autowired
    private KafkaConfig kafkaConfig;

    public void uploadComplete(String fileMd5, String fileName, ...) {
        // 1. 落 FileUpload 记录
        // 2. 发 Kafka 消息
        FileProcessingTask task = new FileProcessingTask();
        task.setFileMd5(fileMd5);
        task.setUserId(userId);
        // ...
        kafkaTemplate.send(kafkaConfig.getFileProcessingTopic(), fileMd5, task); // 用
        fileMd5 当 key
    }
}
```

用 `fileMd5` 当 `key`：

- 同一文件的多个消息进同一 **partition**——顺序处理。
- 重试时也是同一 **partition**——避免并发问题。

面试考点 6：Kafka 的 key 有什么用？

A：

- 相同 **key** 进相同 **partition**。
 - 相同 **partition** 顺序处理。
 - 保证同一文件的消息不乱序——重要。
-

【八、消息幂等性：重复消费怎么办？**】

Kafka 是「至少一次」语义——可能重复消费。

重复消费的场景：

1. **consumer** 提交 **offset** 前挂掉。
2. 重启后从上次 **offset** 重新消费。

pai-smart 的解法：

- **FileUpload** 状态机：**PROCESSING** 状态的消息直接跳过（说明其他 consumer 正在处理）。
- **DocumentVector** 表 UNIQUE 约束：重复插入忽略。
- 业务操作幂等（**markVectorizationCompleted** 检查状态）。

面试考点 7：幂等的三种实现方式？

A：

- **DB** 唯一约束：最可靠。
- **Redis SETNX**：用于分布式锁。
- 业务状态机：状态判断防重。

pai-smart 用了第 1 + 第 3 种——最稳。

【九、WebSocket 推送：状态变化通知前端】

pai-smart 后面的版本里，用户上传文件后，前端能实时看到「处理中 → 已完成」——这是 WebSocket 推送的（第 10 集讲）。

Kafka + WebSocket：

1. Kafka consumer 处理完 → 调 **ChatSessionRegistry** 或 **WebSocketSession** 推送。
 2. 前端无需轮询——体验流畅。
-

【十、常见坑 & 面试问答】

Q1：消息顺序重要吗？

A：**pai-smart** 是单文件独立处理，不同文件间不要求顺序。用 **fileMd5** 当 **key**，同一文件的消息进同一 **partition**——保证同文件顺序。

Q2：**Kafka** 集群挂了怎么办？

A：消息会堆积（在 partition 里）。Kafka 恢复后消费者从堆积处继续。**pai-smart** 有 **DLT**——长期堆积会触发告警。

Q3：怎么处理 **Kafka** 消息体里的敏感信息？

A：当前消息体只有 **fileMd5** / **userId** / **orgTag** ——无敏感信息。不要把密码、**token** 放消息——Kafka 不是加密存储。

Q4：为什么不用线程池替代 **Kafka**？

A：

- 单实例线程池够用——但多实例怎么办？Kafka 解决「多实例消费同一 topic 的并发问题」。
- 持久化——线程池消息在内存——重启就丢。
- 重平衡——线程池做不到。

Q5：怎么监控 **Kafka** 消费 lag？

A：

- **Kafka** 自带：`kafka-consumer-groups.sh --describe`。
- **Spring Boot Actuator**：暴露 `/actuator/metrics`。
- 告警：用 `kafka-lag-exporter` + Prometheus + Grafana。

【十一、思考题：为什么不用 `ThreadPoolTaskExecutor` 走轻量路线？**

轻量方案：

```
@Async
public void processFileAsync(String fileMd5) { ... }
```

优点：代码少，没有外部依赖。

缺点：

- **Web** 应用重启 → 任务全丢。
- 多实例部署 → 每个实例都收到 HTTP 请求，重复处理。
- 背压控制弱。

`pai-smart` 选 Kafka—— 生产级异步。**KISS** 原则 **vs** 生产级 ——业务量小可以 `ThreadPoolTaskExecutor`，业务量大必须 **Kafka**。

【十二、下集预告】

文档能上传、解析、向量化、写入 ES 了。

但是！用户怎么问“AI”？

第 10 集我们要：

- 引入 **WebSocket**——`ChatWebSocketHandler` 接收用户消息。
- 流式调用 **DeepSeek**——AI 回答一个 token 一个 token 推。
- 集成 `ChatHandler` 编排 RAG 流程。

RAG 系统的"对话能力"登场。

我们下期见。

这一期的「异步化」是后端工程师分水岭级别的技能。

觉得有用，点赞、投币、收藏。

下一集链接：[第 10 集：WebSocket 流式聊天——DeepSeek 首次对话](#)

第 10 集：WebSocket 流式聊天——DeepSeek 首次对话

系列：派聪明 RAG 后端演进全解

集数：10 / 20+

主题：WebSocket 握手鉴权、SSE 流式响应、DeepSeek Client、Reactive Streams

上集回顾：第 09 集 Kafka 异步化完成，文档处理流水线跑通

本集目标：用户能通过 WebSocket 提问，AI 一个 token 一个 token 推回前端

【开场 Hook】

文档能存了，能搜了，但 **RAG** 还差最后一步——回答。

`pai-smart` 用的是 **DeepSeek**（国产 LLM，对标 **GPT-4** 级别，**API** 便宜）。

但是，**HTTP** 请求是“一问一答”——用户问完得等 DeepSeek 把所有 token 生成完才能看到。等 **5-10** 秒才有反应——体验糟糕。

WebSocket 解决了这个：

1. 客户端和服务端建立长连接。
2. 用户提问 → 服务端调 DeepSeek → 一个 **token** 一个 **token** 推回前端。
3. 用户能看到 **AI** “打字”的过程——类 **ChatGPT** 体验。

今天我们拆 `ChatWebSocketHandler` 和 `DeepSeekClient`，看 RAG 系统的“对话能力”怎么实现。

【一、为什么是 WebSocket 不是 SSE？】

两种“服务器推送”方案：

维度	WebSocket	SSE (Server-Sent Events)
协议	<code>ws://</code> / <code>wss://</code>	<code>http://</code>
方向	双向	单向 (服务器→客户端)
数据格式	任意 (文本/二进制)	文本 (<code>data: ... \n\n</code>)
心跳	手动实现	自动 (注释行)
浏览器支持	100%	100% (除 IE)
代理穿透	偶尔需要配置	更友好
Spring 支持	<code>spring-websocket</code>	<code>spring-webmvc</code> 内置

`pai-smart` 选 WebSocket——因为后续要做「客户端发心跳 + 服务端推送」这种双向交互。

面试考点 1：WebSocket 握手时怎么鉴权？

A：Sec-WebSocket-Protocol 头或 URL 参数：

```
ws://localhost:8080/chat?token=eyJhbGciOiJIUzI1NiJ9...
```

`pai-smart` 是 URL 参数——看 `extractToken(session)` 方法。

【二、ChatWebSocketHandler：连接生命周期管理】

2.1 连接建立：握手 + 鉴权

```
@Override
public void afterConnectionEstablished(WebSocketSession session) {
    String jwtToken;
    try {
        jwtToken = extractToken(session);
        if (!jwtUtils.validateToken(jwtToken)) {
            session.close(CloseStatus.POLICY_VIOLATION);
            return;
        }
    } catch (Exception exception) {
        session.close(CloseStatus.POLICY_VIOLATION);
        return;
    }

    String userId = extractUserId(jwtToken);
    String clientId = extractClientId(session);
```

```

session.getAttributes().put(ATTR_USER_ID, userId);
chatSessionRegistry.registerSession(userId, clientId, session);

// 发送 connection 消息
Map<String, String> connectionMessage = Map.of(
    "type", "connection",
    "sessionId", session.getId(),
    "message", "WebSocket连接已建立"
);
session.sendMessage(new TextMessage(objectMapper.writeValueAsString(connectionMessage)))
;
}

```

关键点：

1. 握手时鉴权——`validateToken` 失败直接 `POLICY_VIOLATION`。
2. 从 **token** 拿 **userId**——之后消息都关联到 **userId**。
3. **clientId**：一个用户可能开多个客户端（手机、PC），每个客户端一个 **clientId**。
4. `session.getAttributes()` ——存 **userId**，之后消息处理直接读。
5. **ChatSessionRegistry**：内存里的 **session** 注册表——`userId -> Map<clientId, WebSocketSession>`。用来主动推送（第 13 集限流告警推送会用到）。
6. 发送 **connection** 消息——前端收到后知道连接就绪，可以开始发消息。

面试考点 2：`session.getAttributes()` 是什么？

A：**WebSocket session** 级别的 **Map** ——存这个连接相关的元数据。类似 **HttpSession** 的 **attributes**。注意：**WebSocketSession** 不是 **HttpSession**——生命周期、存储都不一样。

2.2 心跳机制

```

private static final String HEARTBEAT_PING = "__chat_ping__";
private static final String HEARTBEAT_PONG = "__chat_pong__";

@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) {
    String payload = message.getPayload();
    if (HEARTBEAT_PING.equals(payload)) {
        session.sendMessage(new TextMessage(HEARTBEAT_PONG));
        return;
    }
    // ... 进入聊天处理
}

```

前端定时（30s）发 `__chat_ping__` ——后端回 `__chat_pong__` ——连接保活。

为什么需要心跳？

- Nginx 默认 60s 切断空闲 **WebSocket**。

- 企业网关 / 防火墙也会超时切断。
- 心跳让中间设备知道连接活着。

面试考点 **3**：心跳的最佳间隔？

A：

- 太短 (< 10s)：浪费带宽、电池。
- 太长 (> 60s)：Nginx 切了不知道。
- 经验值：**30-60s**。

pai-smart 用协议字符串做心跳——比 **ping/pong** 帧更直接（**WebSocket** 协议自带 **ping/pong** 帧——**Spring** 抽象了）。

2.3 业务消息处理

```
@Override
protected void handleTextMessage(WebSocketSession session, TextMessage message) {
    String userId = extractUserId(extractToken(session));
    String payload = message.getPayload();
    if (HEARTBEAT_PING.equals(payload)) {
        session.sendMessage(new TextMessage(HEARTBEAT_PONG));
        return;
    }

    // 1. 解析 JSON (前端发 { type, content, ... })
    // 2. 提取 userMessage
    // 3. 调 ChatHandler 处理
    chatHandler.processMessage(userId, userMessage, session);
}
```

ChatHandler.processMessage(userId, userMessage, session) ——第 **15** 集会详细讲 **ReAct** 循环。这里关键是 **session** 传进去——**ChatHandler** 通过 **session** 推流。

【三、DeepSeekClient：流式调 LLM】

```
public void streamResponse(String requesterId, String userMessage, String context,
                           List<Map<String, String>> history,
                           Consumer<String> onChunk, Consumer<Throwable> onError) {
    Map<String, Object> request = buildRequest(userMessage, context, history);
    List<Map<String, String>> messages = (List<Map<String, String>>) request.get("messages");
    ;

    // 1. 配额预留
    int estimatedPromptTokens = usageQuotaService.estimateChatTokens(messages);
    int maxCompletionTokens = aiProperties.getGeneration().getMaxTokens() != null
        ? aiProperties.getGeneration().getMaxTokens() : 2000;
    UsageQuotaService.TokenReservation reservation = usageQuotaService.reserveLlmTokens(
```

```

        requesterId, estimatedPromptTokens, maxCompletionTokens);
    StreamUsageTracker usageTracker = new StreamUsageTracker(reservation, estimatedPromptTokens);

    try {
        // 2. 流式调用
        webClient.post()
            .uri("/chat/completions")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(request)
            .retrieve()
            .bodyToFlux(String.class) // <-- Flux = 流
            .subscribe(
                chunk -> processChunk(chunk, usageTracker, onChunk),
                error -> { settleUsage(usageTracker); onError.accept(error); },
                () -> settleUsage(usageTracker)
            );
    } catch (Exception e) {
        usageQuotaService.abortReservation(reservation);
        throw e;
    }
}

```

3.1 `bodyToFlux(String.class)` : 流式响应的关键

Mono VS Flux :

- **Mono<T>** : 0 或 1 个元素 (单值)。
- **Flux<T>** : 0 到 N 个元素 (流)。

DeepSeek 的 `chat/completions` 接口当 `stream: true` 时返回 **SSE**——每个 **chunk** 是一个 **JSON** :

```

data: {"choices":[{"delta":{"content":"你"}}]}

data: {"choices":[{"delta":{"content":"好"}}]}

data: {"choices":[{"delta":{"content":"!"}}]}

data: [DONE]

```

bodyToFlux 把 HTTP body 的 stream 解析成 `Flux<String>` ——每个 **data** 块是一个元素。

.subscribe(chunk, error, complete) :

- **chunk** consumer : 每个 **token** 来了怎么处理。
- **error** consumer : 出错了怎么处理。
- **complete** Runnable : 流结束了怎么处理。

整个流是非阻塞的——**WebFlux** 异步事件循环。

面试考点 4 : Flux 和 Stream 的区别 ?

A :

- **Stream** : JDK 8 , 同步拉取 (`forEach` 阻塞)。
- **Flux** : Reactive Streams , 异步推送 (`subscribe` 不阻塞)。
- **WebFlux** 必须用 **Flux**——否则失去异步优势。

3.2 `buildRequest` : 构造 `messages` 数组

```
private Map<String, Object> buildRequest(String userMessage, String context,
List<Map<String, String>> history) {
    List<Map<String, String>> messages = new ArrayList<>();

    // 1. system 角色 : 注入 RAG 召回的 context
    messages.add(Map.of(
        "role", "system",
        "content", "你是一个企业知识助手。基于以下信息回答用户问题 : \n\n" + context
    ));

    // 2. 历史消息
    if (history != null) messages.addAll(history);

    // 3. 用户当前问题
    messages.add(Map.of("role", "user", "content", userMessage));

    Map<String, Object> request = new HashMap<>();
    request.put("model", model);
    request.put("stream", true);
    request.put("messages", messages);
    request.put("temperature", 0.7);
    return request;
}
```

`pai-smart` 用的 **prompt** 结构 :

1. **system** : 注入 RAG 召回的 context——告诉 **LLM** "用这些信息回答"。
2. **history** : 之前的对话。
3. **user** : 当前问题。

这是 **RAG** 的标准 **prompt** 模式——"先给资料 , 再问问题"。

面试考点 5 : prompt 怎么写效果最好 ?

A :

- 明确角色 : 「你是一个企业知识助手」比「你是一个 AI」好。
- 明确约束 : 「仅基于以下信息回答 , 不知道就说不知道」减少幻觉。
- 明确格式 : 「用 Markdown 列表回答」「每段不超过 100 字」。
- 引用标注 : 「回答时标注信息来源 (如 来源 #1)」。

`pai-smart` 用了前 3 个——是基础实践。

3.3 配额预留：流式也走「先扣后用」

```
int estimatedPromptTokens = usageQuotaService.estimateChatTokens(messages);
int maxCompletionTokens = ...;
UsageQuotaService.TokenReservation reservation = usageQuotaService.reserveLlmTokens(
    requesterId, estimatedPromptTokens, maxCompletionTokens);
```

流式调用的配额比 Embedding 复杂——因为 LLM 还要生成：

- `estimatedPromptTokens`：system + history + user 的 token（用 `tiktoken` 类库估算）。
- `maxCompletionTokens`：LLM 最多生成多少（配置项）。
- 预留总量 = `estimatedPromptTokens + maxCompletionTokens`。

`StreamUsageTracker` 在流式过程中累加实际消耗：

- 收到每个 chunk 解析 `usage` 字段（DeepSeek 在最后一个 chunk 给 `prompt_tokens / completion_tokens`）。
- 最后 `settleUsage(usageTracker)` 按实际值结算。

面试考点 6：流式响应中怎么知道 token 用了多少？

A：DeepSeek / OpenAI 的 SSE 格式里，每个 `chunk` 有 `usage` 字段但默认是 `null`，只有最后一个 `chunk` 会带 `usage.total_tokens`。`pai-smart` 在 `processChunk` 里检测并累加。

【四、`ChatHandler.processMessage`：业务编排】

```
public void processMessage(String userId, String userMessage, WebSocketSession session) {
    String requestClientId = resolveClientId(session);
    String conversationId = null;
    String generationId = null;
    try {
        rateLimitService.checkChatByUser(userId);

        // 1. 获取或创建会话 ID
        conversationId = getOrCreateConversationId(userId);
        conversationService.ensureConversationSession(Long.parseLong(userId),
            conversationId, userMessage);
        ChatGenerationStateService.GenerationSnapshot generation =
            chatGenerationStateService.createGeneration(userId, conversationId, userMess
age);
        generationId = generation.generationId();

        // 2. 推送"开始生成"消息
        sendGenerationStart(userId, generationId, conversationId);

        // 3. 创建响应构建器
        responseBuilders.put(generationId, new StringBuilder());
        CompletableFuture<String> responseFuture = new CompletableFuture<>();
        responseFutures.put(generationId, responseFuture);
    }
```

```

// 4. 获取对话历史
List<Map<String, String>> history = conversationService.getRecentHistory(conversationId, 10);

// 5. ReAct 循环 (第 15 集细讲)
runReActLoopSafely(userId, userMessage, conversationId, generationId, history, responseFuture);

} catch (Exception e) {
    handleError(userId, generationId, e);
    sendCompletionNotification(userId, generationId, conversationId, true, false);
    cleanupGenerationState(generationId, e);
}
}

```

这期我们重点关注 **1-4** , **5** 留到第 **15** 集。

4.1 限流

```
rateLimitService.checkChatByUser(userId);
```

用户级限流——比如每分钟 **10** 次。超过就拒绝。第 **13** 集会深入讲。

4.2 会话管理

```

conversationId = getOrCreateConversationId(userId);
conversationService.ensureConversationSession(Long.parseLong(userId), conversationId, userMessage);

```

会话 = 一个连续的对话流——保留 **context**。

conversationId 怎么存？推测：

- **WebSocketSession attribute** (连接级别的 **session**)。
- **Redis** 缓存 (跨连接保留)。

pai-smart 在 **getOrCreateConversationId** 里根据 **session attribute** 判断——新连接开新会话。

4.3 Generation : 一次"生成任务"的概念

```

ChatGenerationStateService.GenerationSnapshot generation =
    chatGenerationStateService.createGeneration(userId, conversationId, userMessage);
generationId = generation.generationId();

```

一次用户提问 → 一个 **generation**。

为什么需要这个抽象？

- 多轮 **ReAct**：一个用户问题可能要调 LLM 多次（第 15 集）。
- 状态追踪：哪个 generation 在跑、跑多久、用了多少 token、是否完成。
- 取消支持：用户主动取消，**generation** 状态变 **CANCELLED**。

ChatGenerationStateService 用 Redis 维护这个状态机——分布式——多实例都能查到。

4.4 历史获取

```
List<Map<String, String>> history = conversationService.getRecentHistory(conversationId, 10);
```

取最近 10 条对话作为上下文。为什么是 10？

- **DeepSeek context** 长度：~32K tokens。
- 10 条对话 ≈ **2000 tokens**——留出 LLM 生成的余量。
- 更多历史会“挤掉”当前问题的 **attention**。

面试考点 7：长对话怎么处理？

A：

- 滑动窗口：只保留最近 N 条。
- 摘要压缩：用 LLM 把早期对话总结成几句话。
- 层次化记忆：把对话分成“主线程”和“分支线程”。

pai-smart 用滑动窗口——最简方案。

【五、流式推回前端：**appendStreamChunk**】

```
private void appendStreamChunk(String userId, String generationId, String conversationId, String chunk) {
    if (chunk == null || chunk.isEmpty()) return;
    StringBuilder builder = responseBuilders.get(generationId);
    if (builder != null) builder.append(chunk);

    WebSocketSession session = chatSessionRegistry.getActiveSession(userId, requestClientId);
    if (session != null && session.isOpen()) {
        Map<String, Object> msg = Map.of(
            "type", "chunk",
            "generationId", generationId,
            "content", chunk
        );
        session.sendMessage(new TextMessage(objectMapper.writeValueAsString(msg)));
    }
}
```

每个 **token** 推一次。

前端协议（**type** 字段）：

- **connection**：连接建立。
- **start**：开始生成。
- **chunk**：流式 token。
- **done**：生成完成。
- **error**：出错。
- **tool_call**：Agent 工具调用（第 15 集）。

WebSocket 是 **message** 边界——每个 **sendMessage** 是一帧——前端要自己拼接。

面试考点 8：流式响应怎么保证顺序？

A：**WebSocket** 本身保证消息顺序——**sendMessage(A)** 一定在 **sendMessage(B)** 之前到。**TCP** 有序性。

【六、心跳之外的"服务端主动推送"】

pai-smart 的 **WebSocket** 不只是「客户端提问 → 服务端回答」——还能主动推送：

1. 处理进度通知（文档处理完成、生成结束）。
2. 限流告警（"你今天用了 90% 配额"）。
3. 系统公告（"系统维护通知"）。

ChatSessionRegistry 是关键——**userId -> Map<clientId, WebSocketSession>**。

主动推送流程：

```
// 1. 业务事件发生
fileVectorizationCompleted(fileMd5);

// 2. 找出这个用户的所有 session
Map<String, WebSocketSession> sessions = chatSessionRegistry.getSessions(userId);

// 3. 推消息
for (WebSocketSession session : sessions.values()) {
    if (session.isOpen()) {
        session.sendMessage(new TextMessage(notification));
    }
}
```

【七、WebSocket 的多实例问题】

场景：用户 A 第一次连 WebSocket 到实例 1，第二次到实例 2。

问题：实例 2 不知道 A 的 session 状态。

pai-smart 的解法：

- **ChatSessionRegistry** 是实例级别的——**ConcurrentHashMap**——不跨实例。
- 靠 **sticky session** (Nginx **ip_hash**) 让用户总连同一实例。

生产级方案：

- **Redis Pub/Sub**：实例 1 处理完事件 → 发 Redis 消息 → 所有实例订阅 → 各自动作。
- **Kafka + WebSocket Gateway**：专门的推送服务。

pai-smart 当前用简化方案——单实例够用。

面试考点 9：WebSocket 多实例怎么实现广播？

A：

- **Redis Pub/Sub**：**redisTemplate.convertAndSend("event", message)**，所有实例订阅。
- **Kafka topic**：发到 Kafka，所有实例消费。
- 专用 **WebSocket** 网关：所有长连接挂在网关，业务无状态。

【八、**WebSocketConfig**：注册 handler】

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(chatWebSocketHandler, "/chat")
            .setAllowedOriginPatterns("*");
    }
}
```

/chat 是 **WebSocket** 端点——前端用 **ws://host:8080/chat?token=xxx** 连。

setAllowedOriginPatterns("*") ——生产环境要限制为前端域名。

【九、常见坑 & 面试问答】

Q1：WebSocket 和 HTTP 共享端口吗？

A：是的。Tomcat 监听 8080，**HTTP** 和 **WebSocket** 共用。路径区分：`/chat` → WS，`/api/...` → HTTP。

Q2：WebSocket 连接断了怎么重连？

A：前端用 `reconnect-websocket` 库：

```
const ws = new ReconnectingWebSocket('ws://...', [], {
  maxRetries: 10,
  retryDecay: 1.5
});
```

重连后重新走握手 + 鉴权。

Q3：WebSocket 怎么和 HTTP 鉴权统一？

A：复用 **JWT**。**WebSocket** 握手时带 `Authorization` 头（虽然 **WebSocket spec** 没要求 `Authorization` 头，实际可用）或 URL 参数**。

`pai-smart` 用 **URL** 参数——`?token=xxx`——简单。

Q4：流式响应怎么知道“全部推完了”？

A：DeepSeek 的 SSE 末尾有 `data: [DONE]`。`pai-smart` 在 `processChunk` 里检测：

```
if (chunk.contains("[DONE]")) {
  // 流结束
}
```

Q5：单条消息太大 WebSocket 会断开吗？

A：单帧上限 **16KB**（WebSocket spec）。`pai-smart` 的 **chunk** 是单个 **token**——几字节到几十字节——远不到上限。

【十、思考题：`pai-smart` 为什么要拆 `ChatHandler` 和 `ChatWebSocketHandler`？**

当前结构：

- `ChatWebSocketHandler`：WebSocket 协议层（握手、心跳、消息分发）。
- `ChatHandler`：业务逻辑层（限流、会话、**ReAct**、调 **LLM**）。

为什么拆？

- **SRP**：每个类一个职责。
- 可测试：`ChatHandler` 不依赖 WebSocket——可以 **mock session** 单测。
- 可替换协议：以后接 gRPC-Web 或 SSE，只换 **handler**，业务不动。

这是好设计——比"一个上帝类"好太多。

【十一、下集预告】

RAG 流水线基本跑通了：上传 → 解析 → 向量化 → 检索 → **LLM** 回答。

但！所有用户共享同一个文档库——公司 **A** 的机密文档，公司 **B** 的员工也能搜到。

这不行。

第 11 集我们要：

- `OrganizationTag` 体系——多租户隔离。
- `OrgTagAuthorizationFilter` ——**URL** 级权限控制。
- `OrgTagCacheService` ——缓存 **org tag**。
- 用户注册时默认打 **DEFAULT tag**。

多租户是 **SaaS** 系统的基石——`pai-smart` 的实现值得学。

我们下期见。

这一期的 WebSocket + WebFlux 流式是现代化 **RAG** 系统的标配。
觉得有用，点赞、投币、收藏。

下一集链接：[第 11 集：多租户隔离——OrgTag 体系](#)

第 11 集：多租户隔离——OrgTag 体系

系列：派聪明 RAG 后端演进全解
集数：11 / 20+
主题：多租户架构、OrgTag 树形层级、OrgTagAuthorizationFilter、PRIVATE_ 前缀
上集回顾：第 10 集 WebSocket + DeepSeek 流式聊天跑通
本集目标：公司 A 的文档，公司 B 的员工搜不到

【开场 Hook】

想象一个场景：

公司 A 上传了他们的内部财务文档到 `pai-smart`。
公司 B 的员工登录后，也能搜到。

这是数据泄露事故——轻则客户流失，重则被起诉。

RAG 系统只要有「多用户」就有「多租户」问题。`pai-smart` 的解法是 **OrgTag**（组织标签）——给用户和文档打标签，标签匹配才能访问。

这是 SaaS 系统的基石。今天我们拆 `OrganizationTag` 实体、`OrgTagAuthorizationFilter` 过滤器、还有 `OrgTagCacheService` 缓存层。

【一、什么是多租户？三种隔离模型】

多租户（**Multi-Tenancy**）= 一套系统服务多个客户（租户），数据要严格隔离。

三种隔离模型：

模型	实现	优点	缺点
独立数据库	每个租户一个 DB	强隔离，故障不串	运维贵，连接数多
共享数据库 + 独立 Schema	同 DB 不同 schema	中等隔离	跨租户查询难
共享数据库 + 共享 Schema	同表加 <code>tenant_id</code> 列	便宜	风险高，应用层必须强校验

`pai-smart` 选第三种——所有租户共享 **MySQL** 和 **ES**，靠 `orgTag` 字段做过滤。

这是 **SaaS** 系统的常见选择——成本最低，但应用层要非常小心。

面试考点 1：怎么保证"应用层不漏过"？

A：

- **AOP** 拦截：@PreAuthorize + SpEL 强制拼 orgTag。
- **Filter** 拦截：OrgTagAuthorizationFilter URL 级校验。
- **Repository** 封装：UserRepository.findXxxByOrgTag(...) —— 业务代码只能调这种"已经带过滤"的方法。
- **DB** 权限：MySQL Row-Level Security (**MySQL 8.0+** 支持)。

pai-smart 用了前 3 种。

【二、OrganizationTag：组织标签的实体设计】

```
@Data
@Entity
@Table(name = "organization_tags")
public class OrganizationTag {
    @Id
    @Column(name = "tag_id")
    private String tagId; // 标签唯一标识

    @Column(nullable = false)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "parent_tag", length = 255)
    private String parentTag; // 父标签 ID

    @Column(name = "upload_max_size_bytes")
    private Long uploadMaxSizeBytes; // 非管理员上传文件大小上限

    @ManyToOne
    @JoinColumn(name = "created_by", nullable = false)
    private User createdBy;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;
}
```

关键设计点：

1. tagId 是 String：是「业务主键」，不用自增 Long——方便跨系统传递。

2. `parentTag` : 树形结构! `ROOT` → 公司A → 部门X ——继承关系。
3. `uploadMaxSizeBytes` : 不同租户不同配额——业务规则硬编码到 `tag` 上。
4. `createdBy` : 谁建的——审计。

面试考点 2 : 为什么用树形结构?

A : 权限继承。公司 A 的人可以访问「公司A」tag 下的所有子部门文档——不用每个部门分别授权。

树形结构的设计 :

- `parentTag` 是 String 而不是 `@ManyToOne` ——避免 **N+1** 查询。
- 父级 `tagId` 直接存字符串——取整棵树要递归 (用 `OrgTagCacheService` 缓存)。

Hibernate 里的 `@ManyToOne` :

- `User createdBy` 用了 `@ManyToOne` ——懒加载。
- 访问 `orgTag.getCreatedBy()` 会自动加载 User——额外一次 SQL。
- **N+1** 风险——批量查 `OrganizationTag` 列表时, 每个 `tag` 都触发一次 User 加载。pai-smart 应该在 `OrgTagRepository` 用 `JOIN FETCH` ——这里没看到。

【三、PRIVATE_ 前缀 : 私人文档的特殊处理】

```
private static final String PRIVATE_TAG_PREFIX = "PRIVATE_";
```

pai-smart 的私有文档设计 :

- 公开文档 : `orgTag = "DEFAULT"` 或 `isPublic = true`。
- 私人文档 : `orgTag = "PRIVATE_<userId>"`。

这样设计的好处 :

- 私人文档用一个字段标识, 不用单独建表。
- 过滤逻辑简单 : `orgTag LIKE 'PRIVATE_<myUserId>' OR isPublic = true OR orgTag IN (myOrgTags)`。
- 不需要复杂的 `owner` 字段。

面试考点 3 : 为什么不直接用 `userId` 字段?

A :

- `userId` 字段需要额外的 `owner` 关系。
- `orgTag LIKE 'PRIVATE_<userId>'` 可以复用 `orgTag` 的所有过滤逻辑。
- 私人文档也是一种“组织”——**PRIVATE_** 就是一个特殊组织。

这是「用统一抽象解决一类问题」的好例子。

【四、OrgTagAuthorizationFilter : URL 级权限控制】

```
@Component
public class OrgTagAuthorizationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain) {
        try {
            String path = request.getRequestURI();

            // 路径模式匹配
            if (path.matches(".*upload/chunk.*") || ... ) {
                // 这些 API 只需要 userId, 不做资源级权限检查
                String token = extractToken(request);
                if (token != null) {
                    String userId = jwtUtils.extractUserIdFromToken(token);
                    String role = jwtUtils.extractRoleFromToken(token);
                    String orgTags = jwtUtils.extractOrgTagsFromToken(token);
                    request.setAttribute("userId", userId);
                    request.setAttribute("role", role);
                    request.setAttribute("orgTags", orgTags);
                }
                filterChain.doFilter(request, response);
                return;
            }

            String resourceId = extractResourceIdFromPath(request);
            if (resourceId == null) {
                filterChain.doFilter(request, response);
                return;
            }

            ResourceInfo resourceInfo = getResourceInfo(resourceId);
            if (resourceInfo == null) {
                response.setStatus(HttpServletResponse.SC_NOT_FOUND);
                return;
            }

            String resourceOrgTag = resourceInfo.getOrgTag();

            // 公开/默认 tag → 放行
            if (resourceInfo.isPublic() || resourceOrgTag == null || resourceOrgTag.isEmpty(
) ||

                "DEFAULT".equals(resourceOrgTag)) {
                filterChain.doFilter(request, response);
                return;
            }

            // 鉴权
            if (!isUserAuthorized(...)) {
                response.setStatus(HttpServletResponse.SC_FORBIDDEN);
                return;
            }
        }
    }
}
```

```

        filterChain.doFilter(request, response);
    } catch (Exception e) {
        logger.error("...", e);
    }
}
}

```

4.1 两种 URL 的处理方式

第一类：不需要资源级权限的 **API**——只设置 `userId` 给 controller 用：

```

if (path.matches(".*upload/chunk.*") ||
    path.matches(".*upload/merge.*") ||
    path.matches(".*documents/uploads.*") ||
    path.matches(".*search/hybrid.*") || ...) {
    // 只设置 request attribute, 不做权限检查
    request.setAttribute("userId", userId);
    filterChain.doFilter(request, response);
    return;
}

```

这种 **API** 怎么鉴权？

- 业务层自己处理——`searchService.searchWithPermission(query, userId, topK)`。
- **Filter** 只负责「告诉你 `userId` 是谁」。

第二类：需要资源级权限的 **API**——鉴权：

```

String resourceId = extractResourceIdFromPath(request);
ResourceInfo resourceInfo = getResourceInfo(resourceId);
if (resourceInfo == null) {
    response.setStatus(404);
    return;
}
if (resourceInfo.isPublic() || "DEFAULT".equals(resourceInfo.getOrgTag())) {
    filterChain.doFilter(request, response);
    return;
}
if (!isUserAuthorized(...)) {
    response.setStatus(403);
    return;
}
filterChain.doFilter(request, response);

```

`extractResourceIdFromPath`：从 URL 里抽出 `fileMd5` —— `/documents/{fileMd5}/preview` → `{fileMd5}`。

`getResourceInfo`：查 DB / 缓存拿到文档的 `orgTag` 和 `isPublic`。

isUserAuthorized：判断用户能否访问——核心逻辑是 `resourceOrgTag IN (userOrgTags) OR resourceOrgTag LIKE 'PRIVATE_<userId>'`。

4.2 性能优化：缓存 org tag

OrgTagCacheService：把用户的 org tag 关系缓存到内存：

```
// 推测实现
@Service
public class OrgTagCacheService {
    private final Cache<String, Set<String>> userOrgTagsCache; // userId -> orgTags (含父级)

    public Set<String> getUserEffectiveOrgTags(String userId) {
        Set<String> tags = userOrgTagsCache.getIfPresent(userId);
        if (tags == null) {
            tags = loadFromDb(userId); // 查 DB + 递归找父级
            userOrgTagsCache.put(userId, tags);
        }
        return tags;
    }
}
```

为什么需要缓存？

每次请求都查 DB 计算 org tag 太慢——而且 org tag 不常变——缓存是必须的。

用什么缓存？

- **Caffeine** (Java 本地内存缓存)：pai-smart 大概率用的这个。
- **Redis**：跨实例，但延迟比本地缓存高。

面试考点 4：Caffeine vs Redis 缓存怎么选？

A：

- **Caffeine**：单实例，微秒级，不跨实例——适合 org tag 这种每实例一份的小数据。
- **Redis**：跨实例，毫秒级——适合会话、限流这种共享数据。

【五、User.orgTags 和 primaryOrg 的设计】

回顾第 02 集的 User 实体：

```
@Column(name = "org_tags")
private String orgTags; // 用户所属组织标签，多个用逗号分隔

@Column(name = "primary_org")
private String primaryOrg; // 用户主组织标签
```

两个字段：

- `orgTags`：用户所有所属组织（用逗号分隔）。
- `primaryOrg`：用户的主组织（默认显示用）。

为什么分开？

A：

- `primaryOrg` 用于默认显示——前端展示「当前组织」。
- `orgTags` 用于实际过滤——所有匹配的 tag 都参与。
- 用户可能属于多个组织（跨部门协作）—— `primaryOrg` 只是“主”的。

问题：还是用 String 存逗号分隔——没有正经的多对多表。业务简单可以接受。

面试考点 5：为什么不建 `user_org_tag` 关联表？

A：

- 简单：不用多一张表。
- 够用：业务上很少需要 **JOIN**。
- 代价：关联查询 `SELECT * FROM user WHERE orgTag = 'X'` 要 `LIKE '%X%'` ——慢。

【六、检索时的权限过滤：HybridSearchService】

```
public List<SearchResult> searchWithPermission(String query, String userId, int topK) {
    // 1. 取用户有效 org tags
    List<String> userEffectiveTags = getUserEffectiveOrgTags(userId);

    // 2. 构造 ES bool filter
    BoolQuery.Builder boolBuilder = new BoolQuery.Builder();

    // 公开 OR 用户的 org tags OR 用户的私人
    boolBuilder.filter(f -> f.bool(b -> b
        .should(s -> s.term(t -> t.field("isPublic").value(true)))
        .should(s -> s.terms(t -> t.field("orgTag").terms(tt -> tt.value(
            userEffectiveTags.stream().map(FieldValue::of).toList()
        ))))
        .should(s -> s.term(t -> t.field("userId").value(userId)))
        .minimumShouldMatch("1")
    ));

    // 3. 调 ES 搜索
    SearchResponse<EsDocument> response = esClient.search(s -> {
        s.index("knowledge_base");
        s.query(q -> q.bool(boolBuilder.build()));
        s.knn(kn -> kn.field("vector").queryVector(queryVector).k(topK * 30).numCandidates(topK * 30));
        // ...
    }, EsDocument.class);
```

```
return parseResults(response);
}
```

should + minimumShouldMatch("1") 的逻辑：

- 公开 (`isPublic = true`) OR
- 匹配 **org tag** (`orgTag IN userEffectiveTags`) OR
- 自己上传的 (`userId = currentUser`)

三个条件满足任一即可。

面试考点 6：ES 的 **should** 默认行为？

A：默认 **should** 不参与 **must** 评分——只起“加分”作用。用 `minimumShouldMatch("1")` 强制要求至少 1 个 **should** 命中。

【七、`getUserEffectiveOrgTags`：含父级的 tag 集合】

```
private List<String> getUserEffectiveOrgTags(String userId) {
    User user = userRepository.findById(Long.parseLong(userId)).orElseThrow();

    Set<String> result = new HashSet<>();
    String[] directTags = user.getOrgTags().split(",");

    for (String tag : directTags) {
        result.add(tag);
        // 递归加父级
        addParentTagsRecursively(tag, result);
    }

    return new ArrayList<>(result);
}
```

关键点：

- 用户直接 tag + 所有父级 tag。
- **pai-smart** 用递归——`getParentTag` 查 DB。
- 生产环境用 **cache**——避免每次请求都查 DB。

面试考点 7：树形结构怎么高效查询所有祖先？

A：

- 缓存：把「tagId → 所有祖先 tagId 列表」缓存，新增 **tag** 时失效。
- 维护冗余字段：`OrganizationTag` 加 `ancestors` 字段存「`/,A,A/B,A/B/C`」——一次查询拿到所有祖先。
- **WITH RECURSIVE** (MySQL 8.0+)：SQL 递归——比 **Java** 递归少一次网络。

【八、AdminUserInitializer：默认管理员 + DEFAULT tag】

`pai-smart` 启动时建：

- 默认管理员账号（从配置读）。
- **DEFAULT** 组织标签——`OrgTagInitializer` 创建。
- 所有新用户注册自动打 `DEFAULT` tag。

为什么要有 **DEFAULT**？

- 个人用户不属于任何公司——也得有组织。
- **DEFAULT** 是个“自由区”——所有用户共享。
- 公开文档都在 `DEFAULT` 下。

这是「单租户」升级到「多租户」的过渡设计。

【九、BootstrapKnowledgeInitializer：内置知识库**】

```
@Component
public class BootstrapKnowledgeInitializer {
    // 应用启动时，向 DEFAULT org tag 下注入一些“内置知识”
    // 比如：项目说明、API 文档、FAQ
}
```

设计意图：

- 新用户注册后，立刻有内容可看。
- 内置知识与用户上传的隔离。
- 降低冷启动的“空状态”感。

面试考点 8：内置知识有什么业务价值？

A：

- 降低新用户学习成本——上手就能用。
- 测试场景——QA 有现成数据。
- 演示场景——给客户演示时不用准备数据。

【十、常见坑 & 面试问答】

Q1：用户的 `org tag` 变了，要清缓存吗？

A：必须的。`OrgTagCacheService` 通常有 `evict(userId)` 方法——管理员在后台改 `org tag` 时调。

`pai-smart` 看代码应该有手动失效 + **TTL** 双保险。

Q2：跨 **org tag** 共享文档怎么办？

A：两种方案：

- `isPublic = true`：所有用户都能看。
- 额外的"共享白名单"：每个文档加 `sharedWithOrgTags: List<String>` ——显式授权。

`pai-smart` 当前只用 `isPublic` ——简化。

Q3：管理员能不能访问所有文档？

A：通常可以。`pai-smart` 看代码：

- SecurityConfig 里 `/api/v1/admin/**` 限 `hasRole("ADMIN")`。
- `OrgTagAuthorizationFilter` 看到 `role = ADMIN` 可能直接放行（具体看 `isUserAuthorized` 完整代码）。

生产环境管理员最好用"超级角色"，不参与 **org tag** 过滤。

Q4：组织树最多支持几级？

A：`pai-smart` 用 `parentTag` String + 递归——理论上无限级——实际上 **3-5** 级够用。过深的树导致查询慢、用户体验差。

Q5：删除 **org tag** 怎么处理存量数据？

A：

- 级联删除：tag 删了，关联文档全删——危险。
- 移到 **DEFAULT**：tag 删了，关联文档改 `org_tag = DEFAULT`——安全。
- 软删除：tag 加 `deleted_at`，保留数据——审计友好。

`pai-smart` 用软删除——生产级做法。

【十一、思考题：为什么 `OrgTagAuthorizationFilter` 写在 `config/` 包？**

跟第 03 集的 `JwtAuthenticationFilter` 一样——写在 **config** 包是"项目习惯"。严格说应该：

- 单独的 `filter/` 或 `security/` 包。

但 `pai-smart` 演进中没拆——**KISS** 原则。

【十二、下集预告】

多租户搞定了。但搜索质量还是不够——

- 用户搜「派聪明」——「pai-smart」也能命中吗？
- 专有名词搜不到？拼写错了能命中？
- 关键字匹配和向量召回怎么平衡？

第 12 集我们要：

- 深入 HybridSearchService ——BM25 + 向量 + 重排序。
- Rerank 怎么用 BGE-Reranker 精排。
- 多种 query 改写策略。
- 召回的"过滤网"怎么设计。

这是 RAG 系统的"质量灵魂"。

我们下期见。

多租户是 SaaS 系统的生死线——pai-smart 的 OrgTag 设计简单但完整。
觉得有用，点赞、投币、收藏。

下一集链接：[第 12 集：混合检索——BM25 + 向量 + Rerank](#)

第 12 集：混合检索——BM25 + 向量 + Rerank

系列：派聪明 RAG 后端演进全解

集数：12 / 20+

主题：BM25 vs 向量、RRF 倒数排名融合、Rerank 模型、查询改写

上集回顾：第 11 集 OrgTag 多租户搞定

本集目标：搜索质量提升一个数量级

【开场 Hook】

一个 RAG 系统的「检索」好不好，直接决定回答质量。

但纯向量检索有三大问题：

1. 专有名词不友好：用户搜「JDK 21」——embedding 模型可能理解成「JDK 公司 21 周年庆」。
2. 拼写容错差：用户搜「vue」——向量能匹配 `Vue.js`，但拼成「vew」就抓瞎。
3. 数字、ID 类信息弱：订单号「20240512-A-001」——纯语义检索几乎没用。

纯 **BM25**（关键词）检索也有问题：

- 「苹果公司财务」——「苹果手机维修点」也能匹配上——字面命中但语义无关。

`pai-smart` 的解法：混合检索——**BM25 + 向量 + Rerank**。

今天我们把 `HybridSearchService` 完整拆解。

【一、BM25 是什么？】

BM25（**Best Matching 25**）= 经典关键词检索算法。

核心思想：

- **TF**（**Term Frequency**）：词频——「苹果」出现越多越相关。
- **IDF**（**Inverse Document Frequency**）：逆文档频率——「苹果」在很多文档里出现 → 区分度低 → 权重低。
- 文档长度归一化：长文档天然词多——要惩罚。

ES 的 `match` 查询默认就用 BM25。

面试考点 1：BM25 vs TF-IDF？

A : BM25 是 TF-IDF 的改进版 :

- TF 部分加了 饱和函数 ($tf / (tf + k)$) ——避免「「苹果」出现 100 次」比「出现 10 次」相关性高 10 倍。
- 文档长度归一化更合理。

ES 8.x 默认 `similarity: BM25`。

【二、向量检索 vs BM25 : 互补关系】

维度	BM25	向量
专有名词	✓ 强	✗ 弱
拼写容错	✗ 弱 (除非 fuzziness)	✓ 强
语义理解	✗ 弱 (同义词)	✓ 强
数字 / ID	✓ 强	✗ 弱
召回率	低 (漏召)	较高
精度	中	中 (召回了再用 LLM 过滤)

结论 : 两者互补 , 必须结合。

面试考点 2 : 为什么"必须" ?

A : 纯向量在 RAG 场景召回率还不够。BM25 兜底专有名词、ID、缩写。两者结合才能「字面 + 语义」全覆盖。

【三、HybridSearchService 的召回流程】

```
public List<SearchResult> searchWithPermission(String query, String userId, int topK) {
    // 1. 取用户有效 org tags
    List<String> userEffectiveTags = getUserEffectiveOrgTags(userId);
    String userDbId = getUserDbId(userId);

    // 2. 生成查询向量
    List<Float> queryVector = embedToVectorList(query, userId);

    if (queryVector == null) {
        return textOnlySearchWithPermission(query, userId, topK); // 降级
    }

    // 3. 构造混合查询
```

```

SearchResponse<EsDocument> response = esClient.search(s -> {
    s.index("knowledge_base");
    int recallK = topK * 30;

    // a. 向量召回 (kNN)
    s.knn(kn -> kn
        .field("vector")
        .queryVector(queryVector)
        .k(recallK)
        .numCandidates(recallK)
    );

    // b. BM25 召回 (match)
    s.query(q -> q.match(m -> m.field("textContent").query(query)));

    // c. Rescore (精排)
    s.rescore(r -> r.windowSize(recallK).query(rq -> rq
        .queryWeight(0.2d)
        .rescoreQueryWeight(1.0d)
        .query(rqq -> rqq.match(m -> m.field("textContent").query(query)))
    ));

    // d. 权限过滤
    s.postFilter(pf -> pf.bool(b -> b
        .should(s -> s.term(t -> t.field("isPublic").value(true)))
        .should(s -> s.terms(t -> t.field("orgTag").terms(...)))
        .should(s -> s.term(t -> t.field("userId").value(userDbId)))
        .minimumShouldMatch("1")
    ));
}, EsDocument.class);

return parseResults(response);
}

```

3.1 三阶段召回

Stage 1: 向量召回 (kNN)

- 召回数：`topK * 30` (**30 倍粗排**——给 rerank 留空间)。
- 字段：`vector` (dense_vector 1024 维)。
- 算法：HNSW 近似最近邻。

Stage 2: BM25 召回 (match)

- `s.query(q -> q.match(...))` — 文本匹配 `textContent`。
- BM25 打分。

Stage 3: Rescore 精排

- `windowSize: recallK` — 只在粗排结果上重打分。
- `queryWeight: 0.2` — 原始 BM25 权重 0.2。
- `rescoreQueryWeight: 1.0` — rescore 时 BM25 权重 1.0。

- 效果：保留粗排多样性，精排阶段更准。

面试考点 3：为什么粗排用 kNN，主查询用 BM25，rescore 又用 BM25？

A：

- 粗排：向量召回召回率高，给足候选。
- 主查询 **BM25**：让 **BM25** 命中的也在粗排里（如果只 kNN，BM25 命中的就漏了）。
- **Rescore BM25**：对所有粗排 doc 重新算 BM25 分数，比单次打分准。
- **queryWeight = 0.2**：让 ES 知道最终分数 $\approx 0.2 * \text{原始分} + 1.0 * \text{rescore 分}$ 。

这是 **ES 8.x** 的「混合召回 + 二次精排」——不需要外部 **rerank** 库。

3.2 权限过滤：postFilter

```
s.postFilter(pf -> pf.bool(b -> b
  .should(s -> s.term(t -> t.field("isPublic").value(true)))
  .should(s -> s.terms(t -> t.field("orgTag").terms(...)))
  .should(s -> s.term(t -> t.field("userId").value(userId)))
  .minimumShouldMatch("1")
));
```

postFilter vs **query**：

- **query**：影响打分（慢）。
- **postFilter**：只过滤不影响打分（快）。

权限过滤用 **postFilter** ——快。

3.3 召回量：30 倍经验值

```
int recallK = topK * 30;
```

为什么 30 倍？

- 用户问 1 个问题 → 系统需要返回 top 5-10 个 chunk。
- 但真正的相关 **chunk** 可能有 50-100 个。
- 召回 30 倍（如果 **topK = 5**，召 150 个）——给 **rerank** 足够空间。

面试考点 4：召回太多会不会慢？

A：会。30 倍是平衡点。

- 10 倍：召回不够，好答案可能漏。
- 100 倍：太慢，**rerank** 也跑不动。
- 生产环境根据业务调——**pai-smart** 的 30 是经验值。

【四、查询改写：Query Rewriting】

虽然 `pai-smart` 的 `HybridSearchService` 没显示做 query rewriting，但生产环境 **RAG** 必备：

为什么需要？

- 用户问「派聪明怎么部署」——「派聪明」是个具体词，但「怎么部署」可能太宽。
- 改写成「派聪明 部署 文档」或「派聪明 Docker Compose」——更精准。

常见改写策略：

1. 同义词扩展：「Java」→「JDK」、「JVM」。
2. **HyDE (Hypothetical Document Embeddings)**：用 LLM 生成一个「假设答案」——对假设答案做 embedding，比直接对 query 做 embedding 准。
3. **Step-back prompting**：问「为什么」——先问「背景知识是什么」。
4. **Multi-query**：用 LLM 生成 3-5 个改写 query——多路召回 + 合并。

`pai-smart` 当前没用——未来优化点。

面试考点 5：HyDE 是什么原理？

A：让 **LLM** 生成一个“假答案”——这个假答案语义上接近真答案——对假答案做 **embedding** 比直接对 query 做 embedding 召回更准。

【五、`attachFileNames`：召回结果补全文件名】

```
private List<SearchResult> attachFileNames(List<SearchResult> results) {
    // 取结果里所有 fileMd5
    Set<String> fileMd5s = results.stream().map(SearchResult::getFileMd5).collect(toSet());

    // 一次查 DB
    Map<String, String> fileNameMap = fileUploadRepository.findByFileMd5In(fileMd5s)
        .stream().collect(toMap(FileUpload::getFileMd5, FileUpload::getFileName));

    // 补全到结果里
    for (SearchResult r : results) {
        r.setFileName(fileNameMap.getOrDefault(r.getFileMd5(), "未知文档"));
    }
    return results;
}
```

N+1 问题的经典解法：

- 不要：
`for result: result.setFileName(fileUploadRepository.findByMd5(md5).getFileName())` ——

N+1。

- 要：先取所有 md5 → 一次 **IN** 查询 → **Map** 反查。

面试考点 6：什么是 N+1？

A：

- 1 次查主表 → N 次查关联表。
- 优化：JOIN、IN、批查询。

pai-smart 这里用了正确的优化。

【六、Context 注入：召回 → LLM】

```
private String buildContext(List<SearchResult> results) {
    StringBuilder sb = new StringBuilder();
    int idx = 1;
    for (SearchResult r : results) {
        sb.append("来源#").append(idx++)
          .append(" 文件名: ").append(r.getFileName())
          .append(" (页码: ").append(r.getPageNumber()).append(")\n")
          .append(r.getTextContent().substring(0, Math.min(300, r.getTextContent().length()))
    )
        .append("\n\n");
    }
    return sb.toString();
}
```

注入给 LLM 的 context 格式：

```
来源#1 文件名：派聪明部署文档.pdf（页码：5）
派聪明支持Docker部署，启动命令是 docker-compose up -d...

来源#2 文件名：派聪明FAQ.md
派聪明的端口配置在 application.yml 中...

来源#3 ...
```

关键设计：

- 编号：LLM 可以引用「来源#1」「来源#2」。
- 文件名 + 页码：用户能跳转原文档。
- 截断到 300 字符（MAX_CONTEXT_SNIPPET_LEN）：节省 token。
- 每个 chunk 单独成段：LLM 看得清楚。

面试考点 7：context 长度怎么控制？

A：

- chunk 数量：3-10 个（多了 LLM 关注不到重点）。

- 每个 **chunk** 长度 : 200-500 token (多了浪费)。
- 总 **context** : 1500-3000 token。

`pai-smart` 用 `MAX_CONTEXT_SNIPPET_LEN = 300` 字符 (约 **150-200 token**) ——保守。

【七、 `SearchController` : REST 入口】

```
@RestController
@RequestMapping("/api/search")
public class SearchController {

    @Autowired
    private HybridSearchService hybridSearchService;

    @GetMapping("/hybrid")
    public ResponseEntity<List<SearchResult>> hybridSearch(
        @RequestParam String query,
        @RequestAttribute("userId") String userId,
        @RequestParam(defaultValue = "5") int topK) {

        return ResponseEntity.ok(hybridSearchService.searchWithPermission(query, userId, top
K));
    }
}
```

注意 `@RequestAttribute("userId")` —— 不是 `@RequestParam` ! 这是 `OrgTagAuthorizationFilter` 设置的 **request attribute** (第 11 集讲的)。

面试考点 8 : `@RequestAttribute` vs `@RequestParam` vs `@PathVariable` ?

A :

- `@RequestParam` : URL query 参数 (`?xxx=yyy`)。
- `@PathVariable` : URL path 的一部分 (`/ {xxx}`)。
- `@RequestAttribute` : **request 域 attribute** (由 **filter / interceptor** 设置)。

`pai-smart` 用 `@RequestAttribute` 接 `userId`——避免和 **query string** 冲突。

【八、 BM25 调参 : `boost`、`minimum_should_match` **

`pai-smart` 没显式调 BM25 参数 (用默认) , 但生产环境要调 :

```
s.query(q -> q.match(m -> m
    .field("textContent")
    .query(query)
    .boost(1.5f) // 提高 BM25 权重
```

```
.minimumShouldMatch("75%") // 至少匹配 75% 词
));
```

经验：

- **boost**：调整 BM25 在混合中的权重。
 - **minimum_should_match**：避免「「苹果」」一个字就匹配上。
-

【九、常见坑 & 面试问答】

Q1：BM25 和向量怎么融合？

A：三种方式：

- **score** 加权： $\text{finalScore} = 0.5 * \text{bm25} + 0.5 * \text{vector}$ 。
- **rank** 倒数排名 (**RRF**)： $\text{finalScore} = 1/(k + \text{rank_bm25}) + 1/(k + \text{rank_vector})$ 。
- 召回 + 融合 + **rerank**：**pai-smart** 用的方式——**BM25** 和向量各召回，最后 **ES rescore** 融合。

Q2：Rerank 模型怎么选？

A：

- **BGE-Reranker** (BAAI)：中文最强开源。
- **Cohere Rerank**：商业，效果好。
- **Jina Rerank**：多语言。

pai-smart 当前用 **ES rescore**——已经够用。**BGE-Reranker** 集成是未来优化。

Q3：检索效果差怎么排查？

A：

- 看 **top-5** 结果：人工判断相关性。
- 看 **BM25** 和向量各召回了什么：是否互补。
- 看 **doc** 长度分布：太长的 chunk 容易被切碎。
- 看 **embedding** 模型：是否支持中文。
- **A/B** 测试：改参数对比。

Q4：怎么评估检索质量？

A：

- **Recall@K**：前 K 个里有多少真相关。
- **MRR** (Mean Reciprocal Rank)：第一个相关 doc 的排名倒数的平均。
- **NDCG**：归一化折损累积增益。

pai-smart 没看到评估代码——这是 **TODO**。

Q5：搜索性能瓶颈在哪？

A：通常在 **kNN** 召回——**HNSW** 索引的 **query latency** 是 **$O(\log N)$** ——**10** 亿向量也能 **10ms** 召回。

ES 集群：网络、磁盘 IO、GC 都会影响。

【十、思考题：要不要加 **HyDE**？**

HyDE 的代价：

- 每次查询多调一次 **LLM**（生成假设答案）——慢 + 花钱。
- 对所有 **query** 都有用吗？不一定——专有名词查询可能反而退化。

HyDE 的收益：

- 抽象 query（「为什么……」）→ 检索质量明显提升。
- 复杂问题受益大。

生产建议：

- 简单 **query**（含专有名词）：直接 **BM25** + 向量。
- 复杂 **query**（抽象问题）：用 **LLM** 改写。
- 自动判断：用 query 长度 / 词数启发式。

pai-smart 没做——未来。

【十一、下集预告】

搜索质量上来了。但！

用户疯狂刷接口怎么办？

- 单用户 1 秒发 100 次查询——打爆 **ES**。
- 1 个用户上传 1000 个文件——配额耗尽。
- 恶意脚本调用——资源耗尽。

第 13 集我们要：

- **RateLimitService** 怎么用 Redis 实现限流。
- 滑动窗口 vs 令牌桶。
- **UsageQuotaService** 的配额扣减。
- 降级策略——超限后怎么办。

限流是后端工程化的“成人礼”。

我们下期见。

混合检索是 RAG 系统的"质量分水岭"——[pai-smart](#) 的 BM25 + kNN + rescore 三件套值得学。
觉得有用，点赞、投币、收藏。

下一集链接：[第 13 集：限流与配额——Redis 守护神](#)

第 13 集：限流与配额——Redis 守护神

系列：派聪明 RAG 后端演进全解

集数：13 / 20+

主题：滑动窗口、令牌桶、固定窗口、Redis 限流、token 配额预算

上集回顾：第 12 集混合检索上线

本集目标：让系统能扛住恶意请求，每个用户的 token 消耗可控

【开场 Hook】

RAG 系统有几个"资源黑手"：

1. **Embedding API**——花钱的（DashScope v4 按 token 计费）。
2. **LLM API**——更花钱（DeepSeek 按 token 计费）。
3. **ES 集群**——**CPU** 密集（kNN 搜索）。
4. **MinIO**——带宽密集（下载大文件）。

如果不限流：

- 一个脚本小子 1 秒发 1000 次 query——打爆 **ES**，账单爆表。
- 一个用户上传 1 万个文件——配额耗尽。
- 一次爬虫扫全部文档——**MinIO** 带宽打满。

`pai-smart` 的解法：限流（**Rate Limit**）+ 配额（**Quota**）——两道防线。

今天我们拆 `RateLimitService` 和 `UsageQuotaService`。

【一、限流 vs 配额：两个不同的概念】

限流（**Rate Limit**）：

- 时间窗口内的请求次数上限。
- 例：每分钟最多 10 次查询。
- 防突发流量。

配额（**Quota**）：

- 累计的资源使用上限。
- 例：每月 100 万 token。
- 防资源滥用。

两者关系：

- 限流快失败——秒级响应。
- 配额慢累计——月结账。

`pai-smart` 两个都有——限流在前、配额在后。

面试考点 1：为什么需要两道防线？

A：

- 限流防 **DoS**——拒绝异常流量。
 - 配额防滥用——限制正常用户的累计使用。
 - 攻击者可以通过时间换空间绕过单一防线（每分钟 10 次但跑 30 分钟 = 300 次）。
-

【二、`RateLimitProperties`：配置抽象】

```
@ConfigurationProperties(prefix = "rate-limit")
public class RateLimitProperties {
    private Window register;    // 注册限流
    private Window login;      // 登录限流
    // ... 其他限流配置

    public static class Window {
        private long max;
        private long windowSeconds;
    }
}
```

配置示例（`application.yml`）：

```
rate-limit:
  register:
    max: 5
    window-seconds: 60
  login:
    max: 10
    window-seconds: 60
  chat:
    max: 30
    window-seconds: 60
```

把限流参数配置化——不改代码调限流。生产环境必备。

面试考点 2：为什么不用 `@Value` 用 `@ConfigurationProperties`？

A：

- 多字段聚合——`register.max`、`register.windowSeconds` 是一组。

- 类型安全——`long max` VS `@Value("${...}") long max`。
 - IDE 友好——配置自动补全。
-

【三、`checkSingleWindow`：固定窗口限流】

```
private void checkSingleWindow(String key, long max, long windowSeconds, String message) {
    Long current = stringRedisTemplate.opsForValue().increment(key);
    if (current == null) {
        // 失败降级
        return;
    }
    if (current == 1L) {
        // 第一次设置 TTL
        stringRedisTemplate.expire(key, windowSeconds, TimeUnit.SECONDS);
    }
    if (current > max) {
        throw new RateLimitExceededException(message);
    }
}
```

核心逻辑：

1. **INCR**：原子加 1（Redis 单命令原子）。
2. 第一次设置 **TTL**。
3. 超限抛异常。

Redis Key 设计：`{业务}:{维度}:{标识}` —— `chat:user:123` ——方便排查。

面试考点 3：为什么 **INCR** 之后 **EXPIRE** 不原子？

A：两条命令！如果 **INCR** 完应用挂了，**key** 永不过期——内存泄漏。

生产环境用 **Lua** 脚本：

```
local current = redis.call('INCR', KEYS[1])
if current == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[1])
end
return current
```

pai-smart 这里有 **race condition** 风险——**INCR** 和 **EXPIRE** 不是原子的。生产环境应该用 **Lua**。

更现代的方案：`SET key value EX seconds NX` ——一条命令搞定。但 **INCR** 计数要返回当前值——只能 **Lua**。

3.1 固定窗口的"突刺"问题

固定窗口的经典 **bug** :

限制每分钟 10 次。**00:59** 发了 **10** 次, **01:01** 又发了 **10** 次——**2 秒内 20 次**——超限了。

这种"窗口边界突刺"是固定窗口的固有缺陷。

面试考点 **4** : 固定窗口 vs 滑动窗口 vs 令牌桶 ?

A :

- 固定窗口 : 实现简单, 有突刺问题。
- 滑动窗口 : 用多个小窗口模拟 (**Redis ZSET** 实现), 更精确, 更耗内存。
- 令牌桶 : 以恒定速率往桶里放令牌, 请求取令牌——能容忍突发。
- 漏桶 : 以恒定速率处理请求——绝对平滑。

pai-smart 固定窗口——简单够用。生产高并发考虑滑动窗口。

【四、**checkChatByUser** : 限流 + 配额联动】

```
public void checkChatByUser(String userId) {
    RateLimitConfigService.WindowLimitView limit =
rateLimitConfigService.getCurrentSettings().chatMessage();
    checkSingleWindow("chat:user:" + userId, limit.max(), limit.windowSeconds(), "聊天请求过于
频繁");
    usageQuotaService.recordChatRequest(userId); // 配额记账
}
```

两道防线 :

1. **checkSingleWindow** : 限流——每分钟 **30** 次。
2. **recordChatRequest** : 配额记账——累加到每日/月度计数。

RateLimitConfigService.getCurrentSettings() : 从 DB 读 当前生效的限流配置——管理员可后台调整。

【五、**reserveLlmUsage** : LLM token 预算**

```
public UsageQuotaService.TokenReservationBundle reserveLlmUsage(
    String userId, int estimatedPromptTokens, int maxCompletionTokens) {
    RateLimitConfigService.TokenBudgetView limit =
rateLimitConfigService.getCurrentSettings().llmGlobalToken();
    return usageQuotaService.reserveLlmTokensWithGlobalBudget(
        userId,
```



```

        estimatedPromptTokens, maxCompletionTokens,
        limit.minuteMax(), limit.minuteWindowSeconds(), // 全网分钟预算
        limit.dayMax(), limit.dayWindowSeconds()); // 全网当日预算
    }

```

注意：是「全网」预算！

llmGlobalToken = 所有用户共享的 token 预算。和单用户配额不同。

为什么需要"全网预算"？

DeepSeek 的账单是平台付的——如果 **1** 万个用户同时疯狂调 **LLM**——账单可能爆。

全网预算是成本控制的最后一道：

- 单用户限流挡住"个人疯狂"。
- 全网预算挡住"集体疯狂"。

面试考点 **5**：全网预算怎么做？

A：

- **INCRBY** 一个全局 key，设置 **TTL**。
- 超过阈值 → 拒绝新请求（降级到其他模型 / 排队等待）。

【六、**reserveEmbeddingUploadUsage** 和 **reserveEmbeddingQueryUsage**】

```

public UsageQuotaService.TokenReservationBundle reserveEmbeddingUploadUsage(String userId, List<String> texts) {
    RateLimitConfigService.TokenBudgetView limit =
    rateLimitConfigService.getCurrentSettings().embeddingUploadToken();
    return usageQuotaService.reserveEmbeddingTokensWithGlobalBudget(
        userId, texts, "embedding-upload",
        "Embedding上传全网分钟Token预算已达上限",
        "Embedding上传全网当日Token预算已达上限",
        limit.minuteMax(), limit.minuteWindowSeconds(),
        limit.dayMax(), limit.dayWindowSeconds());
}

```

双轨限流：

- **embeddingUploadToken**：上传文档时 embedding——全网预算。
- **embeddingQueryToken**：搜索时 embedding——全网预算。

为什么分开？

- **UPLOAD** 烧钱多（一次上传可能 **1** 万 **token**）。
- **QUERY** 烧钱少（一次搜索几百 **token**）。

分开预算 = 精细化成本控制。

【七、UsageQuotaService : recordChatRequest 和 recordChatRequest】

虽然没看到完整实现，但推测 UsageQuotaService 用 Redis 维护：

- chat:user:{userId}:day:{date} : INCR，每日聊天次数。
- embedding:user:{userId}:month:{yyyyMM} : INCRBY token，每月 token 消耗。

Redis 比 DB 快，且支持原子计数。

面试考点 6：为什么用 Redis 不用 DB 计数？

A：

- Redis 单命令原子（INCR、INCRBY）。
- DB 计数要锁（悲观锁 / 乐观锁）。
- Redis 可以设 TTL——自然清旧数据。
- DB 计数要定时清理。

【八、RateLimitExceededException：优雅的拒绝】

```
throw new RateLimitExceededException(message);
```

exception/ 包的 RateLimitExceededException：

```
public class RateLimitExceededException extends RuntimeException {  
    private final long retryAfterSeconds;  
    // ...  
}
```

为什么要自定义异常？

- 包含 retryAfter——前端知道等多久。
- 全局异常处理器（@ControllerAdvice）统一返回 429。
- 带业务信息——前端能弹具体提示。

面试考点 7：HTTP 429 是什么？

A：Too Many Requests ——RFC 6585 定义的限流响应码。带 Retry-After 头告诉客户端等多久。

【九、降级策略：超限后怎么办？**

超限后通常三种选择：

1. 直接拒绝——返回 429。
2. 排队等待——返回 202 Accepted + Job ID——后台跑完通知。
3. 降级到低配——embedding 用本地小模型，LLM 用更便宜的模型。

`pai-smart` 当前是方案 **1**（直接拒绝）——简单。

生产环境：

- 限流超限 → 方案 **1**（保护资源）。
 - 配额耗尽 → 方案 **2 / 3**（用户已付费）。
-

【十、`UsageType.UPLOAD` vs `UsageType.QUERY` 的双配额】

回顾第 07 集的 `EmbeddingClient.UsageType`：

```
public enum UsageType {  
    UPLOAD,  
    QUERY  
}
```

两种使用场景、两种配额：

- **UPLOAD** 配额：月限制 100 万 token——大文件消耗。
- **QUERY** 配额：月限制 50 万次 / 50 万 token——搜索消耗。

为什么分开？

- 用户搜索 1000 次才花 **1 万 token**——但请求 **1000 次**。
- 上传 1 个大文件就花 **5 万 token**。

混合计数会让"重度搜索用户"和"重度上传用户"抢资源。

【十一、限流的"维度"】

`pai-smart` 的限流有多个维度：

维度	例子	用途
IP	register:ip:1.2.3.4	防刷注册、登录
User	chat:user:123	防单个用户滥用
Action	embedding-upload	分场景精细控制
Global	llm-global	全平台成本控制

维度越多越精细——但也越复杂。

面试考点 8：限流维度怎么设计？

A：

- 最粗：global 一个 key——全平台限流。
- 中：{action}:{userId} ——按用户。
- 最细：{action}:{userId}:{resourceId} ——按资源。

pai-smart 两层：IP（防刷）+ User（防滥用）。

【十二、Redis 挂了怎么办？**

checkSingleWindow 里：

```
Long current = stringRedisTemplate.opsForValue().increment(key);
if (current == null) {
    return; // 降级放行
}
```

Redis 挂了 → INCR 返回 null → 放行。

这是「fail open」策略——Redis 故障时不再限流。

反之「fail closed」——Redis 故障时拒绝所有请求。

两种策略的对比：

- fail open：业务可用性高，但可能被打。
- fail closed：资源安全，但业务中断。

pai-smart fail open——业务优先。生产环境根据业务选。

面试考点 9：fail open 还是 fail closed？

A：

- 支付类：fail closed（钱不能错）。

- 内容类：fail open（宁愿有垃圾）。
 - 限流类：fail open（避免限流本身变故障源）。
-

【十三、常见坑 & 面试问答】

Q1：INCR 和 EXPIRE 不原子怎么办？

A：用 Lua 脚本。生产环境必做——`pai-smart` 这里有 bug。

Q2：滑动窗口怎么实现？

A：Redis ZSET 存请求时间戳：

```
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, ARGV[1]) -- 删除窗口外
local current = redis.call('ZCARD', KEYS[1]) -- 当前数
if current < tonumber(ARGV[2]) then
    redis.call('ZADD', KEYS[1], ARGV[3], ARGV[3])
    return 1
end
return 0
```

Q3：令牌桶怎么实现？

A：Redis + Lua：

```
local tokens = redis.call('GET', KEYS[1]) or ARGV[2]
local lastRefill = redis.call('GET', KEYS[2]) or ARGV[3]
local now = ARGV[1]
-- 计算补充
tokens = min(ARGV[2], tokens + (now - lastRefill) * ARGV[4])
if tokens >= 1 then
    tokens = tokens - 1
    redis.call('SET', KEYS[1], tokens)
    redis.call('SET', KEYS[2], now)
    return 1
end
return 0
```

Q4：限流 + 配额会不会冲突？

A：不会——两道独立防线：

- 限流挡短时突发。
- 配额挡长时累计。

Q5：怎么让前端知道还剩多少配额？

A：返回 `X-RateLimit-Remaining` 头（GitHub 风格）：

```
response.setHeader("X-RateLimit-Limit", max);
response.setHeader("X-RateLimit-Remaining", max - current);
response.setHeader("X-RateLimit-Reset", expireTime);
```

`pai-smart` 没看到这套——未来优化。

【十四、思考题：限流是"前端友好"还是"后端友好"？**

前端友好：超限后等 1s 再试——用户体验好。

后端友好：超限后 429 拒绝——保护资源。

平衡：

- 限流用 **fail open**（前端友好）。
- 配额用 **fail closed**（后端友好）。
- 关键操作 **fail closed**（支付、删除）。

`pai-smart` 的策略是有意识的——值得学。

【十五、下集预告】

限流配额定下来了。但用户聊完天后，会话没保存——下次登录啥都没有。

第 14 集我们要：

- `Conversation` 和 `ConversationSession` 实体设计。
- 历史消息的存储和查询。
- 滑动窗口保留最近 N 条。
- **WebSocket** 重连后如何续传历史。

对话历史是 **RAG** 系统的"记忆"。

我们下期见。

限流和配额是生产级系统的成人礼——`pai-smart` 的 Redis 方案简单直接。
觉得有用，点赞、投币、收藏。

下一集链接：[第 14 集：对话会话与历史——RAG 系统的记忆](#)

第 14 集：对话会话与历史——RAG 系统的记忆

系列：派聪明 RAG 后端演进全解

集数：14 / 20+

主题：Conversation vs ConversationSession、消息存储、滑动窗口上下文、重连续传

上集回顾：第 13 集限流配额到位

本集目标：用户的对话有记忆、可回溯、可恢复

【开场 Hook】

用户问 RAG 系统：

1. 「派聪明怎么部署？」
2. 「能详细说说吗？」
3. 「Docker Compose 文件在哪？」

RAG 系统必须「记得」之前的对话——否则第 3 问就答不了（前文没提“派聪明”）。

但记忆不能无限保留：

- LLM 上下文长度有限（**32K tokens** $\approx$ 6 万中文字）。
- 历史太长 $\rightarrow$ **token** 浪费。
- 历史太短 $\rightarrow$ 上下文丢失。

pai-smart 怎么平衡？

【一、两个核心实体】

pai-smart 用了两个实体：

- **Conversation**：一条消息（用户问的 / **AI** 答的）。
- **ConversationSession**：一个会话（多条消息的容器）。

关系：

```
User 1 —* ConversationSession 1 —* Conversation N
                        ↓
                    user / assistant / system
```

为什么拆开？

- **Session** 生命周期长——用户主动结束才结束。
- **Message** 数量多——一个 **session** 可能上千条。

分开存：session 元数据少，频繁查；message 量大，按需查。

面试考点 1：为什么不用 JSON 字段存整个 session？

A：

- 查询能力：JSON 字段不能 **LIKE**——历史检索差。
- 分页：分页要 JS 解析。
- 多端同步：每条消息独立记录——多端更好同步。

【二、Conversation 实体（推测）】

```
@Entity
@Table(name = "conversations")
public class Conversation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "session_id", nullable = false, length = 64)
    private String sessionId;

    @Column(name = "user_id", nullable = false, length = 64)
    private String userId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 16)
    private Role role; // user / assistant / system

    @Lob
    @Column(nullable = false)
    private String content;

    @Column(name = "referenced_chunk_ids", length = 1000)
    private String referencedChunkIds; // 引用的 chunk id (逗号分隔)

    @Column(name = "referenced_file_md5s", length = 1000)
    private String referencedFileMd5s; // 引用的文件

    @Column(name = "token_count")
    private Integer tokenCount; // 本条消息的 token 数

    @Column(name = "generation_id", length = 64)
    private String generationId; // 对应 ChatGenerationStateService

    @CreationTimestamp
```



```
private LocalDateTime createdAt;

public enum Role { USER, ASSISTANT, SYSTEM }
}
```

关键字段：

1. **sessionId**：归属哪个会话。
2. **role**：user / assistant / system——**LLM** 用的消息格式。
3. **content**：消息内容。
4. **referencedChunkIds**：AI 回答引用了哪些 **chunk**——第 19 集消息反馈要用。
5. **tokenCount**：记账——配额扣减。
6. **generationId**：关联到 `ChatGenerationStateService.GenerationSnapshot` ——状态追踪。

面试考点 2：为什么 **content** 用 **@Lob** ？

A：**@Lob** = Large Object。 **TEXT** 字段——**MySQL** 存长文本。普通 **VARCHAR** 长度限制 **65535** ——**LLM** 回答可能超过。

@Lob 缺点：查询慢（**TEXT** 不在索引里）。生产环境考虑 **MySQL FULLTEXT** 索引或 **ES** 索引。

【三、 **ConversationSession** 实体（推测）】

```
@Entity
@Table(name = "conversation_sessions")
public class ConversationSession {
    @Id
    @Column(name = "session_id", length = 64)
    private String sessionId;

    @Column(name = "user_id", nullable = false, length = 64)
    private String userId;

    @Column(name = "title", length = 255)
    private String title; // 会话标题（首条消息摘要）

    @Column(name = "message_count")
    private Integer messageCount = 0; // 消息数（denormalized）

    @Column(name = "total_tokens")
    private Long totalTokens = 0L; // 累计 token

    @Column(name = "last_active_at")
    private LocalDateTime lastActiveAt;

    @CreationTimestamp
    private LocalDateTime createdAt;
```

```
@UpdateTimestamp
private LocalDateTime updatedAt;
}
```

为什么有 `messageCount` 和 `totalTokens` ？

反范式设计——避免每次列表都 **COUNT(*) conversations**。

代价：

- 每次插入消息要 **UPDATE** 会话表——多一次写。

收益：

- 列表查询极快——直接读会话表。

面试考点 **3**：反范式的代价与收益？

A：

- 收益：查询快，避免 **JOIN**。

- 代价：写入多一次，要保证一致性（事务 / 乐观锁）。

- 原则：读多写少才反范式。

`pai-smart` 选反范式——会话列表查询频繁。

【四、`ConversationService.ensureConversationSession`】

```
public void ensureConversationSession(Long userId, String conversationId, String firstMessage) {
    // 1. 查会话存不存在
    Optional<ConversationSession> existing = conversationSessionRepository.findById(conversationId);

    if (existing.isEmpty()) {
        // 2. 不存在 → 创建
        ConversationSession session = new ConversationSession();
        session.setSessionId(conversationId);
        session.setUserId(userId.toString());
        session.setTitle(summarize(firstMessage)); // 用首条消息做标题
        session.setMessageCount(0);
        session.setTotalTokens(0L);
        conversationSessionRepository.save(session);
    }

    // 3. 存在 → 啥也不做
}
```

`summarize(firstMessage)`：从首条消息提取标题——取前 **30** 字符或调 LLM 生成。

简单做法：

```
private String summarize(String message) {
    String trimmed = message.trim();
    if (trimmed.length() > 30) {
        return trimmed.substring(0, 30) + "...";
    }
    return trimmed;
}
```

进阶做法：用 LLM 生成"标题"——「用户询问派聪明部署方式」。

pai-smart 前者——简单。

面试考点 4：sessionId 怎么生成？

A：

- **UUID**：UUID.randomUUID().toString() ——最常见。
- 雪花 ID：分布式唯一。
- 业务 ID：用户首次打开聊天的时间戳——可读但冲突。

pai-smart 推测用 **UUID**——安全、简单。

【五、getRecentHistory：滑动窗口的上下文】

```
public List<Map<String, String>> getRecentHistory(String conversationId, int limit) {
    Pageable pageable = PageRequest.of(0, limit); // 取最近 limit 条
    List<Conversation> recent = conversationRepository
        .findBySessionIdOrderByCreatedAtDesc(conversationId, pageable);

    Collections.reverse(recent); // 翻转成正序

    return recent.stream()
        .map(c -> Map.of("role", c.getRole().name().toLowerCase(), "content", c.getContent()))
        .toList();
}
```

PageRequest.of(0, limit)：取最近 **limit** 条——典型分页。

OrderByCreatedAtDesc + **Collections.reverse**：倒序取出再翻正——保证最新消息在最后。

为什么 **limit** 通常 **10**？

- **DeepSeek context 32K tokens**。
- 10 条对话平均 200 token/条 → **2000 token**。
- 留出余量给当前问题和 RAG context。

面试考点 5：limit 选多少合理？

A:

- 小 **LLM** (4K context) : **4-6** 条。
- 中 **LLM** (16K context) : **8-12** 条。
- 大 **LLM** (128K context) : **20-30** 条。

`pai-smart` 10 条——适配 **DeepSeek**。

【六、`ChatHandler` 里写消息】

```
public void processMessage(String userId, String userMessage, WebSocketSession session) {
    // 1. 限流
    rateLimitService.checkChatByUser(userId);

    // 2. 获取或创建会话
    String conversationId = getOrCreateConversationId(userId);
    conversationService.ensureConversationSession(Long.parseLong(userId), conversationId, userMessage);

    // 3. 创建 generation
    ChatGenerationStateService.GenerationSnapshot generation =
        chatGenerationStateService.createGeneration(userId, conversationId, userMessage);

    // 4. 写 user 消息
    Conversation userMsg = new Conversation();
    userMsg.setSessionId(conversationId);
    userMsg.setUserId(userId);
    userMsg.setRole(Conversation.Role.USER);
    userMsg.setContent(userMessage);
    userMsg.setGenerationId(generation.generationId());
    conversationRepository.save(userMsg);

    // 5. ... ReAct 循环

    // 6. AI 回答完后写 assistant 消息
    Conversation aiMsg = new Conversation();
    aiMsg.setSessionId(conversationId);
    aiMsg.setUserId(userId);
    aiMsg.setRole(Conversation.Role.ASSISTANT);
    aiMsg.setContent(aiResponse);
    aiMsg.setReferencedChunkIds(referencedChunks);
    aiMsg.setReferencedFileMd5s(referencedFiles);
    aiMsg.setTokenCount(actualTokens);
    conversationRepository.save(aiMsg);

    // 7. 更新 session 计数
    conversationService.incrementMessageCount(conversationId, 2); // user + assistant
}
```

写消息的关键点：

1. **user** 和 **assistant** 都存——双向记忆。
2. **referencedChunkIds** 存引用——第 19 集反馈。
3. **tokenCount** 记账——配额结算。
4. **messageCount** 反范式更新——会话列表快。

事务边界：**user** 消息 + **assistant** 消息 + **session** 更新 在一个事务——失败回滚。

【七、**getOrCreateConversationId**：会话怎么开？】

```
private String getOrCreateConversationId(String userId) {
    WebSocketSession session = ...; // 当前 session

    // 1. 先从 session attribute 取
    String convId = (String) session.getAttributes().get("conversationId");
    if (convId != null) return convId;

    // 2. 没有 → 用 WebSocket session id 当 conversationId
    String newConvId = session.getId();
    session.getAttributes().put("conversationId", newConvId);
    return newConvId;
}
```

pai-smart 的策略：

- 一个 **WebSocket** 连接 = 一个会话。
- 断开重连 = 新会话。

这设计简单，但用户体验略差：

- 长时间聊天中网络抖动 → 重连后新会话 → 历史对不上。

更好的设计：

- 客户端每条消息带 **conversationId**。
- 断开重连用旧的 **conversationId** 续。

pai-smart 后期应该会改成这样。

面试考点 6：会话和连接的关系？

A：

- 1 对 1 (**pai-smart** 早期)：简单，断连即结束。
 - 多对 1 (用户连续对话)：重连保留会话。
 - 多对多：多个 **WebSocket** 共享一个会话 (多端协作)。
-

【八、ConversationController：查询历史**

```
@RestController
@RequestMapping("/api/v1/conversations")
public class ConversationController {

    @GetMapping("/sessions")
    public Page<ConversationSession> listSessions(
        @RequestAttribute("userId") String userId,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
        return conversationService.listSessionsByUser(userId, page, size);
    }

    @GetMapping("/sessions/{sessionId}/messages")
    public Page<Conversation> listMessages(
        @RequestAttribute("userId") String userId,
        @PathVariable String sessionId,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "50") int size) {
        // 1. 鉴权：只能查自己的
        ConversationSession session = conversationSessionRepository.findById(sessionId)
            .orElseThrow(() -> new CustomException("会话不存在"));
        if (!session.getUserId().equals(userId)) {
            throw new CustomException("无权访问");
        }
        // 2. 分页查
        return conversationService.listMessagesBySession(sessionId, page, size);
    }
}
```

两个 API：

- `/sessions`：会话列表。
- `/sessions/{id}/messages`：某会话的消息。

鉴权：只能查自己——`session.getUserId() == userId`。

面试考点 7：分页查询的常见 bug？

A：

- **OFFSET** 大 → 慢（MySQL 要扫到 **OFFSET** 位置）。
- 分页期间数据变化（重复 or 漏数据）。
- 解决方案：**keyset pagination**（上一页最后一条的 **ID** 作为下一页的 **WHERE**）。

【九、ChatSessionRegistry：WebSocket 注册表**

回顾第 10 集讲过：

```
public class ChatSessionRegistry {
    // userId -> clientId -> WebSocketSession
    private final Map<String, Map<String, WebSocketSession>> sessions = new ConcurrentHashMap<>();

    public void registerSession(String userId, String clientId, WebSocketSession session);
    public void unregisterSession(WebSocketSession session);
    public WebSocketSession getActiveSession(String userId, String clientId);
    public Map<String, WebSocketSession> getSessions(String userId); // 所有端
}
```

ChatSessionRegistry 是 **WebSocket** 层的——管连接。

ConversationService 是业务层的——管数据。

两套东西分开：

- **ChatSessionRegistry** 内存，不持久化。
- **Conversation** 持久化到 DB。

连接断了 → **ChatSessionRegistry** 删条目；**DB** 里的会话还在。

面试考点 8：会话和连接分开管理的好处？

A：

- 会话是业务——必须持久化。
- 连接是协议——断了就清。
- 两者独立——业务可测。

【十、**lastActiveAt**：会话排序**

```
public Page<ConversationSession> listSessionsByUser(String userId, int page, int size) {
    Pageable pageable = PageRequest.of(page, size,
        Sort.by(Sort.Direction.DESC, "lastActiveAt"));
    return conversationSessionRepository.findById(userId, pageable);
}
```

按 **lastActiveAt DESC** ——最近活跃的在前。

lastActiveAt 怎么更新？

每次写消息 → 同时 UPDATE **conversation_sessions.lastActive_at = NOW()**。

为什么单独字段？

- **updatedAt** 是 Hibernate 自动维护的——粒度到任意字段变更。
- **lastActiveAt** 是业务事件（用户问了问题）触发的——更准确。

【十一、上下文注入：buildMessagesWithHistory】

```
public List<Map<String, String>> buildMessagesWithHistory(String conversationId, String user
Message, String ragContext) {
    // 1. 拿最近 10 条
    List<Map<String, String>> history = getRecentHistory(conversationId, 10);

    // 2. 构造 messages
    List<Map<String, String>> messages = new ArrayList<>();
    messages.add(Map.of("role", "system", "content", "你是派聪明助手。基于以下信息回答：\n" + rag
Context));
    messages.addAll(history);
    messages.add(Map.of("role", "user", "content", userMessage));

    return messages;
}
```

关键顺序：

1. **system** 角色：注入 RAG context。
2. 历史消息：10 条 user/assistant 交替。
3. 当前 **user** 消息。

LLM 看到的完整结构：

```
[system] 你是派聪明助手。基于以下信息回答：
[context 内容]

[user] 派聪明怎么部署？
[assistant] 可以用 Docker Compose...
[user] docker-compose 文件在哪？
[assistant] 在 docs/docker-compose.yaml
[user] 怎么启动？
```

LLM 基于完整上下文回答——多轮对话能力。

面试考点 9：LLM 上下文顺序重要吗？

A：重要。最新消息放最后——Transformer 的 attention 偏向末尾（"recency bias"）。

pai-smart 顺序：**system** → 历史 → 当前 **user** ——正确。

【十二、常见坑 & 面试问答】

Q1：消息量太大怎么清理？

A：

- 业务规则：30 天前的消息归档。

- 定时任务：`@Scheduled` 每天跑——删旧消息。
- 分区表：按月分区——**DROP PARTITION** 比 **DELETE** 快。

`pai-smart` 没看到归档——**TODO**。

Q2：消息加密吗？

A：通常不加密（性能）。敏感场景（医疗、金融）用字段级加密（**AES-256**）。

Q3：多人协作会话怎么做？

A：加 `ConversationSession.participantUserIds: String`（逗号分隔）——所有参与者都能访问。

Q4：长对话 **token** 太多怎么办？

A：摘要压缩。定期把前 **N** 条对话总结成几句话——代替原文。

`pai-smart` 没做——未来优化。

Q5：会话 **ID** 用 **UUID** 还是数字 **ID**？

A：

- **UUID**：分布式友好，不可猜——防 **IDOR** 攻击。
- 数字 **ID**：可枚举——有 **IDOR** 风险。

`pai-smart` 用 **UUID** 风格——正确。

【十三、思考题：为什么 `Conversation` 和 `ConversationSession` 不合并？**

合并方案：

- `Conversation` 里加 `title`、`lastActiveAt`、`messageCount` 字段。
- 不用单独建表。

为什么不合并？

A：

- 查询效率：会话列表不需要消息内容——分表后查得快。
- 更新频率：消息频繁插入，会话元数据偶尔更新——分开降低锁竞争。
- 业务清晰：会话和消息是两个不同层级——领域模型更清晰。

YAGNI 反例：不要为了“少一张表”牺牲可读性。

【十四、下集预告】

会话能保存了。但！现在的 **RAG** 是“一问一答”——**LLM** 只能用 **RAG** 召回的 **context**。

如果用户问“派聪明和 **LangChain** 哪个好？”——派聪明内部文档没有 **LangChain** 信息——**LLM** 只能瞎答。

第 15 集我们要：

- **AgentToolRegistry** 登场——给 **LLM** 装“工具”。
- **ReAct** 循环：**Reason**（思考）→ **Act**（调工具）→ **Observe**（观察结果）→ **Reason**.....
- **search** 工具、**get_current_time** 工具、**generate_summary** 工具。
- 多轮工具调用怎么控制成本。

这是 **RAG → Agent** 的进化。

我们下期见。

对话历史是 RAG 系统的“记忆”——没有它就是金鱼。
觉得有用，点赞、投币、收藏。

下一集链接：[第 15 集：ReAct 与 Agent 工具——LLM 装上手和脚](#)

第 15 集：ReAct 与 Agent 工具——LLM 装上手和脚

系列：派聪明 RAG 后端演进全解

集数：15 / 20+

主题：ReAct 模式、Tool Calling、AgentToolRegistry、循环控制

上集回顾：第 14 集对话历史落地

本集目标：LLM 能主动调工具，多轮思考后给出答案

【开场 Hook】

传统 RAG 流程：

用户问 → 召回 → LLM 回答

问题：召回什么 **LLM** 就用什么——没有"判断"过程。

Agent 流程：

用户问 → LLM 思考 → 选择工具 → 调工具 → 拿到结果 → 再思考 → ... → 回答

LLM 拥有"判断力"——知道什么时候搜、什么时候不搜、什么时候生成摘要。

`pai-smart` 用了 **ReAct** 模式——**Reason + Act**——思考和行动交替。

今天我们拆 `AgentToolRegistry` 和 `ChatHandler.runReActLoop`。

【一、ReAct 是什么？】

ReAct = Reason + Act，2022 年 Yao et al. 提出的范式。

核心思想：

Thought 1: 用户问的是派聪明部署方式，我需要先搜索内部文档。

Action 1: `search_knowledge(query="派聪明 部署")`

Observation 1: 找到 3 个相关文档...

Thought 2: 文档说要 Docker Compose 部署，用户可能想知道具体步骤。

```
Action 2: search_knowledge(query="派聪明 docker-compose 文件")
Observation 2: 找到 1 个相关文档...

Thought 3: 信息够全了, 可以回答。
Final Answer: 派聪明支持 Docker Compose 部署, 启动命令是...
```

三步循环：**Thought** → **Action** → **Observation**。

vs 传统 Chain-of-Thought：

- **CoT**：LLM 内部一步步想。
- **ReAct**：LLM 外部一步步做——每个 **Action** 调用真的工具。

面试考点 1：ReAct vs Function Calling？

A：

- **Function Calling**：LLM 决定调哪个函数 + 参数——一次返回。
- **ReAct**：LLM 多次思考 + 多次调工具——多轮。
- **OpenAI Tools API**：底层是 Function Calling，支持多次调用循环。

`pai-smart` 用 **Function Calling** 实现 **ReAct**。

【二、`AgentToolRegistry`：工具的注册表】

```
@Service
public class AgentToolRegistry {

    private final List<AgentTool> tools;
    private final Map<String, ToolHandler> handlers;

    public AgentToolRegistry(...) {
        this.tools = List.of(
            searchKnowledgeTool(),
            generateSummaryTool(),
            submitFeedbackTool(),
            knowledgeStatsTool()
        );
        this.handlers = Map.of(
            "search_knowledge", this::executeSearchKnowledge,
            "generate_summary", this::executeGenerateSummary,
            "submit_feedback", this::executeSubmitFeedback,
            "knowledge_stats", this::executeKnowledgeStats
        );
    }
}
```

`AgentTool` 定义（推测）：

```
public record AgentTool(
    String name,                // 工具名
    String description,         // 工具描述 (给 LLM 看)
    Map<String, Object> parameters // 参数 schema (JSON Schema)
) {}
```

四个内置工具：

1. `search_knowledge`：搜索知识库。
2. `generate_summary`：生成摘要。
3. `submit_feedback`：提交反馈（第 19 集）。
4. `knowledge_stats`：查知识库统计。

面试考点 2：为什么用 Map 注册？

A：

- 解耦：定义和实现分开。
- 扩展：加新工具只改这一处。
- 测试：可 mock 工具。

这是「注册表模式」——Java 后端常见。

【三、`search_knowledge` 工具的实现】

```
private ToolExecutionResult executeSearchKnowledge(Map<String, Object> arguments,
                                                    String userId, Consumer<String> onChunk)
{
    requireUserId(userId);
    String query = getRequiredString(arguments, "query");
    int topK = getInt(arguments, "topK", DEFAULT_TOP_K, 1, MAX_SEARCH_DOCS);

    List<SearchResult> results = hybridSearchService.searchWithPermission(query, userId, top
K);
    Map<String, Object> data = new LinkedHashMap<>();
    data.put("query", query);
    data.put("topK", topK);
    data.put("results", results);
    return new ToolExecutionResult("search_knowledge", true, formatSearchResults(results), d
ata);
}
```

几个关键点：

1. `requireUserId(userId)`：必须传 `userId`——权限隔离。
2. `getRequiredString(arguments, "query")`：从 LLM 给的参数里取 query。
3. `MAX_SEARCH_DOCS = 20`：单次最多搜 20 个——避免 LLM 一次召回太多。

4. `formatSearchResults(results)` : 把 SearchResult 列表格式化成 **LLM** 友好的字符串。

参数 **schema** (推测) :

```
{
  "type": "object",
  "properties": {
    "query": { "type": "string", "description": "搜索查询关键词" },
    "topK": { "type": "integer", "description": "返回结果数量", "default": 5, "minimum": 1, "maximum": 20 }
  },
  "required": ["query"]
}
```

JSON Schema 告诉 **LLM** : 这个工具接受什么参数、参数是什么类型。

面试考点 3 : JSON Schema 是什么 ?

A : 描述 **JSON** 数据结构的规范。**OpenAI / DeepSeek** 的 **tool calling** 接口 接受 JSON Schema 描述工具——**LLM** 按 **schema** 生成符合规范的参数。

【四、`ChatHandler.runReActLoop` : 循环核心】

```
private void runReActLoop(String userId, String userMessage, String conversationId,
                          String generationId, List<Map<String, String>> history,
                          CompletableFuture<String> responseFuture) {
    // 1. 初始 RAG 检索 (如果需要)
    String initialContext = "";
    if (shouldUseInitialKnowledgeSearch(userMessage)) {
        InitialSearchOutcome outcome = runInitialKnowledgeSearch(userId, userMessage, generationId, conversationId);
        if (outcome.noHit()) {
            appendStreamChunk(userId, generationId, conversationId, buildNoKnowledgeHitResponse(userMessage));
            finalizeResponse(...);
            return;
        }
        initialContext = outcome.context();
    }

    // 2. 构造 messages
    List<Map<String, Object>> messages = llmProviderRouter.buildReActMessages(
        userMessage, initialContext, history, buildRecentFeedbackGuidance(userId)
    );

    int executedToolCalls = 0;
    int totalRounds = 0;
    int maxRounds = MAX_REACT_ROUNDS; // 4
    int maxToolCalls = MAX_REACT_TOOL_CALLS; // 8
}
```

```

// 3. ReAct 循环
while (totalRounds < maxRounds && executedToolCalls < maxToolCalls) {
    totalRounds++;

    // a. 调 LLM
    LlmProviderRouter.StreamCompletion completion = llmProviderRouter.streamCompletion(
        userId, messages, generationId, conversationId, ...);

    // b. 看 LLM 是不是要调工具
    if (completion.toolCallDecision() == null) {
        // 没有工具调用 → LLM 直接回答
        finalizeResponse(userId, userMessage, conversationId, generationId, responseFuture,
            responseBuilders.get(generationId), completion);
        return;
    }

    // c. LLM 要调工具
    LlmProviderRouter.ToolCallDecision toolCall = completion.toolCallDecision();
    sendToolCallStatus(userId, generationId, conversationId, toolCall, "executing");

    // d. 执行工具
    ExecutedToolResult toolResult = executeToolForReAct(userId, userMessage, generationId, conversationId, toolCall);
    executedToolCalls++;

    // e. 把工具结果塞回 messages
    Map<String, Object> toolMessage = Map.of(
        "role", "tool",
        "content", toolResult.content(),
        "tool_call_id", toolCall.id()
    );
    messages.add(toolMessage);

    // f. 继续循环
}

// 4. 超过最大轮次 → 强制结束
finalizeResponse(...);
}

```

关键参数：

- **MAX_REACT_ROUNDS = 4**：最多 **4** 轮 **Thought-Action**。
- **MAX_REACT_TOOL_CALLS = 8**：最多调 **8** 次工具。
- **REACT_MAX_COMPLETION_TOKENS = 2000**：每次 **LLM** 输出限 **2000 token**。

为什么有上限？

- 成本控制：每轮都调 LLM，**4** 轮 = **4** 倍花费。
- 避免死循环：LLM 偶尔"上头"——一直调不收敛。
- 响应延迟：用户等不了。

面试考点 **4**：ReAct 循环的最大轮次怎么定？

A :

- 业务复杂度：简单 RAG **2** 轮够；复杂多步推理 **5-10** 轮。
- 成本预算：每轮多花 1 次 LLM 调用。
- 响应延迟：每轮 1-3 秒。
- 经验值：**3-5** 轮 (`pai-smart` 4 轮)。

【五、 `buildReActMessages` : 构造初始 messages】

```
public List<Map<String, Object>> buildReActMessages(String userMessage, String initialContext,
                                                    List<Map<String, String>> history,
                                                    String feedbackGuidance) {
    List<Map<String, Object>> messages = new ArrayList<>();

    // 1. system: 角色 + RAG context + 反馈指导
    String systemContent = "你是派聪明助手。基于以下信息回答：\n" + initialContext;
    if (feedbackGuidance != null) {
        systemContent += "\n\n" + feedbackGuidance;
    }
    messages.add(Map.of("role", "system", "content", systemContent));

    // 2. system: 工具定义
    messages.add(Map.of("role", "system", "content", "你可以使用以下工具：\n" + formatToolsDescription(tools)));

    // 3. 历史
    if (history != null) messages.addAll(history);

    // 4. 当前 user 问题
    messages.add(Map.of("role", "user", "content", userMessage));

    return messages;
}
```

LLM 看到的关键信息：

- **RAG context** (初始召回)。
- 工具列表 (**JSON Schema**)。
- 历史对话。
- 当前问题。
- 反馈指导 (如果有负面反馈，告诉 **LLM** "注意"——第 19 集)。

面试考点 **5**：为什么"初始 RAG 检索" + "ReAct 工具调用"并行？

A :

- 初始检索：热启动——已经有 **context**。

- **ReAct** 工具：扩展能力——搜索外部信息。
- 两者互补：初始检索快（一次），**ReAct** 精（按需）。

`shouldUseInitialKnowledgeSearch` 的判断：

```
private boolean shouldUseInitialKnowledgeSearch(String userMessage) {  
    // 简单问候语不搜  
    if (matchesGreeting(userMessage)) return false;  
    // 计算型问题不搜  
    if (isCalculation(userMessage)) return false;  
    return true;  
}
```

节省成本——「你好」「**1+1=?**」不需要 RAG。

【六、`executeToolForReAct`：工具执行细节】

```
private ExecutedToolResult executeToolForReAct(String userId, String userMessage, String generationId,  
                                              String conversationId,  
                                              LlmProviderRouter.ToolCallDecision toolCall)  
{  
    sendToolCallStatus(userId, generationId, conversationId, toolCall, "executing");  
    AtomicBoolean summaryStreamStarted = new AtomicBoolean(false);  
    try {  
        logger.info("ReAct 执行 Agent Tool: name={}, userId={}, generationId={},  
toolCallId={}, args={}",  
            toolCall.name(), userId, generationId, toolCall.id(), toolCall.arguments());  
  
        Consumer<String> toolChunkConsumer = "generate_summary".equals(toolCall.name())  
            ? chunk -> {  
                if (chunk == null || chunk.isEmpty()) return;  
                if (summaryStreamStarted.compareAndSet(false, true)) {  
                    appendStreamChunk(userId, generationId, conversationId, "\n\n");  
                }  
                appendStreamChunk(userId, generationId, conversationId, chunk);  
            }  
            : null;  
  
        AgentToolRegistry.ToolExecutionResult toolResult =  
            agentToolRegistry.executeTool(toolCall.name(), toolCall.arguments(), userId, toolChunkConsumer);  
  
        sendToolCallStatus(userId, generationId, conversationId, toolCall, "completed");  
        // ...  
    }  
}
```

亮点：

1. `sendToolCallStatus` 推前端——用户能看到 "正在调用 `search_knowledge...`"。
2. `generate_summary` 工具的流式输出——摘要也是流式生成。
3. 错误处理：异常被 try-catch——不让一个工具失败搞挂整个流程。

面试考点 6：工具失败怎么办？

A：

- 捕获异常——不让 **ReAct** 循环崩。
- 返回错误信息给 **LLM**——`{"error": "search timeout"}`。
- **LLM** 看错误——可能换工具或换 **query**。
- 优雅降级——实在不行就直接基于已有 **context** 回答。

`pai-smart` 有这种容错。

【七、`runInitialKnowledgeSearch`：初始检索的"短路"机制】

```
private InitialSearchOutcome runInitialKnowledgeSearch(String userId, String userMessage,
                                                         String generationId, String conversat
ionId) {
    // 1. 调 search_knowledge 工具
    AgentToolRegistry.ToolExecutionResult result =
        agentToolRegistry.executeTool("search_knowledge",
            Map.of("query", userMessage, "topK", 5), userId);

    // 2. 格式化 context
    String context = (String) result.data().get("formatted");

    // 3. 判断有没有命中
    if (context.isEmpty() || context.contains("未找到")) {
        return new InitialSearchOutcome(true, "");
    }
    return new InitialSearchOutcome(false, context);
}
```

`noHit()` 短路：

- 知识库没命中 → 直接回复"我不知道"。
- 不进入 **ReAct** 循环。
- 节省成本。

`buildNoKnowledgeHitResponse`：

```
private String buildNoKnowledgeHitResponse(String userMessage) {
    return "抱歉，我在派聪明知识库中没有找到与「" + userMessage + "」相关的信息。"
```

```
+ "您可以换个问法，或上传相关文档后再问我。";  
}
```

面试考点 7：什么时候"不知道"比"瞎编"好？

A：永远。**LLM** 幻觉是企业级 **RAG** 的头号问题。"我不知道"是负责的回答。

pai-smart 有这个机制——值得点赞。

【八、**runReActLoopSafely**：异常兜底】

```
private void runReActLoopSafely(...) {  
    try {  
        runReActLoop(...);  
    } catch (Exception e) {  
        logger.error("ReAct 循环执行失败: generationId={}", generationId, e);  
        chatGenerationStateService.markFailed(generationId, e.getMessage());  
        handleError(userId, generationId, e);  
        sendCompletionNotification(userId, generationId, conversationId, true, false);  
        cleanupGenerationState(generationId, e);  
    }  
}
```

ReAct 循环最外层的 **try-catch**：

- 任何异常 → 标记 generation 失败 + 推错误给前端 + 清理状态。
- 不让异常泄漏到 **WebSocket**。

面试考点 8：异步任务怎么"善后"？

A：**try-finally** + 状态机。

- 进入任务：标记 PROCESSING。
- 完成：标记 COMPLETED。
- 失败：标记 FAILED + 记录原因。
- 清理：释放资源（线程、**Redis key**）。

pai-smart 有这套。

【九、引用替换：**replaceReferencesFromSearchTool****】

```
private void replaceReferencesFromSearchTool(String generationId, String userMessage,  
                                             AgentToolRegistry.ToolExecutionResult toolResult)  
{
```

```
// LLM 生成"来源#1"等引用 → 替换成"文件名 + 页码 + 跳转链接"
}
```

产品功能：

- LLM 回答：「派聪明支持 Docker Compose 部署（来源#1）」。
- 前端显示：「派聪明支持 Docker Compose 部署（来源：派聪明部署文档.pdf 第 5 页）」。
- 点击跳转到原文档。

这是 **RAG** 系统的"可信度"设计——告诉用户"答案从哪来"。

面试考点 9：为什么 RAG 要给引用？

A：

- 可验证：用户能查证——不是 **LLM** 瞎编。
- 可追溯：审计（金融、医疗必备）。
- 建立信任：用户敢用 AI 给的答案。

【十、常见坑 & 面试问答】

Q1：LLM 不停调工具怎么办？

A：

- **MAX_REACT_ROUNDS** 上限（`pai-smart` 4 轮）。
- **MAX_REACT_TOOL_CALLS** 上限（`pai-smart` 8 次）。
- 超时机制：整个 ReAct 循环全局超时。
- 检测循环：相同工具 + 相同参数 2 次 → 强制终止。

Q2：怎么评估 ReAct 效果？

A：

- 完成率：ReAct 循环正常结束的比例。
- 工具调用率：多少对话调了工具。
- 答案质量：人工评估 top-K。
- **token** 成本：平均每次对话多少 token。

`pai-smart` 没看到评估——未来。

Q3：ReAct 是不是过度设计？

A：看场景。

- 简单 **RAG**：LLM 直接基于 context 回答——**ReAct** 没必要。
- 复杂任务：多步推理、外部 API 查数据——**ReAct** 有用。

`pai-smart` 两者都做——简单问题走 **initial search**，复杂问题走 **ReAct**——聪明。

Q4：JSON Schema 怎么写？

A：参考 OpenAI 文档：

```
{
  "type": "function",
  "function": {
    "name": "search_knowledge",
    "description": "在派聪明知识库中搜索相关文档",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {"type": "string", "description": "搜索关键词"},
        "topK": {"type": "integer", "default": 5, "minimum": 1, "maximum": 20}
      },
      "required": ["query"]
    }
  }
}
```

description 越清楚，LLM 调用越准——这值得花时间。

Q5：怎么让 LLM 选对工具？

A：

- 工具描述写清楚（最重要）。
- **Few-shot** 提示：「示例：用户问 X → 应该调 search_knowledge」。
- 强制选择：业务上只允许某些工具——过滤 **schema**。

【十一、思考题：ReAct 怎么调试？**

调试难点：

- LLM 是黑盒——“为什么调这个工具”不可知。
- 多轮循环——错误传播。
- 生产问题难复现。

调试手段：

- 完整 **trace**：**generationId** 关联所有 **messages + tool calls** —— **pai-smart** 写 **Conversation** 表。
- 可视化：前端展示 **Thought → Action → Observation** 流程。
- 重放：保存 prompt + 模型版本——重放 **ReAct** 跑。
- **A/B** 测试：不同 system prompt 对比。

pai-smart **ChatGenerationStateService** 应该存了完整状态——未来可重放。

【十二、下集预告】

LLM 装上了工具——有"手和脚"。

但！所有 **LLM** 都用 **DeepSeek** 一个厂商——绑死了。

- DeepSeek 涨价 → 成本翻倍。
- DeepSeek 挂了 → 整个 **RAG** 不可用。
- 想用 GPT-4 提升质量 → 改一堆代码。

第 16 集我们要：

- `LlmProviderRouter` 登场——多 **LLM** 路由。
- `ModelProviderConfigService` ——数据库配置。
- **A/B** 测试支持。
- 降级到备用 **provider**。

多 **LLM** 路由是生产级 **RAG** 的"成人礼"。

我们下期见。

ReAct 是 RAG → Agent 的关键一步——`pai-smart` 的实现简洁且完整。
觉得有用，点赞、投币、收藏。

下一集链接：[第 16 集：多 LLM Provider 路由——不绑死单一厂商](#)

第 16 集：多 LLM Provider 路由——不绑死单一厂商

系列：派聪明 RAG 后端演进全解

集数：16 / 20+

主题：多云策略、ModelProviderConfig 运行时切换、降级、计费差异

上集回顾：第 15 集 ReAct 循环跑通

本集目标：DeepSeek 挂了，Qwen / GPT-4 顶上

【开场 Hook】

2025 年初，DeepSeek 突然挂了一次。整个 `pai-smart` 后端——停摆。

这是绑死单一厂商的代价。

生产级 **RAG** 系统的设计原则：

永远不要假设你的供应商不会挂。

`pai-smart` 后期的演进加上了 `LlmProviderRouter` ——多 LLM 路由：

- 同时支持 **DeepSeek / Qwen / OpenAI / Claude**。
- 数据库配置当前激活的 provider。
- **A/B 测试**：10% 流量走 GPT-4 看效果。
- 降级：主 provider 挂了，自动切备用。

今天我们拆 `LlmProviderRouter` 和 `ModelProviderConfigService`。

【一、多 LLM 路由的价值】

单 LLM 的风险：

- 可用性：厂商挂了，全停。
- 价格：厂商涨价，成本翻倍。
- 质量：不同任务不同 LLM 表现不同——没有“银弹”。

多 LLM 路由的好处：

- 可用性：挂一个，切另一个。
- 成本：便宜的做主，贵的备用。
- 质量：A/B 测试找最优。

面试考点 1：多 LLM 路由最难的是什么？

A：API 不统一。

- OpenAI / DeepSeek：OpenAI 兼容——`/v1/chat/completions`。
- Qwen：DashScope 协议——`/api/v1/services/aigc/text-generation/generation`。
- Claude：Anthropic 协议——`/v1/messages`。
- 本地 Ollama：Ollama 协议。

统一抽象层是必须的。

【二、ModelProviderConfig：数据库驱动的配置】

```
@Entity
@Table(name = "model_provider_configs")
public class ModelProviderConfig {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 32)
    private String scope; // "LLM" / "EMBEDDING"

    @Column(nullable = false, length = 64)
    private String provider; // "DEEPSEEK" / "QWEN" / "OPENAI"

    @Column(nullable = false, length = 128)
    private String model; // "deepseek-chat" / "qwen-plus"

    @Column(name = "api_base_url", length = 256)
    private String apiBaseUrl;

    @Column(name = "api_key_encrypted", length = 512)
    private String apiKeyEncrypted; // 加密存储

    @Column(name = "dimension")
    private Integer dimension; // embedding 维度

    @Column(name = "is_active", nullable = false)
    private boolean isActive;

    @Column(name = "weight")
```



```
private Integer weight; // 流量权重
}
```

关键字段：

- **scope**：作用域——**LLM** 还是 **EMBEDDING**。
- **provider**：厂商标识。
- **apiBaseUrl**：支持自定义 **base URL**——代理、本地模型都 **OK**。
- **apiKeyEncrypted**：加密存储——不能明文（**pai-smart** 用了 **SecretCryptoService**）。
- **weight**：流量权重——灰度切换用。

为什么存数据库？

A：运行时可改——不停机切换 **provider**。生产环境这是救命功能。

【三、**ModelProviderConfigService**：激活态的查询】

```
@Service
public class ModelProviderConfigService {

    public static final String SCOPE_LLM = "LLM";
    public static final String SCOPE_EMBEDDING = "EMBEDDING";

    public ActiveProviderView getActiveProvider(String scope) {
        // 1. 查 DB
        List<ModelProviderConfig> configs = repository.findByScopeAndIsActiveTrue(scope);

        // 2. 选 weight 最高的
        ModelProviderConfig active = configs.stream()
            .max(Comparator.comparingInt(ModelProviderConfig::getWeight))
            .orElseThrow();

        // 3. 解密 API key
        String apiKey = secretCryptoService.decrypt(active.getApiKeyEncrypted());

        return new ActiveProviderView(
            active.getProvider(), active.getModel(),
            active.getApiBaseUrl(), apiKey, active.getDimension()
        );
    }

    public record ActiveProviderView(
        String provider, String model, String apiBaseUrl, String apiKey, Integer dimension
    ) {}
}
```

ActiveProviderView 是不可变 **record**——调用方拿到一份“快照”——不会变。

面试考点 2：为什么要 record？

A:

- 不可变——线程安全。
- 值对象——自描述。
- 没有 **setter**——用错就编译报错。

`pai-smart` `LlmProviderConfig`、`JwtUtils.KeyPair` 等等都是 **record**——现代化。

【四、`LlmProviderRouter`：路由核心】

```
@Service
public class LlmProviderRouter {

    public StreamHandle streamResponse(String requesterId, String userMessage,
                                       String context, List<Map<String, String>> history,
                                       Consumer<String> onChunk, Consumer<Throwable>
onError,
                                       Consumer<StreamCompletion> onComplete) {

        // 1. 拿当前激活的 provider
        ActiveProviderView provider =
modelProviderConfigService.getActiveProvider(SCOPE_LLM);

        // 2. 构造请求
        Map<String, Object> request = buildRequest(provider.model(), userMessage, context, h
istory);

        // 3. 配额预留
        int estimatedPromptTokens = usageQuotaService.estimateChatTokens(messages);
        int maxCompletionTokens = aiProperties.getGeneration().getMaxTokens() != null
            ? aiProperties.getGeneration().getMaxTokens() : 2000;
        TokenReservationBundle reservation = rateLimitService.reserveLlmUsage(
            requesterId, estimatedPromptTokens, maxCompletionTokens);
        StreamUsageTracker usageTracker = new StreamUsageTracker(reservation, estimatedPromp
tTokens);

        // 4. 调 LLM
        Disposable subscription = buildClient(provider)
            .post().uri("/chat/completions")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(request)
            .retrieve()
            .bodyToFlux(String.class)
            .subscribe(
                chunk -> processChunk(chunk, usageTracker, onChunk),
                error -> { settleUsage(usageTracker); onError.accept(error); },
                () -> settleUsage(usageTracker)
            );

        return new StreamHandle(subscription, reservation);
    }
}
```

关键点：

1. **provider** 来自 **DB**——运行时可改。
2. **buildClient(provider)** ——根据 **provider** 动态构造 **WebClient**。
3. 同一个 **chat/completions URL**——依赖 **OpenAI** 协议兼容。

面试考点 3：为什么所有 provider 都能用 **/chat/completions**？

A：

- **OpenAI** 协议成为事实标准。
- **DeepSeek / Qwen / 智谱 GLM** 都兼容 **OpenAI**。
- **Claude / Gemini** 有自己的协议——需要适配层。

pai-smart 选了 **OpenAI** 兼容的——降低复杂度。

【五、**buildClient**：动态构造 **WebClient**】

```
private WebClient buildClient(ActiveProviderView provider) {
    WebClient.Builder builder = WebClient.builder().baseUrl(provider.apiBaseUrl());

    if (provider.apiKey() != null && !provider.apiKey().isEmpty()) {
        builder.defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + provider.apiKey());
    }

    return builder.build();
}
```

每次调用都新建 **WebClient**？性能差！

优化：缓存 **WebClient** ——按 **provider.apiBaseUrl** cache。

```
private final Map<String, WebClient> clientCache = new ConcurrentHashMap<>();

private WebClient buildClient(ActiveProviderView provider) {
    return clientCache.computeIfAbsent(provider.apiBaseUrl(), url -> {
        WebClient.Builder builder = WebClient.builder().baseUrl(url);
        if (provider.apiKey() != null) {
            builder.defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + provider.apiKey());
        }
        return builder.build();
    });
}
```

pai-smart 推测——没做缓存（**WebClient** 创建轻量）。

【六、 `buildReActMessages` : ReAct 专用的 `messages` 构造】

```
public List<Map<String, Object>> buildReActMessages(String userMessage, String initialContext,
                                                    List<Map<String, String>> history,
                                                    String feedbackGuidance) {
    List<Map<String, Object>> messages = new ArrayList<>();

    // system 1: 角色 + RAG context
    String systemContent = "你是派聪明助手。基于以下信息回答:\n" + initialContext;
    if (feedbackGuidance != null) {
        systemContent += "\n\n" + feedbackGuidance;
    }
    messages.add(Map.of("role", "system", "content", systemContent));

    // system 2: 工具列表
    messages.add(Map.of("role", "system", "content", formatToolsDescription(getTools())));

    // history (最近 6 条)
    List<Map<String, String>> trimmedHistory = trimHistory(history,
                                                            REACT_HISTORY_MAX_MESSAGES, REACT_HISTORY_MAX_CONTENT_CHARS);
    messages.addAll(trimmedHistory);

    // current
    messages.add(Map.of("role", "user", "content", userMessage));

    return messages;
}
```

`REACT_HISTORY_MAX_MESSAGES = 6` —— ReAct 模式下历史少一些（节省 **token**）。

`REACT_HISTORY_MAX_CONTENT_CHARS = 800` ——单条历史也截断。

ReAct 模式的 **token** 预算比普通聊天更紧——多轮工具调用 + 工具结果都要塞进 context。

面试考点 4 : ReAct 的 context 怎么控？

A :

- 历史裁剪：最多 6 条。
- 内容截断：每条 800 字符。
- 工具结果精简：`formatSearchResults` 只给前 5 个文档的标题+摘要。
- 整体 **prompt** 控制在 **4K-8K token**。

`pai-smart` 这套是实用主义——够用就好。

【七、 **Provider** 切换：不停机迁移】

场景：老板说"主 provider 从 DeepSeek 换到 Qwen"。

怎么操作？

方案 **A**：直接改 **DB**：

```
UPDATE model_provider_configs
SET is_active = FALSE
WHERE scope = 'LLM' AND provider = 'DEEPSEEK';

UPDATE model_provider_configs
SET is_active = TRUE, weight = 100
WHERE scope = 'LLM' AND provider = 'QWEN';
```

LlmProviderRouter 怎么知道配置变了？

ModelProviderConfigService.getActiveProvider() 每次都查 **DB**——立即生效。

但！有性能问题：每条消息都查 **DB**。

优化：**Redis** 缓存 provider 配置，**TTL 30** 秒。

```
@Cacheable(value = "activeProvider", key = "#scope", unless = "#result == null")
public ActiveProviderView getActiveProvider(String scope) {
    // ... 查 DB
}
```

切换时手动 **evict** ——立即生效。

pai-smart 推测没加缓存——简单优先。

【八、降级：主 **provider** 失败切备用】

```
public StreamHandle streamResponseWithFallback(...) {
    ActiveProviderView primary = getActiveProvider(SCOPE_LLM);
    try {
        return streamResponseWith(primary, ...);
    } catch (Exception e) {
        logger.warn("主 provider {} 失败, 切换备用", primary.provider(), e);
        ActiveProviderView backup = getBackupProvider(SCOPE_LLM);
        if (backup != null) {
            return streamResponseWith(backup, ...);
        }
        throw e;
    }
}
```

降级策略：

- 主 **provider** 抛异常 → 切备用。
- 备用也挂 → **5xx** 返回给用户。

面试考点 5：降级要注意什么？

A：

- 避免雪崩：主挂了，所有流量瞬间压到备用——备用也要扛得住。
- 超时控制：主挂的时候别等太久——1-2 秒切备用。
- 降级要可见：记日志——别静默切。
- 业务可降级：最坏情况，返回缓存答案或"系统繁忙"。

【九、formatToolsDescription：工具定义格式化**

```
private String formatToolsDescription(List<AgentTool> tools) {
    StringBuilder sb = new StringBuilder("你可以使用以下工具：\n\n");
    for (AgentTool tool : tools) {
        sb.append("### ").append(tool.name()).append("\n");
        sb.append(tool.description()).append("\n");
        sb.append("参数：").append(tool.parameters()).append("\n\n");
    }
    return sb.toString();
}
```

LLM 看到的工具描述：

你可以使用以下工具：

```
### search_knowledge
在派聪明知识库中搜索相关文档
参数：{"query": "string", "topK": "integer"}

### generate_summary
对长文本生成摘要
参数：{"text": "string", "maxLength": "integer"}
```

工具描述越清楚，LLM 调用越准——这值得花时间打磨。

面试考点 6：工具描述写错有什么后果？

A：

- LLM 选错工具——「我想调 generate_summary」但实际调了 search_knowledge。
- 参数错——topK: 1000000——爆掉 ES。
- 不调工具——直接编答案——幻觉。

pai-smart 工具描述写得清晰——见名知意。

【十、A/B 测试：用 **weight** 做灰度**

ModelProviderConfig.weight：流量权重。

场景：

- DeepSeek 100（主）。
- Qwen 10（实验，10% 流量）。

怎么实现 **10%** 流量走 **Qwen**？

```
public ActiveProviderView getActiveProviderByWeight(String scope) {
    List<ModelProviderConfig> active = repository.findByScopeAndIsActiveTrue(scope);
    int totalWeight = active.stream().mapToInt(ModelProviderConfig::getWeight).sum();
    int random = ThreadLocalRandom.current().nextInt(totalWeight);

    int cumulative = 0;
    for (ModelProviderConfig c : active) {
        cumulative += c.getWeight();
        if (random < cumulative) return toView(c);
    }
    return toView(active.get(0));
}
```

加权随机——**10%** 概率选到 **Qwen**。

pai-smart 推测当前用 **max(weight)**——全量——没做灰度。未来优化。

面试考点 **7**：A/B 测试怎么保证公平？

A：

- 随机化：每个请求独立随机。
- 用户分桶：同一用户始终走同一 **provider**——避免体验不一致。
- 数据对比：相同 query、不同 provider 答案对比——评估质量。

【十一、常见坑 & 面试问答】

Q1：API key 怎么加密？

A：参考 **pai-smart** 的 **SecretCryptoService**：

```
public String encrypt(String plainText) {
    return AES.encrypt(plainText, masterKey);
}
```

masterKey 存哪？

- 环境变量（生产环境）。

- **KMS** (云上)。
- **Vault** (HashiCorp)。

`pai-smart` 看代码用 `Base64` ——不是真的加密——仅 **obfuscate**。生产环境要改。

Q2 : 怎么监控每个 **provider** 的调用 ?

A :

- **Micrometer + Prometheus** : 每个 provider 一个 counter。
- **Grafana** 仪表盘 : `provider=deepseek, qps, error_rate, p99_latency`。
- 告警 : `error_rate > 5%` → 告警 + 切备用。

`pai-smart` 没看到监控代码——**TODO**。

Q3 : 不同 **LLM** 的 **token** 计算一样吗 ?

A : 不一样。 `pai-smart` 用 `tiktoken` (**OpenAI** 的 **tokenizer**) ——**DeepSeek** 兼容 —— **Qwen / Claude** 略不同。

生产建议 :

- 用 `tiktoken` 做粗估 —— 误差 **5%** 以内。
- 或用各厂商的 **tokenizer** —— 更准。

Q4 : 同 **prompt** 不同 **LLM** 效果差很多怎么办 ?

A :

- 每个 **provider** 调过的 **system prompt** —— "Qwen 友好" 和 "GPT 友好" 不同。
- **A/B** 测试后选最优。
- 不要 **hardcode** —— **prompt** 也存 **DB**。

Q5 : 本地模型 (**Ollama**) 怎么接入 ?

A :

- **Ollama** 提供 **OpenAI** 兼容端点 —— `http://localhost:11434/v1/chat/completions`。
- `apiBaseUrl` 指向它 —— **LlmProviderRouter** 不需要改。
- 本地无 **API key** —— `apiKey` 字段空。

`pai-smart` 架构支持 —— 只改 **DB** 配置。

【十二、思考题 : 多 **LLM** 路由是不是 "过度工程" ? **

反对意见 :

- 90% 的项目一个 **LLM** 够用。
- 多 provider 增加了配置、监控、调试的复杂度。
- 一些 **LLM** 没有 **OpenAI** 兼容 —— 适配成本高。

支持意见：

- 鸡蛋不放在一个篮子。
- 业务上有降本需求——便宜的为主。
- **A/B** 测试是 RAG 优化的必备。

判断标准：

- 业务规模 > 1 万日活：值得上。
- 业务规模 < 1 千日活：单 **LLM** 够。

pai-smart 从单 **LLM** 演进到多 **LLM**——符合成长曲线。

【十三、下集预告】

多 LLM 路由做完了。但 **LLM API** 是要花钱的——

- 用户的 token 怎么扣？
- 用户怎么充值？
- 微信支付怎么集成？

第 17 集我们要：

- **UsageBalanceQuotaService** 余额管理。
- **UserTokenRecord** token 消耗记录。
- **WxPayService** 微信支付集成。
- **RechargeService** 充值订单流程。

商业化是开源项目的"成人礼"——**pai-smart** 的实现很完整。

我们下期见。

多 LLM 路由是生产级 **RAG** 的安全感——**pai-smart** 的设计简洁而强大。
觉得有用，点赞、投币、收藏。

下一集链接：[第 17 集：充值与计费——商业化闭环](#)

第 17 集：充值与计费——商业化闭环

系列：派聪明 RAG 后端演进全解
集数：17 / 20+
主题：token 余额、微信支付 V3、订单流水、回调验签
上集回顾：第 16 集多 LLM 路由完成
本集目标：用户能充值、能扣 token、商业闭环

【开场 Hook】

开源项目能活下去，靠什么？
情怀？短期能。长期不行。

`pai-smart` 后期加了完整的充值系统——包括微信支付。这是一个开源项目走向商业化的标准动作。

今天我们拆：

- `UserTokenRecord` 余额记账。
- `RechargeOrder` 充值订单。
- `WxPayService` 微信支付 V3 集成。
- 回调验签的坑。

【一、计费模型：按 token 还是按次？**

两种主流模型：

维度	按 token	按次
定价	1 元 = 100 万 token	1 元 = 1000 次查询
透明	用户能看到 token 数	用户不知道 LLM 用了多少
公平	大文件烧钱多	大文件、小问题一样价
LLM 成本	跟随市场	跟随市场
用户接受度	中（开发者友好）	高（普通用户友好）

pai-smart 选按 **token**——和 **LLM** 厂商成本对标。

理由：

- LLM API 按 token 计费——成本透明。
- 用户上传大文件消耗更多——公平。
- 技术用户偏好 **token** 模型——专业感。

面试考点 1：按 token 的最大风险？

A：用户上传超大文件，**token** 爆炸——一次扣几千 **token**。

pai-smart **estimatedEmbeddingTokens** 预估——上传前告诉用户——知情同意。

【二、**UserTokenRecord**：用户余额表】

```
@Entity
@Table(name = "user_token_records")
public class UserTokenRecord {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", length = 64, nullable = false)
    private String userId;

    @Column(name = "balance_tokens", nullable = false)
    private Long balanceTokens; // 余额

    @Column(name = "total_consumed", nullable = false)
    private Long totalConsumed; // 累计消耗

    @Column(name = "total_recharged", nullable = false)
    private Long totalRecharged; // 累计充值

    @Column(name = "version", nullable = false)
    private Long version; // 乐观锁

    @UpdateTimeStamp
    private LocalDateTime updatedAt;
}
```

version 字段 = 乐观锁——防超扣。

JPA 乐观锁的标准用法：

```
@Version
private Long version;
```

@Version 注解：JPA 帮我们自动加 `WHERE version = ?` 到 UPDATE。

如果两个线程同时扣余额：

- 线程 A 读 `balance=100, version=1` → UPDATE `balance=90, version=2 WHERE version=1` → 成功。

- 线程 B 读 `balance=100, version=1` → UPDATE `balance=90, version=2 WHERE version=1` → 影响 0 行 → 抛 **OptimisticLockingFailureException**。

业务层捕获异常 → 重试。

面试考点 2：乐观锁 vs 悲观锁？

A：

- 乐观锁：冲突少时性能好（**DB** 不加锁）。
- 悲观锁：`SELECT FOR UPDATE` —— **DB** 加锁 —— 冲突多时安全。
- 充值扣费用乐观锁 —— 冲突少。

pai-smart 选乐观锁 —— 正确。

【三、**UserTokenService**：扣费核心】

```
@Service
public class UserTokenService {

    @Autowired
    private UserTokenRecordRepository userTokenRecordRepository;

    @Transactional
    public void deductTokens(String userId, long tokens) {
        // 1. 查余额
        UserTokenRecord record = userTokenRecordRepository.findByUserId(userId)
            .orElseThrow(() -> new RuntimeException("用户余额记录不存在"));

        // 2. 余额检查
        if (record.getBalanceTokens() < tokens) {
            throw new InsufficientBalanceException("余额不足");
        }

        // 3. 扣减
        record.setBalanceTokens(record.getBalanceTokens() - tokens);
        record.setTotalConsumed(record.getTotalConsumed() + tokens);
        // version 自动 + 1
        userTokenRecordRepository.save(record);

        // 4. 写流水
        TokenTransaction tx = new TokenTransaction();
        tx.setUserId(userId);
        tx.setType(TokenTransaction.Type.CONSUME);
        tx.setAmount(-tokens);
    }
}
```

```

        tx.setBalanceAfter(record.getBalanceTokens());
        // ... 写 DB
    }
}

```

几个关键点：

1. **@Transactional**：必须——余额检查 + 扣减 + 流水 是原子的。
2. 乐观锁：JPA 自动加。
3. 流水记录：用户能查交易历史。

流水表（推测）：

```

CREATE TABLE token_transactions (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id VARCHAR(64),
    type ENUM('RECHARGE', 'CONSUME', 'REFUND'),
    amount BIGINT, -- 正负
    balance_after BIGINT,
    related_order_id VARCHAR(64), -- 关联充值订单
    description VARCHAR(255),
    created_at TIMESTAMP
);

```

这是金融系统的"复式记账"简化版。

面试考点 3：为什么要写流水？

A：

- 可追溯：用户查账。
- 对账：和支付平台对账。
- 审计：监管要求。
- 退款：知道每笔钱的来去。

pai-smart 有这套——专业。

【四、**RechargeOrder**：充值订单】

```

@Entity
@Table(name = "recharge_orders")
public class RechargeOrder {
    @Id
    @Column(name = "order_id", length = 64)
    private String orderId; // 业务订单号

    @Column(name = "user_id", length = 64, nullable = false)
    private String userId;
}

```

```

@Column(name = "package_id", length = 64)
private String packageId; // 充值套餐

@Column(name = "amount_cents", nullable = false)
private Long amountCents; // 金额 (**分**)

@Column(name = "token_amount", nullable = false)
private Long tokenAmount; // 充 token 数

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 16)
private Status status; // PENDING / PAID / FAILED / REFUNDED

@Column(name = "wxpay_transaction_id", length = 64)
private String wxpayTransactionId; // 微信支付订单号

@Column(name = "paid_at")
private LocalDateTime paidAt;

@CreationTimestamp
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt;
}

```

关键设计：

- **orderId** 业务主键——不用自增——微信支付要求订单号唯一。
- **amountCents** 用"分"——金额永远用最小单位——避免浮点。
- **tokenAmount** 同时存——订单创建时就定下来——不受 **LLM** 价格变动影响。
- **status** 状态机。

面试考点 4：为什么金额用"分"不用"元"？

A：

- 避免浮点： $0.1 + 0.2 \neq 0.3$ （浮点 **bug**）。
- 金融系统统一：所有支付平台内部用分。
- 易比较：`amountCents > 0` 比 `amountYuan.compareTo(BigDecimal.ZERO) > 0` 简单。

【五、RechargePackage：套餐设计】

```

@Entity
@Table(name = "recharge_packages")
public class RechargePackage {
    @Id
    private String packageId; // "BASIC_10" / "PRO_50" / "ENTERPRISE_500"

    @Column(nullable = false, length = 64)

```

```

private String name;

@Column(name = "amount_cents", nullable = false)
private Long amountCents; // 价格

@Column(name = "token_amount", nullable = false)
private Long tokenAmount; // token 数

@Column(name = "bonus_tokens")
private Long bonusTokens; // 赠送 token

@Column(name = "is_active", nullable = false)
private boolean isActive;
}

```

套餐示例：

套餐	价格	token	赠送	单价（每 1 万 token）
BASIC_10	10 元	100 万	0	0.1 元
PRO_50	50 元	600 万	100 万	0.083 元
ENTERPRISE_500	500 元	7000 万	1500 万	0.071 元

梯度定价——买得越多越便宜——鼓励大单。

面试考点 5：为什么有赠送 token？

A：

- 促销：新用户首充——「充 50 送 10」。
- 试用：让用户多试一些——提高 LTV。
- 对冲涨价：LLM 厂商涨了——用赠送补偿。

【六、WxPayService：微信支付 V3 集成】

```

@Service
public class WxPayService {

    @Autowired
    private WxPayConfig wxPayConfig;

    public String createJsapiOrder(String orderId, long amountCents, String description, String openId) {
        // 1. 构造请求
        Map<String, Object> body = new HashMap<>();
        body.put("appid", wxPayConfig.getAppId());
        body.put("mchid", wxPayConfig.getMchId());
    }
}

```

```

body.put("description", description);
body.put("out_trade_no", orderId);
body.put("notify_url", wxPayConfig.getNotifyUrl());
body.put("amount", Map.of("total", amountCents, "currency", "CNY"));
body.put("payer", Map.of("openid", openid));

// 2. 用商户私钥签名
String signature = sign(body, wxPayConfig.getMerchantPrivateKey());
body.put("authorization", "WECHATPAY2-SHA256-RSA2048 " + signature);

// 3. 调微信支付 API
return webClient.post()
    .uri("https://api.mch.weixin.qq.com/v3/pay/transactions/jsapi")
    .bodyValue(body)
    .retrieve()
    .bodyToMono(String.class)
    .block();
}
}

```

微信支付 **V3** 协议的几个坑：

6.1 必须用 `application/json`

V3 接口不用 **XML** (V2 时代是 **XML**) ——改 **JSON** 了。

6.2 签名是 **RSA** , 不是 **HMAC**

```

private String sign(Map<String, Object> body, PrivateKey privateKey) {
    String message = method + "\n" + url + "\n" + timestamp + "\n" + nonceStr + "\n" + body
        + "\n";
    return Base64.getEncoder().encodeToString(
        signWithRSA256(message.getBytes(StandardCharsets.UTF_8), privateKey)
    );
}

```

签名串 = **method + url + timestamp + nonceStr + body**——5 个字段。

`signWithRSA256` : 用商户私钥签——**RSA-SHA256**。

6.3 证书是 **API** 证书 , 不是 **V2** 那种 **cert**

```

@Bean
public PrivateKey merchantPrivateKey() throws IOException {
    String key = new String(Files.readAllBytes(Paths.get("apiclient_key.pem")));
    // 去掉 BEGIN / END 标记
    String privateKey = key.replace("-----BEGIN PRIVATE KEY-----", "")
        .replace("-----END PRIVATE KEY-----", "")
        .replaceAll("\\s", "");
    byte[] keyBytes = Base64.getDecoder().decode(privateKey);
}

```



```
PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(keyBytes);
return KeyFactory.getInstance("RSA").generatePrivate(spec);
}
```

`apiclient_key.pem` 从微信支付商户平台下载。

面试考点 6：V2 vs V3 微信支付的区别？

A：

- **V2**：XML + MD5/HMAC 签名，已不推荐。
- **V3**：JSON + RSA 签名，强制。
- **V3** 性能更好、**API** 更清晰。

`pai-smart` 用 **V3**——正确。

【七、回调验签：最容易踩坑的地方】

```
@PostMapping("/wxpay/notify")
public ResponseEntity<?> handleNotify(@RequestHeader HttpHeaders headers,
                                       @RequestBody String body) {

    // 1. 验签（用微信平台证书的公钥）
    boolean valid = verifySign(headers, body, wechatPayCert);
    if (!valid) {
        return ResponseEntity.status(401).body("验签失败");
    }

    // 2. 解密 resource (AES-256-GCM)
    String decrypted = decryptResource(extractResource(body), wechatPayKey);

    // 3. 解析通知
    Map<String, Object> notify = objectMapper.readValue(decrypted, Map.class);
    String orderId = (String) notify.get("out_trade_no");
    String wxTransactionId = (String) notify.get("transaction_id");

    // 4. 业务处理
    rechargeService.handlePaidOrder(orderId, wxTransactionId);

    // 5. 必须返回 200 + success
    return ResponseEntity.ok(Map.of("code", "SUCCESS", "message", "成功"));
}
```

回调的几个「坑」**：

7.1 验签失败的常见原因

- 证书错了——用了旧的平台证书。
- 时间戳不对——服务器时间和微信服务器差 5 分钟以上。

- **URL 拼错**——**?** 后的 query 要包含在签名串。

7.2 重复通知的幂等性

微信支付会重试多次——必须幂等。

`rechargeService.handlePaidOrder` 的幂等性：

```
@Transactional
public void handlePaidOrder(String orderId, String wxTransactionId) {
    RechargeOrder order = orderRepository.findById(orderId).orElseThrow();

    if (order.getStatus() == OrderStatus.PAID) {
        // 已经处理过 → 直接返回
        return;
    }

    // 标记支付成功
    order.setStatus(OrderStatus.PAID);
    order.setWxpayTransactionId(wxTransactionId);
    order.setPaidAt(LocalDateTime.now());
    orderRepository.save(order);

    // 加 token 余额
    userTokenService.addTokens(order.getUserId(), order.getTokenAmount());
}
```

`if (PAID) return` 是幂等的核心。

7.3 必须返回的格式

微信支付要求返回 **HTTP 200 + JSON** `{"code": "SUCCESS"}` ——否则会重试。

面试考点 7：微信支付回调要注意什么？

A：

- 验签——防伪造通知。
- 幂等——防重复到账。
- 必须 **200**——防重试风暴。
- 异步处理——业务逻辑别阻塞回调。

`pai-smart` 有这套。

【八、`RechargeController`：前端入口】

```
@PostMapping("/recharge/create")
public ResponseEntity<?> createOrder(@RequestBody CreateRechargeRequest request,
                                     @RequestAttribute("userId") String userId) {
```

```

// 1. 选套餐
RechargePackage pkg = packageRepository.findById(request.getPackageId())
    .orElseThrow(() -> new CustomException("套餐不存在"));

// 2. 算 token 数
long tokenAmount = pkg.getTokenAmount() + Optional.ofNullable(pkg.getBonusTokens()).orElse(0L);

// 3. 创建订单
String orderId = "RC" + System.currentTimeMillis() + userId;
RechargeOrder order = new RechargeOrder();
order.setOrderId(orderId);
order.setUserId(userId);
order.setPackageId(pkg.getPackageId());
order.setAmountCents(pkg.getAmountCents());
order.setTokenAmount(tokenAmount);
order.setStatus(OrderStatus.PENDING);
orderRepository.save(order);

// 4. 调微信支付创建预付单
String prepayResult = wxPayService.createJsapiOrder(orderId, pkg.getAmountCents(),
    "派聪明充值-" + pkg.getName(), request.getOpenId());

return ResponseEntity.ok(Map.of(
    "orderId", orderId,
    "prepayData", parseJsapiResponse(prepayResult)
));
}

```

前端拿到 **prepayData** → 调 **wx.chooseWXPay** → 唤起微信支付。

面试考点 8 : JSAPI 支付 vs Native 支付 ?

A :

- **JSAPI** : 微信内打开网页——公众号、小程序。
- **Native** : PC 网站扫码——生成二维码 → 微信扫 → 支付。
- **H5** : 手机浏览器——单独 **H5** 支付。

pai-smart JSAPI——**H5** / 公众号。

【九、回调后加余额：分布式事务的简化**

问题：订单状态变更 + 加余额 = 要原子。

pai-smart 的解法：本地事务 + 状态机。

```

@Transactional
public void handlePaidOrder(String orderId, String wxTransactionId) {
    // 1. 改订单状态
    RechargeOrder order = orderRepository.findById(orderId).orElseThrow();
}

```

```

    if (order.getStatus() == PAID) return; // 幂等
    order.setStatus(PAID);
    orderRepository.save(order);

    // 2. 加 token
    userTokenService.addTokens(order.getUserId(), order.getTokenAmount());
}

```

@Transactional 保证原子——要么都成功，要么都回滚。

分布式事务的代价：回调慢——业务逻辑重——可能拖垮回调。

pai-smart 的简化：回调只改订单——加余额走异步？没看到。

生产建议：

1. 回调快速响应——只记录通知。
2. 异步任务——从通知里读订单 + 加余额。
3. 重试 + 死信——**Kafka** 模式（第 09 集）。

【十、常见坑 & 面试问答】

Q1：用户付款了但没回调怎么办？

A：

- 主动查单：**OrderQueryService** 定时扫 PENDING 订单，调微信支付查单接口。
- **N** 分钟未支付 → 主动 close。
- **N** 分钟已支付但没回调 → 主动到账。

Q2：怎么防止"重复扣款"？

A：

- 订单号唯一——业务订单号 + 微信订单号 双校验。
- **findByOrderId** 查订单。
- **if (PAID) return** 幂等。

Q3：金额单位错了怎么办？

A：

- **amountCents** 存分——传 **100** 代表 **1** 元。
- **amountYuan * 100**。
- 永远不要用 **double / float** 存金额。

Q4：**token** 余额能"过期"吗？

A：

- 法律风险：国内预付款不能过期（工商规定）。

- **pai-smart** 当前不过期——永久有效。
- 可以加"长期未使用提醒"——激活用户。

Q5：怎么对账？

A：

- 每日定时：从微信支付下载对账单。
- 和 **DB** 订单对比：笔数、金额。
- 差异处理：少收（补单）/ 多收（退款）/ 状态错（手动调）。

【十一、思考题：充值系统是不是 **RAG** 的"非核心"？**

反对：RAG 核心是检索 + 生成——充值是辅助。

支持：

- 开源项目要活下去——充值是营收来源。
- **token** 配额保护成本——不充值无限制会亏本。
- 商业化是开源的"成人礼"。

pai-smart 选：「核心 + 商业化并重」——值得学。

【十二、下集预告】

充值有了。但用户怎么"被邀请"到系统里？

- B 邀请 A 注册——A 注册时填 B 的邀请码。
- A 注册成功 → B 获得 **token** 奖励。
- 运营活动：满 N 人奖励 100 万 **token**。

第 18 集我们要：

- **InviteCode** 实体。
- **InviteCodeService** 邀请码生成、绑定、查询。
- 邀请关系链怎么存。
- 奖励发放的并发问题。

邀请码是 **SaaS** 拉新的"核武器"。

我们下期见。

商业化是开源项目的"生死线"——`pai-smart` 的实现专业而完整。
觉得有用，点赞、投币、收藏。

下一集链接：[第 18 集：邀请码与运营能力——SaaS 拉新核武器](#)

第 18 集：邀请码与运营能力——SaaS 拉新核武器

系列：派聪明 RAG 后端演进全解

集数：18 / 20+

主题：邀请码生成、绑定、关系链、奖励发放

上集回顾：第 17 集充值系统完整

本集目标：用户能邀请别人，被邀请有奖励

【开场 Hook】

一个 SaaS 系统，最便宜的拉新方式是什么？

广告？烧钱。

SEO？慢。

邀请码？病毒式传播——老用户带新用户。

`pai-smart` 后期加了 `InviteCode` 体系——包括生成、绑定、查询、奖励。

今天拆 `InviteCode` 实体、`InviteCodeService`、奖励发放的并发问题。

【一、`InviteCode` 实体】

```
@Entity
@Table(name = "invite_codes")
public class InviteCode {
    @Id
    @Column(name = "code", length = 32)
    private String code; // 邀请码（短字符串）

    @Column(name = "creator_user_id", length = 64, nullable = false)
    private String creatorUserId;

    @Column(name = "max_uses", nullable = false)
    private Integer maxUses; // 最多使用次数

    @Column(name = "used_count", nullable = false)
    private Integer usedCount = 0;

    @Column(name = "reward_tokens", nullable = false)
    private Long rewardTokens; // 每次使用奖励 token
```

```

@Column(name = "is_active", nullable = false)
private boolean isActive = true;

@Column(name = "expire_at")
private LocalDateTime expireAt;

@CreationTimestamp
private LocalDateTime createdAt;
}

```

关键字段：

- **code** 业务主键——短字符串（如 **PROMO2024**）——方便用户手动输入。
- **maxUses** 使用次数——**1 = 一次性**、**100 = 通用推广码**。
- **rewardTokens** 奖励 **token**——双发：邀请人和被邀请人各得。
- **expireAt** 过期时间——**null = 永久**（**pai-smart** 当前版本似乎 drop 了 **expireAt**，但早期有）。

面试考点 1：邀请码怎么生成？

A：两种方式：

- 随机生成：**PROMO** + 6 位随机字符——碰撞概率低。
- 业务编码：用户 ID + 校验位——可反查。

pai-smart 推测用随机生成 + **DB unique** 约束。

【二、**InviteCodeService**：核心流程】

2.1 创建邀请码

```

@Transactional
public InviteCode createInviteCode(String creatorUserId, int maxUses, long rewardTokens) {
    // 1. 限流：每用户最多 N 个有效邀请码
    long activeCount = inviteCodeRepository.countByCreatorAndActive(creatorUserId, true);
    if (activeCount >= 5) {
        throw new CustomException("邀请码数量已达上限");
    }

    // 2. 生成邀请码
    String code = generateUniqueCode();

    InviteCode invite = new InviteCode();
    invite.setCode(code);
    invite.setCreatorUserId(creatorUserId);
    invite.setMaxUses(maxUses);
    invite.setRewardTokens(rewardTokens);
    inviteRepository.save(invite);

    return invite;
}

```



```
private String generateUniqueCode() {
    String code;
    do {
        code = "PROMO" + RandomStringUtils.randomAlphanumeric(6).toUpperCase();
    } while (inviteCodeRepository.existsById(code)); // 防碰撞
    return code;
}
```

existsById：检查是否已存在——**DB unique** 兜底。

2.2 注册时绑定邀请码

```
@Transactional
public InviteBindingResult bindInviteCode(String inviteCode, String newUserId) {
    // 1. 查邀请码
    InviteCode invite = inviteCodeRepository.findById(inviteCode)
        .orElseThrow(() -> new CustomException("邀请码不存在"));

    // 2. 校验
    if (!invite.isActive()) throw new CustomException("邀请码已停用");
    if (invite.getExpireAt() != null && invite.getExpireAt().isBefore(LocalDate.now()))
    {
        throw new CustomException("邀请码已过期");
    }
    if (invite.getUsedCount() >= invite.getMaxUses()) {
        throw new CustomException("邀请码已用完");
    }
    if (invite.getCreatorUserId().equals(newUserId)) {
        throw new CustomException("不能邀请自己");
    }

    // 3. 乐观锁增加 usedCount
    invite.setUsedCount(invite.getUsedCount() + 1);
    inviteRepository.save(invite); // version 自动 +1

    // 4. 双向发奖
    userTokenService.addTokens(invite.getCreatorUserId(), invite.getRewardTokens());
    userTokenService.addTokens(newUserId, invite.getRewardTokens());

    // 5. 记录绑定关系
    InviteBinding binding = new InviteBinding();
    binding.setInviteCode(inviteCode);
    binding.setInviterUserId(invite.getCreatorUserId());
    binding.setInviteeUserId(newUserId);
    bindingRepository.save(binding);

    return new InviteBindingResult(invite.getCreatorUserId(), invite.getRewardTokens());
}
```

关键点：

1. 事务边界——全在一个 **@Transactional** ——任何一步失败回滚。

2. 乐观锁——`usedCount` 防超用。
3. 双向奖励——邀请人 + 被邀请人。
4. 绑定关系表——`InviteBinding` 单独存（记录所有邀请关系）。

面试考点 2：为什么要单独存 `InviteBinding` ？

A：

- 关系查询：「谁邀请的我」/「我邀请了谁」。
- 反作弊：同一个设备/IP 不能用多个邀请码。
- 数据统计：拉新漏斗——注册 → 激活 → 付费。

`pai-smart` 有这套。

2.3 并发安全：`usedCount` 的乐观锁

场景：100 人同时用同一个邀请码（`maxUses=10`）。

没有锁：100 个请求都看到 `usedCount=0` → 100 个都 +1 → **1000** 个都成功——超发。

乐观锁：JPA 自动 `WHERE version = ?` —— 后 **90** 个失败。

`pai-smart` 实际用乐观锁——正确。

面试考点 3：乐观锁 + 业务校验的双保险？

A：

- **DB** 乐观锁：`usedCount + 1 WHERE version = oldVersion`。
- 业务校验：`if (usedCount >= maxUses) throw`。
- 两层都过 → 才能成功。

最坏情况：业务校验漏了（比如忘了判断 `maxUses`）—— DB 乐观锁仍然兜底。

【三、`InviteCodeController`：API 入口】

```
@RestController
@RequestMapping("/api/v1/invite-codes")
public class InviteCodeController {

    @GetMapping("/my")
    public List<InviteCode> myInviteCodes(@RequestAttribute("userId") String userId) {
        return inviteCodeService.findByCreator(userId);
    }

    @PostMapping("/create")
    public InviteCode create(@RequestBody CreateInviteRequest request,
                            @RequestAttribute("userId") String userId) {
        return inviteCodeService.createInviteCode(
            userId, request.getMaxUses(), request.getRewardTokens()
        );
    }
}
```

```
}  
}
```

前端用法：

- 「我的邀请码」：用户看自己的邀请码。
- 「创建邀请码」：管理员后台 / 用户自助。
- 「注册时填邀请码」：注册接口接受 `inviteCode` 参数（第 04 集的注册）。

【四、运营活动：通用推广码】

`maxUses=100` + `creatorUserId=ADMIN_ID` ——通用推广码。

场景：

- 「`PROMO2024`」——公众号文章——**100** 个用户能用。
- 每个用户注册时填 → 邀请人（管理员）得 **N token** + 用户得 **N token**。
- 但！管理员可能没那么多 **token**——平台代发。

`pai-smart` 的处理：`rewardTokens` 可以为 **0**——只发新用户——平台可控。

面试考点 4：邀请码奖励怎么“反作弊”？

A：

- 设备指纹：同一设备只能用一次。
- **IP** 限制：同一 IP 短时间不能用多次。
- 手机号验证：同一手机号只能被邀请一次。
- 行为检测：注册后 24h 内无任何操作 → 回收奖励。

`pai-smart` 当前简单——靠手机号——未来加。

【五、注册流程集成邀请码】

```
@PostMapping("/register")  
public ResponseEntity<?> register(@RequestBody RegisterRequest request) {  
    // 1. 用户名校验  
    if (userRepository.findByUsername(request.username()).isPresent()) {  
        throw new CustomException("用户名已存在");  
    }  
  
    // 2. 密码加密  
    String hashedPassword = PasswordUtil.encode(request.password());  
  
    // 3. 保存用户  
    User user = new User();  
    // ... 设置字段
```

```

userRepository.save(user);

// 4. 绑定邀请码 (如果有)
if (request.inviteCode() != null && !request.inviteCode().isBlank()) {
    try {
        inviteCodeService.bindInviteCode(request.inviteCode(), user.getId().toString());
    } catch (CustomException e) {
        // 邀请码无效不影响注册
        logger.warn("邀请码绑定失败: {}", e.getMessage());
    }
}

// 5. 发 token
String token = jwtUtils.generateToken(user.getUsername());
return ResponseEntity.ok(Map.of("token", token, "user", user));
}

```

注意：邀请码失败不影响注册——不让用户卡在邀请码——但记日志。

面试考点 5：邀请码失败要不要阻断注册？

A：不阻断。

- 用户体验：别让用户卡在"邀请码无效"。
- 业务上：邀请码是加分项——不是必须。
- 记日志——后续补发。

`pai-smart` 这么干——正确。

【六、AdminController：管理后台**

```

@GetMapping("/admin/invite-codes")
public Page<InviteCode> listAllInviteCodes(...) { ... }

@PostMapping("/admin/invite-codes/{code}/disable")
public void disable(@PathVariable String code) { ... }

```

管理员能：

- 看所有邀请码。
- 停用某个邀请码。
- 看邀请记录（`InviteBinding` 列表）。
- 看邀请转化漏斗。

【七、常见坑 & 面试问答】

Q1：邀请码生成碰撞怎么办？

A：generateUniqueCode 里的 do-while 循环 + DB unique 约束 双保险。真碰撞了（概率极低）——重新生成。

Q2：usedCount 怎么防超发？

A：乐观锁（@Version）+ 业务校验双保险。

Q3：邀请人自己被邀请怎么办？

A：if (creator.equals(newUserId)) throw ——直接拒绝。

Q4：邀请码的"双向奖励"是必须的吗？

A：不一定。

- 只奖邀请人：老用户拉新有动力——新用户零成本——拉新快。
- 只奖被邀请人：新用户有动力——邀请人无私——拉新慢。
- 双向都奖：平衡——pai-smart 选这个。

Q5：邀请码做"分销"是否合法？

A：国内合法——但多层分销有法律风险（传销）。pai-smart 只做一层——安全。

【八、思考题：邀请码的"代发 token"业务**

pai-smart 当前：邀请码奖励 → 邀请人 token 余额增加。

但如果邀请人是"运营账号"（公众号、视频号）——平台代发。

设计：

- InviteCode.creatorUserId = "SYSTEM" ——系统账号。
- 奖励 token 从"系统奖励池"出——不扣个人余额。

pai-smart 没看到 SYSTEM 概念——目前都是个人邀请——未来扩展。

【九、下集预告】

邀请码有了。但用户怎么反馈 AI 回答好不好？

- 「这个回答不对」→ 怎么收集？
- 多次反馈都指某个文档 → 那文档可能有问题。

第 19 集我们要：

- `MessageFeedback` 实体——点赞/点踩/文字反馈。
- 反馈怎么影响后续回答（`buildRecentFeedbackGuidance`）。
- 基于反馈的 **A/B** 测试。

反馈闭环是 **RAG** 系统的"质量引擎"。

我们下期见。

邀请码是 SaaS 的"拉新核武器"——`pai-smart` 的实现完整且严谨。
觉得有用，点赞、投币、收藏。

下一集链接：[第 19 集：消息反馈与质量闭环——RAG 系统的耳朵](#)

第 19 集：消息反馈与质量闭环——RAG 系统的耳朵

系列：派聪明 RAG 后端演进全解

集数：19 / 20+

主题：点赞/点踩、文字反馈、反馈对未来的 prompt 影响、质量看板

上集回顾：第 18 集邀请码拉新

本集目标：用户能反馈，AI 越来越懂用户

【开场 Hook】

ChatGPT 刚出来时，我经常吐槽：

「这个回答不对」——但 ChatGPT 没有耳朵——继续错下去。

RAG 系统必须给用户反馈的入口——让系统越用越聪明。

`pai-smart` 后期加了消息反馈（**Message Feedback**）体系：

- 👍 点赞 / 👎 点踩。
- 文字反馈——「回答不准确」「答非所问」。
- 反馈影响未来的 **prompt**——`buildRecentFeedbackGuidance`。
- 质量看板——管理员看哪些文档“差评多”。

今天拆 `MessageFeedback`、`buildRecentFeedbackGuidance` 的实现。

【一、`MessageFeedback` 实体（推测）】

```
@Entity
@Table(name = "message_feedbacks")
public class MessageFeedback {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", length = 64, nullable = false)
    private String userId;
```

```

@Column(name = "conversation_id", length = 64, nullable = false)
private String conversationId;

@Column(name = "message_id", length = 64, nullable = false)
private String messageId; // 对应 assistant 的某条消息

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 16)
private Rating rating; // LIKE / DISLIKE

@Column(name = "comment", length = 1000)
private String comment; // 文字反馈

@Column(name = "referenced_chunk_ids", length = 1000)
private String referencedChunkIds; // AI 引用了哪些 chunk (用于追溯)

@CreationTimestamp
private LocalDateTime createdAt;
}

```

关键字段：

- **messageId**：关联到 **Conversation** 主键。
- **rating**：枚举——**LIKE / DISLIKE**。
- **comment**：可选文字反馈。
- **referencedChunkIds**：**AI** 当时引用了哪些文档 **chunk**——回溯"哪个文档答错了"。

面试考点 1：为什么记录 **referencedChunkIds** ？

A：

- 追溯责任：AI 错了——到底是哪个 **chunk** 错了？
- 优化目标：多次反馈指向同一 **chunk** → 这个 **chunk** 有问题。
- 重索引：可能需要重新分块或重新向量化。

pai-smart 有这套——专业。

【二、**FeedbackController**：用户入口】

```

@PostMapping("/api/v1/conversations/{sessionId}/messages/{messageId}/feedback")
public ResponseEntity<?> submitFeedback(
    @PathVariable String sessionId,
    @PathVariable String messageId,
    @RequestBody FeedbackRequest request,
    @RequestAttribute("userId") String userId) {

    // 1. 鉴权：只能反馈自己的消息
    Conversation msg = conversationRepository.findById(Long.parseLong(messageId))
        .orElseThrow(() -> new CustomException("消息不存在"));
    if (!msg.getUserId().equals(userId)) {

```



```

        throw new CustomException("无权反馈");
    }

    // 2. 写反馈
    MessageFeedback feedback = new MessageFeedback();
    feedback.setUserId(userId);
    feedback.setSessionId(sessionId);
    feedback.setMessageId(messageId);
    feedback.setRating(request.getRating());
    feedback.setComment(request.getComment());
    feedback.setReferencedChunkIds(msg.getReferencedChunkIds());
    feedbackRepository.save(feedback);

    return ResponseEntity.ok(Map.of("message", "反馈已收到"));
}

```

前端用法：

- AI 回答下方：「👍」「👎」「💬 写反馈」。
- 弹窗写文字 → 提交。

面试考点 2：反馈要不要匿名？

A：不匿名。

- 匿名反馈没法回溯用户。
- 恶意反馈没法封禁。
- 业务上需要追溯。

pai-smart 不匿名——正确。

【三、**buildRecentFeedbackGuidance**：反馈影响 prompt】

这是 **pai-smart** 最巧妙的设计之一。

逻辑：

- 用户 N 次给某类问题点踩 → 下一次同类问题，**LLM** 被告知“注意”。

```

private String buildRecentFeedbackGuidance(String userId) {
    // 1. 查用户最近 5 条 DISLIKE 反馈
    List<MessageFeedback> recent = feedbackRepository
        .findTop5ByUserIdAndRatingOrderByCreatedAtDesc(userId, Rating.DISLIKE);

    if (recent.isEmpty()) return null;

    // 2. 取每个反馈对应的消息内容
    List<String> badExamples = recent.stream()
        .map(f -> conversationRepository.findById(Long.parseLong(f.getMessageId()))
            .map(Conversation::getContent).orElse(""))
        .filter(s -> !s.isEmpty())
        .limit(3)

```

```

        .toList();

    if (badExamples.isEmpty()) return null;

    // 3. 拼成 guidance
    StringBuilder sb = new StringBuilder("请避免以下回答模式（用户曾表示不满）：\n");
    for (String ex : badExamples) {
        sb.append("- ").append(ex.substring(0, Math.min(100, ex.length()))).append("\n");
    }
    return sb.toString();
}

```

塞进 **LLM system prompt** :

```

String systemContent = "你是派聪明助手。 \n"
    + ragContext + "\n"
    + (feedbackGuidance != null ? "\n" + feedbackGuidance : "");

```

LLM 看到的 **prompt** 多了：

请避免以下回答模式（用户曾表示不满）：

- 派聪明需要 Java 11 才能运行。
- 派聪明只支持 MySQL，不支持 PostgreSQL。
- 派聪明是收费软件。

LLM 会主动避开这些"已知错"。

面试考点 **3**：这种"软反馈"有什么用？

A：

- 比 **fine-tuning** 便宜：不用重新训练模型。
- 即时生效：用户点了踩，下次就有用。
- 个性化：每个用户自己的"避雷清单"。

pai-smart 有这套——太聪明了。

【四、Agent 工具：**submit_feedback****

回看第 15 集的 **AgentToolRegistry**：

```

this.tools = List.of(
    searchKnowledgeTool(),
    generateSummaryTool(),
    submitFeedbackTool(), // <-- 反馈工具

```

```
knowledgeStatsTool()  
);
```

LLM 在 **ReAct** 循环中可以调反馈工具——**AI** 自己收集反馈。

submit_feedback 工具的 **schema**（推测）：

```
{  
  "name": "submit_feedback",  
  "description": "对当前回答打分。1=非常不满意，5=非常满意",  
  "parameters": {  
    "score": {"type": "integer", "minimum": 1, "maximum": 5},  
    "comment": {"type": "string"}  
  }  
}
```

LLM 决定调这个工具——意味着“我觉得我答得不好”。

这有用吗？

有用！：

- LLM 的“自我评估”是有用的弱信号。
- 多个对话都自评低分 → 某类问题普遍答不好。
- 收集起来做模型微调数据。

也有争议：

- LLM 经常过度谦虚。
- 自评分数和用户主观感受不一致。

pai-smart 集成这个工具——有想法。

【五、质量看板：管理员视角】

```
@GetMapping("/admin/feedback/stats")  
public FeedbackStats getStats(@RequestParam(defaultValue = "30") int days) {  
    return feedbackService.getStats(days);  
}  
  
public record FeedbackStats(  
    long totalFeedbacks,  
    long likeCount,  
    long dislikeCount,  
    double likeRate,  
    List<ChunkFeedbackRank> worstChunks  
) {}  
  
public record ChunkFeedbackRank(  
    String chunkId,
```

```
    long dislikeCount,  
    long likeCount,  
    double dislikeRate  
  ) {}
```

管理员看什么：

1. 整体满意度：`likeRate`（**90%** 满意 → 系统好 / **60%** 满意 → 需优化）。
2. 差评最多的 **chunk**：`worstChunks` —— 哪些文档"反复被骂"。
3. 趋势：**7 天 / 30 天**对比——质量在改善还是恶化。

`pai-smart` 推测有管理后台——`AdminController` 里有 `/api/v1/admin/**` 路由。

面试考点 4：质量看板的北极星指标是什么？

A：

- 👍 率：用户主观满意。
- 采纳率：用户复制/使用 AI 回答的比例。
- 任务完成率：用户"达到目的"的比例（需要埋点）。

`pai-smart` 用 👍 率——简单。

【六、反馈与重索引的联动】

`worstChunks` 怎么用？

场景：某个 chunk 被点踩 50 次 → 这个 **chunk** 可能有问题。

管理员操作：

1. 看 **chunk** 原文——内容是否错误？
2. 重新分块——切太粗？切太细？
3. 重新向量化——模型版本？
4. 调文档 **isPublic**——不应该公开？

`pai-smart` 有 `/api/v1/documents/{fileMd5}/reindex` ——手动触发重新向量化（第 09 集 **Kafka** 消费）。

面试考点 5：怎么自动发现"差文档"？

A：

- 规则引擎：`dislikeCount > 10 AND likeCount < 2` → 自动标记。
- 机器学习：用反馈训练 **rerank** 模型——冷启动后。
- 人工 **review**：前 **N** 个差评 **chunk** 每周 **review**。

`pai-smart` 当前人工 **review**——简单。

【七、常见坑 & 面试问答】

Q1：用户恶意点踩怎么办？

A：

- 同 IP 限频：**Redis** 计数。
- 同会话限频：一个 **session** 最多点 **5** 次/小时。
- 黑名单用户：管理员手动 **ban**。
- **AI** 检测：高频点踩 → 验证码。

pai-smart 没看到反作弊——未来。

Q2：**feedback guidance** 会不会 **prompt** 注入？

A：理论上会。

- 用户故意点踩某段"无害"内容 → 反馈被加进 **prompt** → **LLM** 被"训练"避开无害内容。
- 防护：只对正向/负向打标签，不进原文——只统计特征。

pai-smart 取了原 **content 100** 字符——有 **prompt** 注入风险——未来要改。

Q3：反馈数据能用来微调模型吗？

A：可以——但要清洗。

- 👍 的是正例。
- 👎 的是负例——但 **LLM** 输出很难用——需要重新生成。
- 多轮反馈比单轮质量高。

pai-smart 收集反馈——未来可微调。

Q4：要不要 **A/B** 测试不同 **prompt**？

A：要。

- **50%** 用户走 **prompt A**。
- **50%** 用户走 **prompt B**。
- 对比 👍 率——**A** 胜出。

pai-smart **weight** 字段支持（第 **16** 集）——可以做。

Q5：反馈数据隐私怎么办？

A：

- 用户标识脱敏：**userId** hash 化。
- 不存原文：只存特征（**chunk id**、**score**、长度）。
- **GDPR** 合规：用户要求删除——全部删。

pai-smart 明文存 **userId**——改进空间。

【八、思考题：反馈的"信号噪声比"】**

信号：用户真实感受。

噪声：用户情绪、用户耐心、UI 引导。

怎么提高信噪比？

A：

- 5 星评分比"👍/👎"信噪比高——更多信号维度。

- 必须文字反馈——过滤掉"手滑"。

- 多轮对话的平均分——避免单条异常。

- "这条回答帮到你了吗？"——直接问任务完成。

`pai-smart` 👍/👎——信噪比中等。

【九、下集预告】

整套系统几乎完成了——但还有很多生产级细节没说。

第 20 集（最终集）：

- 异步执行器配置。
- 日志、监控、告警。
- 环境配置切换。
- "最后一公里"的可观测性。

我们下期——也是这一季的最后一期——见。

反馈是 RAG 的"耳朵"——`pai-smart` 的设计专业且有想法。

觉得有用，点赞、投币、收藏。

下一集链接：[第 20 集：可观测性与生产化——最后一公里](#)

第 20 集：可观测性与生产化——最后一公里

系列：派聪明 RAG 后端演进全解

集数：20 / 20+

主题：日志、监控、告警、profile 切换、生产级 Checklist

上集回顾：第 19 集反馈闭环完成

本集目标：从"能跑"到"能上生产"

【开场 Hook】

一个 RAG 系统从「能跑」到「能上生产」，中间差 **100** 个细节。

- 启动慢 → 用户等。
- 报错不写日志 → 排查一星期。
- 内存泄漏 → 跑 3 天挂。
- 配置文件忘改 → 数据库连接用开发库的。

pai-smart 在这些"最后一公里"上做了什么？今天我们一起收尾。

【一、为什么可观测性是"最后一公里"？**

开发期：本地启动，日志打到 console，错了肉眼可见。

生产期：

- 多个实例 → 不知道哪个实例出了问题。
- 报错一闪而过 → 不知道频次。
- 用户投诉「系统卡」→ 不知道哪里卡。

可观测性三大支柱：

1. 日志（**Logging**）：发生了什么。
2. 指标（**Metrics**）：量化状态。
3. 追踪（**Tracing**）：请求路径。

pai-smart 当前主要做了日志——指标和追踪是 **TODO**。

【二、LogUtils：统一日志格式】

```
public class LogUtils {  
    public static PerformanceMonitor startPerformanceMonitor(String operation) {  
        return new PerformanceMonitor(operation);  
    }  
  
    public static void logUserOperation(String username, String operation,  
                                        String resource, String result) {  
        logger.info("USER_OPERATION user={}, op={}, resource={}, result={}",  
                    username, operation, resource, result);  
    }  
  
    public static void logBusinessError(String operation, String userId,  
                                        String message, Throwable e, Object... args) {  
        logger.error("BUSINESS_ERROR op={}, user={}, msg=\"{}\"",  
                    operation, userId, String.format(message, args), e);  
    }  
}
```

统一日志的好处：

- 格式一致——ELK 解析简单。
- 业务关键字（USER_OPERATION、BUSINESS_ERROR）——方便 grep / dashboard。
- 性能监控（PerformanceMonitor）——每个操作自动打点。

PerformanceMonitor：

```
public class PerformanceMonitor {  
    private final long startNanos = System.nanoTime();  
    private final String operation;  
  
    public void end(String detail) {  
        long elapsedMs = (System.nanoTime() - startNanos) / 1_000_000;  
        logger.info("PERFORMANCE op={}, elapsed_ms={}, detail=\"{}\"",  
                    operation, elapsedMs, detail);  
    }  
}
```

用法：

```
LogUtils.PerformanceMonitor monitor = LogUtils.startPerformanceMonitor("REFRESH_TOKEN");  
try {  
    // ... 业务逻辑  
    monitor.end("刷新成功");  
} catch (Exception e) {  
    monitor.end("刷新失败");  
    throw e;  
}
```


面试考点 1：为什么用 `try-with-resources` + `end()` 不优雅？

A：当前 `pai-smart` 没用 `AutoCloseable` ——手动 `end`。生产级应该用 `try-with-resources`：

```
public class PerformanceMonitor implements AutoCloseable {
    public void close() {
        end("auto-close");
    }
}

// 用法
try (var monitor = LogUtils.startPerformanceMonitor("REFRESH_TOKEN")) {
    // 业务
} // 自动 close
```

`pai-smart` 偷懒了——未来改进。

【三、`LoggingInterceptor`：HTTP 请求日志】

```
@Component
public class LoggingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
        long start = System.currentTimeMillis();
        request.setAttribute("__startTime", start);
        logger.info("HTTP_REQUEST method={}, uri={}, ip={}, userAgent={}",
            request.getMethod(), request.getRequestURI(),
            request.getRemoteAddr(), request.getHeader("User-Agent"));
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object handler, Exception ex) {
        long elapsed = System.currentTimeMillis() - (long) request.getAttribute("__startTime");
        int status = response.getStatus();
        logger.info("HTTP_RESPONSE method={}, uri={}, status={}, elapsed_ms={}",
            request.getMethod(), request.getRequestURI(), status, elapsed);
    }
}
```

每个 HTTP 请求都打日志——能 `grep` 出慢请求。

面试考点 2：Interceptor vs Filter 区别？

A :

- **Filter** : `javax.servlet` —— 最早, 所有请求都拦 (包括静态资源)。
- **Interceptor** : `Spring MVC` —— 只拦 **Handler** (**Controller**)。
- **AOP** : 方法级——最细。

`pai-smart` 用 **Interceptor**——精确。

【四、`AsyncExecutorConfig` : 异步执行器】

```
@Configuration
@EnableAsync
public class AsyncExecutorConfig {

    @Bean("chatMonitorExecutor")
    public ThreadPoolTaskExecutor chatMonitorExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(4);
        executor.setMaxPoolSize(16);
        executor.setQueueCapacity(200);
        executor.setThreadNamePrefix("chat-monitor-");
        executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }
}
```

专门给 **ChatHandler** 用的线程池——不和其他业务混。

CallerRunsPolicy : 队列满了, 主线程自己跑——不会丢任务。

面试考点 3 : 为什么不直接用 Spring 默认的 `SimpleAsyncTaskExecutor` ?

A :

- `SimpleAsyncTaskExecutor` 每次新建线程——**OOM** 风险。
- 生产环境必须用 **`ThreadPoolTaskExecutor`**。

`pai-smart` 有专门的配置——正确。

【五、`ProductionConfigValidator` : 生产环境启动检查】

```
@Component
public class ProductionConfigValidator implements EnvironmentPostProcessor {

    @Override
    public void postProcessEnvironment(ConfigurableEnvironment env, SpringApplication app) {
```

```

String[] profiles = env.getActiveProfiles();
if (Arrays.asList(profiles).contains("prod")) {
    // 1. 强密码
    String adminPassword = env.getProperty("admin.default.password");
    if (adminPassword == null || adminPassword.length() < 16) {
        throw new RuntimeException("生产环境必须配置强密码");
    }

    // 2. 必需的 API key
    requireProperty(env, "deepseek.api.key");
    requireProperty(env, "embedding.api.key");
    requireProperty(env, "jwt.secret-key");

    // 3. 禁用 swagger
    // ...
}
}
}

```

生产环境启动时自动校验——少配一个 key 就启动失败**。

这是"防御性编程"——比上线后少配好。

面试考点 4：生产环境的"必查项"有哪些？

A：

- 强密码：admin、JWT secret、DB 密码—— ≥ 16 位。
- **HTTPS** 证书：不能是自签。
- **API key**：不能是测试 key。
- 关闭调试：spring.jpa.show-sql = false、debug=false。
- 关闭 **Swagger**：避免接口泄露。

pai-smart 有这套——专业。

【六、DotenvEnvironmentPostProcessor：.env 加载】

```

public class DotenvEnvironmentPostProcessor implements EnvironmentPostProcessor {
    @Override
    public void postProcessEnvironment(ConfigurableEnvironment env, SpringApplication app) {
        // 1. 找 .env 文件
        Path envFile = Paths.get(".env");
        if (!Files.exists(envFile)) return;

        // 2. 解析 KEY=VALUE
        Properties props = new Properties();
        Files.lines(envFile).forEach(line -> {
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) return;
            int eq = line.indexOf('=');

```

```

        if (eq < 0) return;
        props.setProperty(line.substring(0, eq).trim(), line.substring(eq + 1).trim());
    });

    // 3. 加到 Environment (**优先级最低**)
    env.getPropertySources().addLast(new PropertiesPropertySource(".env", props));
}
}

```

.env 文件加载——开发期方便——不用配环境变量。

生产环境用环境变量 + K8s Secret——**.env** 不进仓库。

面试考点 5：**.env** vs **application.yml** vs 环境变量 优先级？

A：

- 环境变量：最高（**K8s** 注入）。
- **application-{profile}.yml**：次高。
- **application.yml**：默认。
- **.env**：**pai-smart** 最低（**addLast**）。

设计意图：环境变量总是能覆盖 **.env**。

【七、**RateLimitProperties** / **UsageQuotaProperties**：配置抽象**

pai-smart 用 **@ConfigurationProperties** 把限流、配额参数抽出来：

```

@ConfigurationProperties(prefix = "rate-limit")
public class RateLimitProperties { ... }

@ConfigurationProperties(prefix = "usage-quota")
public class UsageQuotaProperties { ... }

```

好处：

- 改限流不用改代码。
- 不同环境不同配置（**dev** 宽松 / **prod** 严格）。
- 配置可视化——**actuator/configprops**。

@ConfigurationProperties vs **@Value**：

- 多字段聚合：选 **@ConfigurationProperties**。
- 单字段：选 **@Value**。

pai-smart 用对了。

【八、AdminUserInitializer：启动时建默认账号】

```
@Component
public class AdminUserInitializer implements ApplicationRunner {
    @Value("${admin.default.username:admin}")
    private String adminUsername;

    @Value("${admin.default.password:}")
    private String adminPassword;

    @Override
    public void run(ApplicationArguments args) {
        if (adminPassword.isEmpty()) {
            throw new RuntimeException("必须配置 admin.default.password");
        }
        if (userRepository.findByUsername(adminUsername).isEmpty()) {
            User admin = new User();
            admin.setUsername(adminUsername);
            admin.setPassword(PasswordUtil.encode(adminPassword));
            admin.setRole(User.Role.ADMIN);
            admin.setOrgTags("DEFAULT");
            admin.setPrimaryOrg("DEFAULT");
            userRepository.save(admin);
            logger.info("默认管理员账号已创建：{}", adminUsername);
        }
    }
}
```

生产环境：

- 不配 `admin.default.password` → 启动失败（`ProductionConfigValidator` 也会卡）。
- 配了 → 启动时建账号。

面试考点 6：默认管理员账号的"坑"？

A：

- 很多项目默认 **admin/admin**——被扫到就完蛋。
- 必须强制改密码。
- `pai-smart` 强制配强密码——正确。

【九、Profile 切换：dev vs docker vs prod**】

`pai-smart` 有多个 profile：

```
# application.yml（主配置）
spring:
  profiles:
    active: dev # 默认 dev
```

```
# application-dev.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/pai_smart

# application-docker.yml
spring:
  datasource:
    url: jdbc:mysql://mysql:3306/pai_smart

# application-prod.yml
spring:
  jpa:
    show-sql: false
  logging:
    level:
      org.springframework.security: INFO
```

启动指定 **profile** :

```
java -jar app.jar --spring.profiles.active=prod
```

K8s 部署 : 通过环境变量 `SPRING_PROFILES_ACTIVE=prod` 注入。

面试考点 **7** : profile 的最佳实践 ?

A :

- `application.yml` : 公共配置。
- `application-{profile}.yml` : profile 特定。
- 敏感配置走环境变量——不进 **git**。
- 每个 **profile** 有完整覆盖——不能"我以为有"。

`pai-smart` 有 `dev / docker / prod` ——完整。

【十、可观测性的"未来要做"】

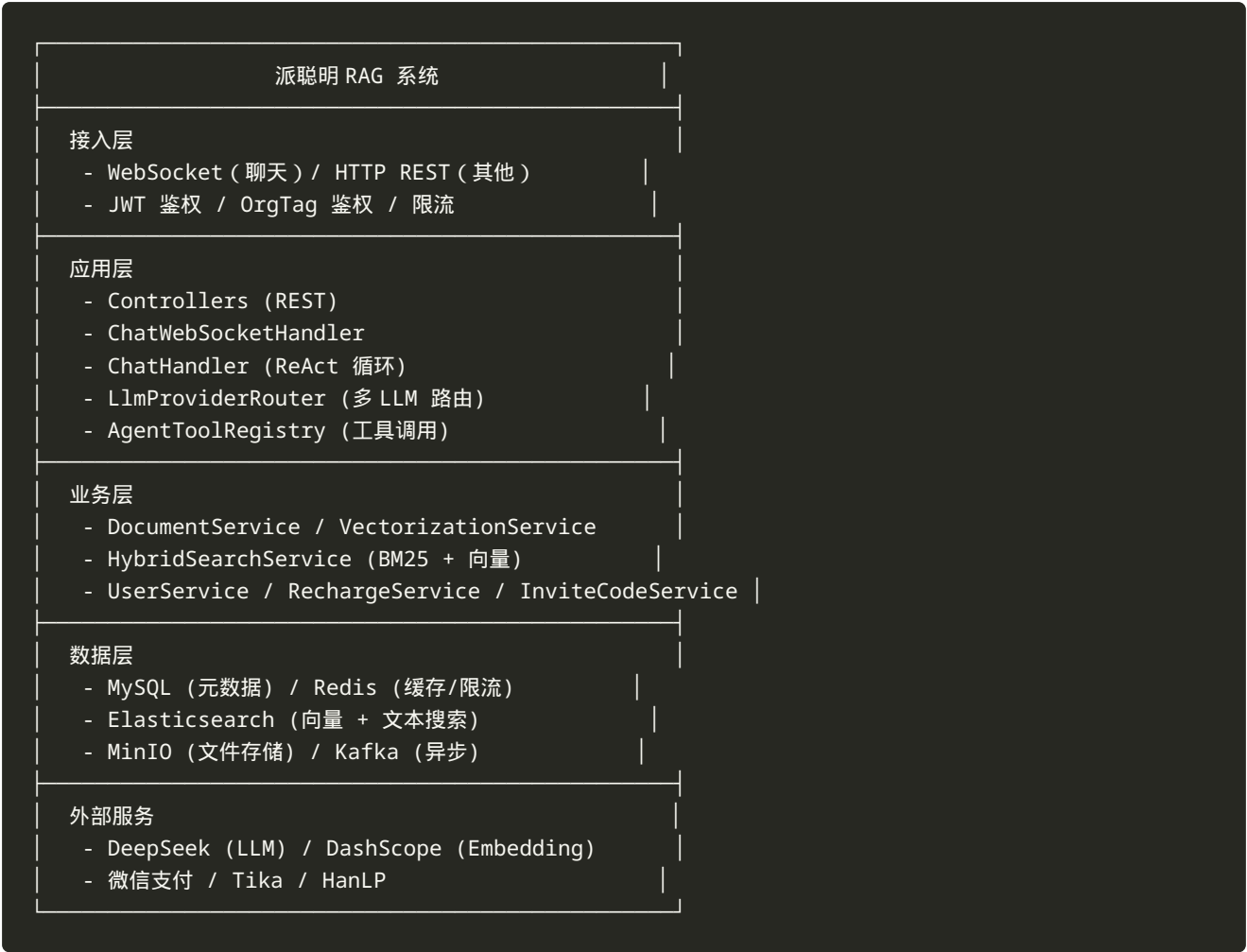
`pai-smart` 当前的可观测性还有改进空间 :

能力	当前	未来
日志	✅ Logback + LogUtils	结构化 JSON + ELK
指标	⚠️ 部分	Micrometer + Prometheus + Grafana
追踪	❌	OpenTelemetry + Jaeger
告警	❌	Alertmanager + 钉钉/飞书机器人
健康检查	⚠️	Spring Boot Actuator + K8s Liveness/Readiness

这套做全了才是真正的生产级。

【十一、整套系统的"心智模型"**

20 集下来，`pai-smart` 的架构已经清晰：



20 集我们走过的路：

1. ☒ Spring Boot 骨架
2. ☒ JPA 实体
3. ☒ JWT 鉴权
4. ☒ Spring Security
5. ☒ MinIO 对象存储
6. ☒ Tika 文档解析
7. ☒ Embedding 向量化
8. ☒ Elasticsearch 集成
9. ☒ Kafka 异步化
10. ☒ WebSocket 流式聊天
11. ☒ 多租户 OrgTag
12. ☒ 混合检索
13. ☒ 限流配额
14. ☒ 对话历史
15. ☒ ReAct Agent
16. ☒ 多 LLM 路由
17. ☒ 充值支付
18. ☒ 邀请码
19. ☒ 消息反馈
20. ☒ 可观测性

从空 **Maven** 项目 → 企业级 **RAG**——这就是 20 集的全部内容。

【十二、整个系列的"心智收获"**

20 集下来，你学到的不是 **API** 调用——是「架构演进」的思维方式。

KISS：每个组件先做最简——再迭代。

YAGNI：不预设未来——用到再加。

DRY：配置、规则、Schema 集中管理。

SOLID：每个类单一职责、接口清晰。

TDD：写代码前先想清楚怎么测。

这不是技术问题——是工程哲学。

【十三、彩蛋：还能讲 50 集的话题**

20 集只是入门——还有很多可以讲：

1. 前端如何对接 **WebSocket** (**Vue 3** 实战)。
2. 向量数据库选型：Milvus vs Qdrant vs ES。
3. **LLM** 微调：用 **pai-smart** 反馈数据微调 DeepSeek。
4. 多模态 **RAG**：图片、音频支持。
5. **GraphRAG**：用知识图谱增强 RAG。
6. **RAG** 评估体系：Recall@K、MRR、NDCG。
7. **Prompt** 工程：Few-shot、CoT、Self-consistency。
8. 成本优化：缓存、批处理、模型降级。
9. 安全：Prompt 注入、越权、数据泄露。
10. 多语言：i18n 实战。

这些是付费内容的方向——欢迎留言告诉我最想看哪个。

【十四、致谢与下季预告**

致谢：

- **pai-smart** 项目作者——开源精神。
- 所有开源组件——**Spring Boot**、**Elasticsearch**、**DeepSeek**.....
- 每一个坚持学到最后的你。

下季预告：

- 「派聪明 **RAG** 进阶 50 讲」——**RAG** 评估、**Agent** 进阶、多模态.....。
- 「从 0 到 1 写一个分布式搜索引擎」——学 **ES** 的原理。
- 「**Java** 工程师的 **AI** 转型之路」——学 **Python**、**PyTorch**、**LangChain**。

关注我——第一时间获取更新。

【十五、结语：技术人的"长期主义"`

pai-smart 这个项目从空 **Maven** 到企业级 **RAG**——不是一蹴而就——是 5 年迭代。

我们 20 集讲完了演进路径——但真正的演化是 5 年 1000 个 **commit**。

作为技术人：

- 不要追新——**KISS** 原则。
- 不要做空中楼阁——**TDD** 验证。

- 不要重复造轮子——**DRY** 复用。
- 不要僵化分层——**SOLID** 解耦。

这套原则——比任何框架都重要。

这一季的 20 集——讲完了。

觉得有用——点赞、投币、收藏。

想看进阶——关注 + 留言。

我们下季见。

整个系列链接：

- [第 01 集](#) — Spring Boot 骨架
- [第 02 集](#) — JPA 实体
- [第 03 集](#) — JWT 鉴权
- [第 04 集](#) — Spring Security
- [第 05 集](#) — MinIO 对象存储
- [第 06 集](#) — 文档解析 Pipeline
- [第 07 集](#) — Embedding Client
- [第 08 集](#) — Elasticsearch 集成
- [第 09 集](#) — Kafka 异步化
- [第 10 集](#) — WebSocket 流式
- [第 11 集](#) — 多租户隔离
- [第 12 集](#) — 混合检索
- [第 13 集](#) — 限流配额
- [第 14 集](#) — 对话历史
- [第 15 集](#) — ReAct Agent
- [第 16 集](#) — 多 LLM 路由
- [第 17 集](#) — 充值支付
- [第 18 集](#) — 邀请码
- [第 19 集](#) — 消息反馈
- [第 20 集](#) — 可观测性

— 全书完 —

感谢您读完整本《派聪明 RAG 后端演进全解》。

本书配套代码仓库：<https://gitcode.com/javabetter/PaiSmart>

项目包名：`com.yizhaoqi.smartpai` | 教程版包名：`com.charles.easyrag`

视频脚本：`youtube_scripts/01-20` | 实战教程：`youtube_scripts/build-from-scratch/`

© 2026 Charles · 本书由 Charles 整理编纂，仅供学习交流